Contents

0.1. Kotlin Temelleri - Değişkenler ve Veri Tipleri 4
0.1.1. Değişken Tanımlama: `val` ve `var` Farkı 4
0.1.2. Temel Veri Tipleri 5
0.1.3. Kod Uygulaması: MainActivity.kt 5
0.1.4. Uygulama Notları ve Fiziksel Cihaz Takibi 6
0.2. Kotlin Temelleri - Değişken Kullanımı ve Metin Şablonları 6
0.2.1. `val` ve `var` Farkının Uygulama Üzerindeki Etkisi 6
0.2.2. 2. Metin Şablonları (String Templates) 7
0.2.3. Kod Uygulaması: MainActivity.kt 7
0.2.4. Ders Notları ve Teknik Detaylar 8
0.3. Kotlin Temelleri - Tip Belirleme ve Temel Matematiksel İşlemler 9
0.3.1. Tip Belirleme: Açık Belirtme ve Otomatik Algılama 9
0.3.2. Temel Aritmetik Operatörler 9
0.3.3. Kod Uygulaması: MainActivity.kt 9
0.3.4. Teknik Detaylar ve Uygulama Notları 11
0.4. Karşılaştırma ve Mantıksal Operatörler 11
0.4.1. 1. Karşılaştırma Operatörleri 11
0.4.2. Mantıksal Operatörler 11
0.4.3. Kod Uygulaması: MainActivity.kt 12
0.4.4. Teknik Detaylar ve Uygulama Notları 13
0.5. Kotlin Temelleri - Null Güvenliği (Null Safety) 13
0.5.1. 1. Null Kavramı Nedir? 13
0.5.2. 2. Güvenli Çağrı (Safe Call) ve Elvis Operatörü 14
0.5.3. Kod Uygulaması: MainActivity.kt 14
0.5.4. Teknik Detaylar ve Uygulama Notları 15
0.6. Fonksiyon Temelleri ve Parametre Kullanımı 16
0.6.1. Fonksiyon Tanımlama (Syntax) 16
0.6.2. Parametreli Fonksiyonlar ve Compose İlişkisi 16
0.6.3. Kod Uygulaması: MainActivity.kt 16
0.6.4. Teknik Detaylar ve Uygulama Notları 18
0.7. Kotlin Temelleri - Varsayılan Parametreler ve İsimlendirilmiş Argümanlar 18
0.7.1. 1. Varsayılan Parametreler (Default Arguments) 18
0.7.2. 2. İsimlendirilmiş Argümanlar (Named Arguments) 18
0.7.3. Kod Uygulaması: MainActivity.kt 19
0.7.4. 4. Teknik Detaylar ve Uygulama Notları 20
0.8. Lambda İfadeleri ve Süslü Parantez Mantığı 20
0.8.1. 1. Lambda İfadeleri Nedir? 21
0.8.2. Trailing Lambda (Sona Eklenen Lambda) Kuralı 21
0.8.3. Kod Uygulaması: MainActivity.kt 21
0.8.4. Teknik Detaylar ve Uygulama Notları 22
0.9. Kotlin Temellerinin Birleştirilmesi ve Genel Uygulama 23
0.9.1. 1. Mantıksal ve Yapısal Bütünleşme 23
0.9.2. Kod Uygulaması: MainActivity.kt 23
0.9.3. Teknik Detaylar ve Uygulama Notları 25
0.10. Karar Yapıları (if-else) ve Dinamik Arayüz 25

0.10.1. Karar Yapılarının Mantığı: `if` ve `else` 25
0.10.2. Kod Uygulaması: MainActivity.kt 26
0.10.3. Teknik Detaylar ve Uygulama Notları 27
0.11. Çoklu Karar Yapıları (`when`) ve Arayüz Durumları 28
0.11.1. `when` Yapısının Gerekliliği ve Mantığı 28
0.11.2. Jetpack Compose'da Durum Yönetimi (UI States) 28
0.11.3. Kod Uygulaması: MainActivity.kt 28
0.11.4. Teknik Detaylar ve Uygulama Notları 30
0.12. Aralıklar (Ranges) ve Gelişmiş Koşul Kontrolleri 31
0.12.1. Aralık (Range) Kavramı ve `in` Operatörü 31
0.12.2. `when` İçerisinde Aralık Kullanımı 31
0.12.3. Kod Uygulaması: MainActivity.kt 31
0.12.4. Teknik Detaylar ve Uygulama Notları 33
0.13. Döngüler ve Tekrarlayan Arayüz Elemanları 33
0.13.1. `repeat` Fonksiyonu: En Basit Tekrar 34
0.13.2. `for` Döngüsü ve Aralıklarla Kullanım 34
0.13.3. Kod Uygulaması: MainActivity.kt 34
0.13.4. Teknik Detaylar ve Uygulama Notları 36
0.14. Listeler (Collections) ve Veri Grupları Üzerinde Döngüler 36
0.14.1. Kotlin'de Liste Tanımlama: `listOf` 36
0.14.2. Listelerin Arayüzde Gösterimi 36
0.14.3. Kod Uygulaması: MainActivity.kt 37
0.14.4. Teknik Detaylar ve Uygulama Notları 38
0.15. Döngüler ve Karar Yapılarının Birlikte Kullanımı 39
0.15.1. Döngü İçinde Karar Verme (Conditional Rendering) 39
0.15.2. Liste ve Mantık Entegrasyonu 39
0.15.3. Kod Uygulaması: MainActivity.kt 39
0.15.4. 4. Teknik Detaylar ve Uygulama Notları 41
0.16. Koşullu Döngüler (while) ve Döngü Kontrolleri 42
0.16.1. `while` Döngüsü Mantığı 42
0.16.2. Döngü Kontrol Komutları 42
0.16.3. 3. Kod Uygulaması: MainActivity.kt 42
0.16.4. Teknik Detaylar ve Uygulama Notları 44
0.17. Veri Filtreleme ve Metin Arama Mantığı 44
0.17.1. Metin Filtreleme Fonksiyonları 45
0.17.2. Filtreleme Algoritmasının Çalışma Prensibi 45
0.17.3. Kod Uygulaması: MainActivity.kt 45
0.17.4. Teknik Detaylar ve Uygulama Notları 47
0.18. Kontrol Yapılarının Birleşimi ve Uygulama Mantığı 47
0.18.1. Mantıksal Yapıların Entegrasyonu 47
0.18.2. Kod Uygulaması: MainActivity.kt 48
0.18.3. Teknik Detaylar ve Uygulama Notları 50
0.19. Karar Yapıları ve Döngülerin Arayüz Kontrolündeki Rolü 50
0.19.1. Koşullu Arayüz Çizimi (Conditional Rendering) 50
0.19.2. Döngülerin Arayüz Tasarımındaki Gücü 51
0.19.3. Kod Uygulaması: MainActivity.kt 51
0.19.4. Teknik Notlar ve Uygulama Detayları 53

0.20. `when` Yapısı ile Ekran Durum ... 53
0.20.1. `when` İfadesinin Compose İçindeki Mantığı ... 53
0.20.2. Durum Odaklı Tasarım (State-Driven Design) ... 53
0.20.3. Kod Uygulaması: MainActivity.kt ... 53
0.20.4. Teknik Ayrıntılar ve Uygulama Notları ... 55
0.21. Döngüler İçinde Karar Yapıları ve Veri Filtreleme ... 56
0.21.1. Döngü ve Koşul Kombinasyonu ... 56
0.21.2. Liste İşleme Mantığı ... 56
0.21.3. Kod Uygulaması: MainActivity.kt ... 56
0.21.4. 4. Teknik Notlar ve Uygulama Detayları ... 58
0.22. Şartlı Döngüler (`while`) ve Döngü Kontrol Komutları ... 59
0.22.1. `while` Döngüsü: Şart Bağlı Tekrar ... 59
0.22.2. Döngü Kesiciler: `break` ve `continue` ... 59
0.22.3. Kod Uygulaması: MainActivity.kt ... 59
0.22.4. Teknik Notlar ve Uygulama Detayları ... 61
0.23. Kotlin Null Güvenliği (Null Safety) (20:00 - 25:00) ... 62
0.23.1. 1. Neden Null Güvenliği? ... 62
0.23.2. 2. Null Verilerle Güvenli Çalışma ... 62
0.23.3. 3. Kod Uygulaması: MainActivity.kt ... 62
0.23.4. 4. Teknik Notlar ve Uygulama Detayları ... 64
0.23.5. 1. Fonksiyon Tanımlama ve Yapısı ... 65
0.23.6. 2. Fonksiyonların Compose İçindeki Önemi ... 65
0.23.7. 3. Kod Uygulaması: MainActivity.kt ... 65
0.23.8. 4. Teknik Notlar ve Uygulama Detayları ... 67
0.23.9. 1. Lambda İfadeleri Nedir? ... 68
0.23.10. 2. Trailing Lambda (Sona Eklenen Lambda) ... 68
0.23.11. 3. Kod Uygulaması: MainActivity.kt ... 68
0.23.12. 4. Teknik Notlar ve Uygulama Detayları ... 70
0.23.13. 1. Data Class (Veri Sınıfı) Mantığı ... 71
0.23.14. 2. Nesne Listesi ve Arayüz Entegrasyonu ... 71
0.23.15. 3. Kod Uygulaması: MainActivity.kt ... 71
0.23.16. 4. Teknik Notlar ve Uygulama Detayları ... 73
0.23.17. 1. Uygulama Mimarisi ve Veri Akışı ... 74
0.23.18. 2. Kod Uygulaması: MainActivity.kt ... 74
0.23.19. 3. Uygulama Notları ve Teknik Detaylar ... 77
0.23.20. 1. Kotlin ve Compose Arasındaki Köprü ... 78
0.23.21. 2. Fonksiyonel Bileşen Yapısı ... 78
0.23.22. 3. Kod Uygulaması: MainActivity.kt ... 78
0.23.23. 4. Teknik Notlar ve Uygulama Detayları ... 81
0.23.24. 1. Koşullu Çizim ve Görünürlük Mantığı ... 81
0.23.25. 2. Değişkenler ile Stil Yönetimi ... 82
0.23.26. 3. Kod Uygulaması: MainActivity.kt ... 82
0.23.27. 4. Teknik Ayrıntılar ve Uygulama Notları ... 84
0.23.28. 1. “when” Yapısının Kontrol Mekanizması ... 85
0.23.29. 2. Jetpack Compose ve Durum Dallanması ... 85
0.23.30. 3. Kod Uygulaması: MainActivity.kt ... 85
0.23.31. 4. Teknik Ayrıntılar ve Uygulama Notları ... 87

0.23.32. 1. Veri Gruplama: Data Class ve List Entegrasyonu 88
0.23.33. 2. Yinelemeli Arayüz (Iterative UI) Mantığı 88
0.23.34. 3. Kod Uygulaması: MainActivity.kt 88
0.23.35. 4. Teknik Notlar ve Uygulama Detayları 91
0.23.36. 1. Döngü Akışını Yönlendirmek 91
0.23.37. 2. Compose İçerisinde Mantıksal Filtreleme 92
0.23.38. 3. Kod Uygulaması: MainActivity.kt 92
0.23.39. 4. Teknik Notlar ve Uygulama Detayları 94
0.23.40. 1. Null (Boşluk) Kavramı ve Riskleri 95
0.23.41. 2. Null Veri Yönetim Operatörleri 95
0.23.42. 3. Kod Uygulaması: MainActivity.kt 95
0.23.43. 4. Teknik Notlar ve Uygulama Detayları 98
0.23.44. 1. Operatörlerin Çalışma Mantığı 98
0.23.45. 2. Arayüzde Fonksiyonel Yaklaşım 99
0.23.46. 3. Kod Uygulaması: MainActivity.kt 99
0.23.47. 4. Teknik Notlar ve Uygulama Detayları 101
0.23.48. 1. Veri Toplulaştırma Fonksiyonları 102
0.23.49. 2. Sıralama (Sorting) Mekanizması 102
0.23.50. 3. Kod Uygulaması: MainActivity.kt 102
0.23.51. 4. Teknik Notlar ve Uygulama Detayları 105
0.23.52. 1. Mantıksal Akış Şeması: Veriden Görsele 105
0.23.53. 2. Kod Uygulaması: MainActivity.kt 106
0.23.54. 3. Kritik Teknik Analiz 108
0.23.55. 4. Uygulama ve Test Notları 109

0.1. Kotlin Temelleri - Değişkenler ve Veri Tipleri

Bu dersin amacı, Jetpack Compose ile bir arayüz oluştururken ekranda göstereceğimiz verilerin (metinler, sayılar, durumlar) kod tarafında nasıl tutulduğunu ve yönetildiğini anlamaktır. Android Studio üzerinde bir proje çalışırken, her bilgi bir değişken içerisinde saklanır.

0.1.1. Değişken Tanımlama: `val` ve `var` Farkı

Kotlin dilinde veri depolamak için iki temel anahtar kelime kullanılır. Bu ayrım, uygulamanın performanslı ve hatasız çalışması için kritiktir.

- `val` **(Value) :** "Salt okunur" (read-only) değişkendir. Bir kez değer atandıktan sonra değiştirilemez. Uygulama içinde sabit kalacak veriler (örneğin: bir kullanıcı adı, bir butonun üzerindeki sabit metin) için kullanılır.
- `var` **(Variable) :** Değeri sonradan değiştirilebilen değişkendir. Uygulama çalışırken güncellenmesi gereken veriler (örneğin: bir sayaç değeri, kullanıcının girdiği şifre alanı) için tercih edilir.

> 🔥 **Önemli Kural**
>
> Jetpack Compose mimarisinde, verilerin gereksiz yere değiştirilmesini önlemek amacıyla, aksi gerekmedikçe her zaman `val` kullanılması önerilir.

0.1.2. Temel Veri Tipleri

Android uygulamalarında en sık karşılaşacağımız veri tipleri şunlardır:

- **String :** Metinsel ifadeleri tutar. Çift tırnak `" "` içerisinde yazılır. (Örn: `"Giriş Yap"`)
- **Int :** Tam sayıları ifade eder. (Örn: `25`, `-5`)
- **Double :** Ondalıklı sayıları ifade eder. Nokta `.` ile ayrılır. (Örn: `15.5`)
- **Boolean :** Sadece iki değer alabilir: `true` (doğru) veya `false` (yanlış). Mantıksal kontrollerde kullanılır.

0.1.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod yapısı, bu hafta öğrendiğimiz değişkenlerin bir Jetpack Compose projesinde nasıl yer aldığını göstermektedir. Bu kodda, tanımladığımız değişkenleri ekranın ortasında bir metin olarak yazdırıyoruz.

```java
package com.example.ilkprojem

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import com.example.ilkprojem.ui.theme.IlkProjemTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            IlkProjemTheme {
                // Uygulamanın arka plan rengini sistem temasına göre ayarlar
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    DegiskenGosterimi()
```

```
                }
            }
        }
    }
}

@Composable
fun DegiskenGosterimi() {
    // 1. Değişken Tanımlamaları
    val okulAdi: String = "Amasya Üniversitesi" // Değiştirilemez metin
    val dersSaati: Int = 45                     // Tam sayı
    var ogrenciSayisi: Int = 20                 // Değiştirilebilir tam sayı
    val dersAktifMi: Boolean = true             // Mantıksal veri

    // 2. Arayüz Oluşturma (UI)
    Column {
        Text(text = "Kurum: $okulAdi")
        Text(text = "Ders Süresi: $dersSaati dakika")
        Text(text = "Mevcut Öğrenci: $ogrenciSayisi")
        Text(text = "Ders Durumu: $dersAktifMi")
    }
}
```

Listing 1 : `MainActivity.kt` *dosyası*

0.1.4. Uygulama Notları ve Fiziksel Cihaz Takibi

Kod içerisinde yer alan `$okulAdi` gibi ifadeler, değişkenin değerini metin içerisine yerleştirmemizi sağlar (String Interpolation).

Öğrenciler, cihazlarını USB ile bağladıktan sonra Android Studio'nun alt panelinde bulunan **Logcat** sekmesini açmalıdır. Kodda bir hata oluştuğunda veya bir değişkenin değerini takip etmek istediğimizde, emülatör yerine doğrudan kendi telefonlarındaki bu çıktıları inceleyeceklerdir.

0.2. Kotlin Temelleri - Değişken Kullanımı ve Metin Şablonları

Dersin bu bölümünde, tanımladığımız değişkenleri kullanıcı arayüzünde (UI) daha dinamik bir şekilde nasıl birleştireceğimizi ve `val` ile `var` arasındaki farkın uygulama üzerindeki etkilerini inceleyeceğiz.Özellikle Jetpack Compose içerisinde metinleri ekrana basarken kullandığımız "String Template" (Metin Şablonu) yapısı üzerinde duracağız.

0.2.1. `val` ve `var` Farkının Uygulama Üzerindeki Etkisi

Kod yazarken bir verinin değişip değişmeyeceğine karar vermek, bellek yönetimi ve uygulama güvenliği açısından önemlidir.

- **Hata Senaryosu :** Eğer `val` ile tanımladığınız bir değişkene (örneğin bir öğrenci numarası) kodun ilerleyen aşamalarında yeni bir değer atamaya çalışırsanız, Android

Studio size “Val cannot be reassigned” (Val değerine yeniden atama yapılamaz) hatası verecektir. Bu, verinin yanlışlıkla değiştirilmesini önleyen bir güvenlik mekanizmasıdır.

- **Değişim İhtiyacı :** Eğer bir skor tabelası veya sayaç gibi sürekli güncellenen bir veri ile çalışıyorsanız, bu değişkeni mutlaka `var` ile tanımlamanız gerekir.

0.2.2. 2. Metin Şablonları (String Templates)

Değişkenleri ekranda gösterirken sadece ham veriyi değil, bu veriyi açıklayan metinleri de yan yana getirmemiz gerekir. Kotlin’de bu işlem için artı `+` operatörü yerine çok daha pratik olan `$` işareti kullanılır.

- **Kullanımı :** Çift tırnak içerisinde bir değişkenin başına `$` işareti koyduğunuzda, Kotlin o noktaya değişkenin adını değil, içinde sakladığı değeri yazar.
- **İşlem Yapma :** Eğer değişken üzerinde bir işlem (toplama, çıkarma vb.) yapıp sonucu metin içinde göstermek istiyorsanız `${ }` yapısı kullanılır. Örneğin: `${sayi1 + sayi2}`.

0.2.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki dosya, değişkenlerin metin şablonları ile kullanımını ve basit aritmetik işlemlerin arayüze nasıl yansıtılacağını içeren tam koddur.

```java
package com.example.jetpackdersi

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            // Uygulama başladığında ekranda görünecek olan ana fonksiyon
            DegiskenKullanimiEkranı()
        }
    }
}

@Composable
fun DegiskenKullanimiEkranı() {
    // 1. Değişkenlerin Tanımlanması
    val dersAdi: String = "Jetpack Compose"
```

```
    val mevcutHafta: Int = 2
    val toplamHafta: Int = 14

    // Basit bir hesaplama değişkeni
    val kalanHafta: Int = toplamHafta - mevcutHafta

    // 2. Arayüz Elemanları (Column dikey olarak alt alta dizer)
    Column(modifier = Modifier.padding(16.dp)) {

        // Doğrudan değişken kullanımı
        Text(text = "Dersin Adı: $dersAdi")

        // String Template ile değişkeni metin içine yerleştirme
        Text(text = "Şu an $mevcutHafta. haftadayız.")

        // Süslü parantez kullanarak metin içinde hesaplama yapma
        Text(text = "Müfredatın bitmesine ${toplamHafta - mevcutHafta} hafta kaldı.")

        // Önceden hesaplanmış değişkeni kullanma
        Text(text = "Hesaplanmış Kalan Süre: $kalanHafta hafta.")

        // Boolean (Mantıksal) değer gösterimi
        val dersBittiMi: Boolean = false
        Text(text = "Ders tamamlandı mı? Answer: $dersBittiMi")
    }
}
```

Listing 2 : `MainActivity.kt` *dosyası*

0.2.4. Ders Notları ve Teknik Detaylar

- **Noktalı Virgül:** Kotlin dilinde satır sonlarına noktalı virgül `;` koyma zorunluluğu yoktur.
- **Tip Belirleme:** `val dersAdi: String = "..."` yazımında `: String` kısmı isteğe bağlıdır. Kotlin, atanan değerden değişkenin türünü otomatik olarak anlayabilir (Type Inference). Ancak yeni öğrenenler için tipin açıkça yazılması kodun okunabilirliğini artırır.
- **Fiziksel Cihaz Testi:** Yazılan bu kodun sonuçlarını görmek için telefonunuzun USB hata ayıklama modunun açık olduğundan emin olun ve "Run" butonuna basarak uygulamayı telefonunuza yükleyin. Ekrandaki metinlerin değişken değerlerine göre otomatik şekillendiğini göreceksiniz.

Bu dersin bir sonraki aşaması olan **"Temel Aritmetik Operatörler ve Mantıksal Karşılaştırmalar"** bölümüne geçmek için hazır mısınız?

0.3. Kotlin Temelleri - Tip Belirleme ve Temel Matematiksel İşlemler

Dersin bu bölümünde, Kotlin'in değişken türlerini nasıl algıladığı (Type Inference) ve uygulama içinde temel aritmetik hesaplamaların nasıl yapılıp ekrana yansıtılacağı ele alınacaktır.

0.3.1. Tip Belirleme: Açık Belirtme ve Otomatik Algılama

Kotlin dilinde bir değişken tanımlarken türünü (String, Int vb.) belirtmek her zaman zorunlu değildir.

- **Açık Belirtme (Explicit Typing):** Değişken isminden sonra iki nokta üst üste koyarak türün yazılmasıdır. Okunabilirliği artırmak ve hataları önlemek için yeni başlayanlar için bu yöntem tavsiye edilir.
  - *Örnek:* `val sayi: Int = 10`
- **Tip Çıkarımı (Type Inference):** Kotlin, değişkene atanan değere bakarak türünü otomatik olarak anlar. Eğer bir değişkene `"Amasya"` değeri atanmışsa, Kotlin bunun bir `String` olduğunu bilir.
  - *Örnek:* `val sehir = "Amasya"` (Kotlin bunu otomatik olarak String kabul eder).

0.3.2. Temel Aritmetik Operatörler

Uygulama içinde sayılarla işlem yaparken standart matematiksel operatörler kullanılır:

- `+` (Toplama)
- `-` (Çıkarma)
- `*` (Çarpma)
- `/` (Bölme)

> 🔥 **Önemli Not**
>
> İki tam sayı (Int) birbirine bölündüğünde sonuç yine bir tam sayıdır. Örneğin `5 / 2` işleminin sonucu `2.5` değil, `2` olarak döner. Ondalıklı sonuç almak için değişkenlerden en az birinin `Double` olarak tanımlanması gerekir.

0.3.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod örneğinde, hem tip çıkarımı hem de açık belirtme kullanılmış; ayrıca temel bir hesaplama yapılarak sonuç ekrana yansıtılmıştır.

```java
package com.example.dersprojesi

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding
```

```
8   import androidx.compose.material3.Text
9   import androidx.compose.runtime.Composable
10  import androidx.compose.ui.Modifier
11  import androidx.compose.ui.unit.dp
12
13  class MainActivity : ComponentActivity() {
14      override fun onCreate(savedInstanceState: Bundle?) {
15          super.onCreate(savedInstanceState)
16          setContent {
17              // Hesaplama fonksiyonunu çağırıyoruz
18              MatematikselIslemlerEkranı()
19          }
20      }
21  }
22
23  @Composable
24  fun MatematikselIslemlerEkranı() {
25      // 1. Değişken Tanımlamaları
26      val vizeNotu: Int = 60           // Açık belirtme (Explicit)
27      val finalNotu = 80               // Tip çıkarımı (Inference - Kotlin Int olduğunu anladı)
28
29      // Ortalama hesabı (Vize %40, Final %60)
30      val vizeEtkisi = vizeNotu * 0.4
31      val finalEtkisi = finalNotu * 0.6
32      val ortalama: Double = vizeEtkisi + finalEtkisi
33
34      // 2. Arayüz Tasarımı
35      Column(modifier = Modifier.padding(20.dp)) {
36          Text(text = "Ders Notu Hesaplama")
37          Text(text = "-------------------------")
38
39          Text(text = "Vize Notunuz: $vizeNotu")
40          Text(text = "Final Notunuz: $finalNotu")
41
42          // İşlem sonucunu ekrana yazdırma
43          Text(text = "Yıl Sonu Ortalaması: $ortalama")
44
45          // Mantıksal karşılaştırma örneği
46          val gectiMi: Boolean = ortalama >= 50
47          Text(text = "Dersten Geçme Durumu: $gectiMi")
48
49          // Bölme işlemi uyarısı için örnek
50          val ornekBolme = 5 / 2
51          Text(text = "Tam Sayı Bölme Örneği (5/2): $ornekBolme")
52      }
```

```
}
```

Listing 3 : `MainActivity.kt` *dosyası*

0.3.4. Teknik Detaylar ve Uygulama Notları

- **Ondalıklı Sayılar :** Kodda görülen `0.4` ve `0.6` gibi ifadeler Kotlin tarafından otomatik olarak `Double` türünde algılanır. Bir `Int` ile `Double` işleme girdiğinde sonuç her zaman daha kapsamlı olan `Double` türünde çıkar.
- **Büyük-Küçük Harf Duyarlılığı :** Veri tipleri yazılırken (`String`, `Int`, `Double`, `Boolean`) her zaman büyük harfle başlamalıdır. Küçük harfle başlaması (örneğin `int`) hata almanıza neden olur.
- **Cihaz Üzerinde Gözlem :** Öğrenciler, yazdıkları kodda `vizeNotu` veya `finalNotu` değerlerini değiştirip "Run" tuşuna bastıklarında, telefon ekranındaki "Ortalama" ve "Geçme Durumu" (true/false) bilgilerinin anlık olarak nasıl güncellendiğini takip etmelidir.

0.4. Karşılaştırma ve Mantıksal Operatörler

Dersin bu bölümünde, uygulamanın karar verme mekanizmalarının temelini oluşturan karşılaştırma operatörleri ve mantıksal operatörler ele alınacaktır. Bu operatörler, ekrandaki bir butonun aktif olup olmayacağı veya bir metnin renginin ne olacağı gibi durumlarda (Logic) kullanılır.

0.4.1. 1. Karşılaştırma Operatörleri

İki veriyi birbiriyle kıyaslamak için kullanılır. Bu işlemlerin sonucu her zaman bir **Boolean** (`true` veya `false`) değer üretir.

- `==` : Eşittir (İki değer birbirine eşit mi?)
- `≠` : Eşit Değildir (İki değer birbirinden farklı mı?)
- `>` : Büyüktür
- `<` : Küçüktür
- `⩾` : Büyük veya Eşittir
- `⩽` : Küçük veya Eşittir

0.4.2. Mantıksal Operatörler

Birden fazla karşılaştırma durumunu birleştirmek için kullanılır.

- **VE Operatörü (`&&`):** Verilen tüm koşulların `true` olması durumunda sonuç `true` olur. (Örn: Hem notu 50'den büyük mü **VE** hem de devamsızlığı sınırın altında mı?)
- **VEYA Operatörü (`||`):** Koşullardan en az birinin `true` olması sonucun `true` olması için yeterlidir.
- **DEĞİL Operatörü (`!`):** Var olan durumu tersine çevirir. `true` ise `false`, `false` ise `true` yapar.

0.4.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kodda, bir öğrencinin hem sınav notu hem de devamsızlık durumu kontrol edilerek “Sertifika Alabilir mi?” sorgusu yapılmaktadır.

```java
package com.example.mantiksalislem

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            // Mantıksal kontrol ekranını başlatıyoruz
            SertifikaKontrolEkranı()
        }
    }
}

@Composable
fun SertifikaKontrolEkranı() {
    // 1. Veri Tanımlamaları
    val sinavNotu = 75
    val devamsizlikGunSayisi = 4
    val katilimBelgesiVarMi = true

    // 2. Mantıksal Koşulların Hesaplanması
    // Örnek: Not 70'ten büyük VE devamsızlık 5'ten az olmalı
    val basariDurumu: Boolean = (sinavNotu >= 70) && (devamsizlikGunSayisi < 5)

    // Örnek: Başarılı OLMALI veya Katılım Belgesi bulunmalı
    val sertifikaAlabilirMi: Boolean = basariDurumu || katilimBelgesiVarMi

    // 3. Arayüz Tasarımı
    Column(modifier = Modifier.padding(20.dp)) {
        Text(text = "Öğrenci Durum Paneli")
        Text(text = "-------------------------")

        Text(text = "Sınav Notu: $sinavNotu")
```

```
        Text(text = "Devamsızlık: $devamsizlikGunSayisi Gün")

        Text(text = "Akademik Başarı Şartı Sağlandı mı?: $basariDurumu")

        Text(text = "Katılım Belgesi Mevcut mu?: $katilimBelgesiVarMi")

        Text(text = "-------------------------")

        // Final Sonucu
        Text(text = "Sertifika Almaya Hak Kazandı mı?: $sertifikaAlabilirMi")

        // Tersine çevirme örneği (!)
        val engelVarMi = !sertifikaAlabilirMi
        Text(text = "Mezuniyetine Engel Var mı?: $engelVarMi")
    }
}
```

Listing 4 : `MainActivity.kt` *dosyası*

0.4.4. Teknik Detaylar ve Uygulama Notları

- **İşlem Önceliği:** Matematikteki işlem önceliği gibi, parantez içindeki mantıksal işlemler her zaman önce yapılır. Karmaşık sorgularda okunabilirliği artırmak için parantez kullanımı (`(koşul1) && (koşul2)`) zorunlu olmasa da önerilir.
- **Boolean Mantığı:** Jetpack Compose'da ilerleyen haftalarda, bir butonun rengini kırmızı veya yeşil yapmak için bu `true/false` değerlerini kullanacağız. Örneğin; `if (basariDurumu) Green else Red`.
- **Cihaz Testi:** Öğrenciler, kendi telefonlarında uygulamayı çalıştırdıktan sonra `sinavNotu` değerini 60 yapıp `basariDurumu` değişkeninin nasıl `false` değerine döndüğünü ve bunun nihai sertifika durumunu nasıl etkilediğini canlı olarak gözlemlemelidir.

0.5. Kotlin Temelleri - Null Güvenliği (Null Safety)

Dersin bu bölümünde, modern programlama dillerinin en kritik konularından biri olan ve uygulamaların çökmesini (crash) önleyen **Null Güvenliği** konusu işlenecektir. Kotlin, değişkenlerin "boş" (null) olması durumunu kod yazım aşamasında denetleyerek hataları minimize eder.

0.5.1. 1. Null Kavramı Nedir?

`null`, bir değişkenin henüz bir değere sahip olmadığını, yani bellekte hiçbir veriyi işaret etmediğini ifade eder. Geleneksel dillerde (Java gibi), "boş" bir değişken üzerinde işlem yapmak uygulamanın aniden kapanmasına (NullPointerException) yol açar. Kotlin ise bu durumu ikiye ayırır:

- **Null Olamaz Tipler (Non-nullable):** Varsayılan olarak tüm değişkenler değer almak zorundadır.
  - `val isim: String = null` -> **HATA** (Kod çalışmaz).
- **Null Olabilir Tipler (Nullable):** Bir değişkenin boş olabileceğini belirtmek için tipin yanına soru işareti `?` konur.
  - `val isim: String? = null` -> **GEÇERLİ**.

0.5.2. 2. Güvenli Çağrı (Safe Call) ve Elvis Operatörü

Boş olabilen bir değişkenle çalışırken sistemi korumak için iki temel araç kullanılır:

1. **Güvenli Çağrı (`?.`):** Değişken eğer `null` ise işlemi yapmaz ve hata vermek yerine kendisi de null döner.
   - `isim?.length` ⟹ İsim `null` ise uygulama çökmez, sonuç `null` olur.
2. **Elvis Operatörü (`?:`):** Eğer değişken `null` ise, onun yerine varsayılan bir değer atanmasını sağlar.
   - `val uzunluk = isim?.length ?: 0` ⟹ İsim `null` ise uzunluğu `0` kabul et.

0.5.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kodda, internetten veya bir veri tabanından "boş" gelebilecek bir kullanıcı bilgisinin, uygulama ekranında hata vermeden nasıl yönetileceği simüle edilmiştir.

```java
package com.example.nullguvenligi

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            // Null güvenlik kontrol ekranı
            NullKontrolEkranı()
        }
    }
}

@Composable
```

```
fun NullKontrolEkranı() {
    // 1. Değişken Tanımlamaları
    // Soru işareti (?) bu değişkenin boş (null) olabileceğini belirtir.
    val kullaniciNotu: String? = null
    val kullaniciAdi: String? = "Hasan"

    // 2. Güvenli İşlemler
    // Eğer not null ise "Bilgi Yok" yazdır, değilse notun kendisini yazdır.
    val gosterilecekNot = kullaniciNotu ?: "Henüz not girilmedi"

    // Güvenli çağrı örneği: Adın karakter sayısını al, null ise 0 döndür.
    val adUzunlugu = kullaniciAdi?.length ?: 0

    // 3. Arayüz Tasarımı
    Column(modifier = Modifier.padding(20.dp)) {
        Text(text = "Kullanıcı Bilgi Sistemi")
        Text(text = "-------------------------")

        // Doğrudan nullable değişkeni yazdırmak "null" metnini basar
        Text(text = "Ham Veri (Not): $kullaniciNotu")

        // Elvis operatörü ile güvenli gösterim
        Text(text = "Güvenli Veri: $gosterilecekNot")

        Text(text = "-------------------------")

        Text(text = "Kullanıcı Adı: $kullaniciAdi")
        Text(text = "Adın Karakter Sayısı: $adUzunlugu")

        // Kompleks kullanım: Karakter sayısına göre bir kontrol
        val uzunMu = (kullaniciAdi?.length ?: 0) > 10
        Text(text = "Adı 10 karakterden uzun mu?: $uzunMu")
    }
}
```

Listing 5 : `MainActivity.kt` *dosyası*

0.5.4. Teknik Detaylar ve Uygulama Notları

- **Neden Önemli?:** Android uygulamalarında en sık karşılaşılan çökme sebebi boş verilerdir. Kotlin bu yapısıyla "Milyar Dolarlık Hata" olarak bilinen `null` referans hatalarını daha kod yazılırken engeller.
- **Arayüz Davranışı:** Jetpack Compose içerisinde bir `Text` bileşenine doğrudan `null` bir değer gönderirseniz, ekranda "null" kelimesi yazılır. Bu durum kullanıcı deneyimi açısından kötüdür. Bu yüzden her zaman `?:` (Elvis) operatörü ile anlamlı bir metin ("Yükleniyor...", "Bilgi Yok" vb.) atanmalıdır.

- **Cihaz Testi:** Öğrenciler, `kullaniciNotu` değişkenine bir metin (örneğin `"Başarılı"`) atayıp uygulamayı tekrar çalıştırdıklarında, ekrandaki "Henüz not girilmedi" yazısının nasıl güncellendiğini fiziksel cihazları üzerinden gözlemlemelidir.

0.6. Fonksiyon Temelleri ve Parametre Kullanımı

Dersin bu bölümünde, Kotlin programlama dilinin en temel yapı taşı olan ve Jetpack Compose arayüzlerinin de üzerine inşa edildiği **Fonksiyonlar** konusu ele alınacaktır. Fonksiyonlar, belirli bir işi yapmak için gruplandırılmış kod bloklarıdır ve kodun tekrar kullanılabilirliğini sağlar.

0.6.1. Fonksiyon Tanımlama (Syntax)

Kotlin'de bir fonksiyon `fun` anahtar kelimesi ile başlar. Fonksiyonların temel yapısı şu şekildedir:

- **fun:** Fonksiyon tanımlayıcı.
- **Fonksiyon Adı:** Küçük harfle başlar (camelCase yapısı).
- **Parametreler:** Fonksiyonun çalışması için dışarıdan ihtiyaç duyduğu veriler (parantez içinde).
- **Geri Dönüş Tipi:** Fonksiyonun işlem sonunda ürettiği verinin türü.
- **Gövde:** Süslü parantezler `{ }` içindeki kod alanı.

0.6.2. Parametreli Fonksiyonlar ve Compose İlişkisi

Jetpack Compose'da yazdığımız her `@Composable` yapısı aslında birer fonksiyondur. Bu fonksiyonlara parametre göndererek arayüzün dinamik olmasını sağlarız. Örneğin, bir butonun üzerindeki yazıyı parametre olarak gönderdiğimizde, aynı fonksiyonu kullanarak farklı metinlere sahip onlarca buton oluşturabiliriz.

0.6.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod örneğinde, hem matematiksel hesaplama yapan normal bir fonksiyon (helper) hem de bu sonucu ekranda gösteren bir Composable fonksiyon bir arada kullanılmıştır.

```java
package com.example.fonksiyontemelleri

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

```

```
13 class MainActivity : ComponentActivity() {
14     override fun onCreate(savedInstanceState: Bundle?) {
15         super.onCreate(savedInstanceState)
16         setContent {
17             // Ana fonksiyonu çağırırken değer gönderiyoruz
18             AnaEkran(ogrenciAdi = "Hasan", ogrenciNotu = 85)
19         }
20     }
21 }
22
23 // 1. Parametre Alan Bir Composable Fonksiyon
24 @Composable
25 fun AnaEkran(ogrenciAdi: String, ogrenciNotu: Int) {
26     // 2. Hesaplama yapan yardımcı fonksiyonun çağrılması
27     val sonucMesaji = notDurumuBelirle(ogrenciNotu)
28
29     Column(modifier = Modifier.padding(20.dp)) {
30         Text(text = "Öğrenci Bilgi Paneli")
31         Text(text = "-------------------------")
32
33         // Parametreden gelen veriyi kullanma
34         Text(text = "Sayın $ogrenciAdi,")
35         Text(text = "Sınav Notunuz: $ogrenciNotu")
36
37         Text(text = "-------------------------")
38
39         // Yardımcı fonksiyondan dönen sonucu kullanma
40         Text(text = "Değerlendirme: $sonucMesaji")
41
42         // Aynı fonksiyonu farklı değerle tekrar çağırma (Reusability)
43         Selamla(isim = ogrenciAdi)
44     }
45 }
46
47 // 3. Geriye Değer Döndüren (Return) Normal Bir Fonksiyon
48 fun notDurumuBelirle(not: Int): String {
49     return if (not >= 50) {
50         "Başarılı, dersi geçtiniz."
51     } else {
52         "Başarısız, tekrar denemelisiniz."
53     }
54 }
55
56 // 4. Basit Bir Selamlama Fonksiyonu
57 @Composable
58 fun Selamla(isim: String) {
```

```
    Text(text = "Sisteme hoş geldin, $isim!")
}
```

Listing 6 : `MainActivity.kt` *dosyası*

0.6.4. Teknik Detaylar ve Uygulama Notları

- **Birim Fonksiyonlar (Unit):** Eğer bir fonksiyon geriye herhangi bir veri döndürmüyorsa (sadece bir iş yapıyorsa), geri dönüş tipi belirtilmez. Kotlin bunu otomatik olarak `Unit` kabul eder.
- **Tek Satırlı Fonksiyonlar:** Eğer fonksiyon sadece tek bir işlem yapıp sonuç döndürüyorsa, süslü parantez yerine eşittir `=` kullanılarak daha kısa yazılabilir.
  - *Örnek:* `fun kareAl(sayi: Int) = sayi * sayi`
- **Okunabilirlik:** Parametrelere isim vermek (`ogrenciAdi`, `ogrenciNotu` gibi), fonksiyonun ne iş yaptığını anlamayı kolaylaştırır. Karmaşık projelerde bu isimlendirmeler hayati önem taşır.
- **Cihaz Testi:** Öğrenciler, `MainActivity` içindeki `AnaEkran` çağrısındaki parametreleri (isim ve not) değiştirerek telefon ekranındaki çıktının nasıl anlık olarak farklılaştığını gözlemlemelidir. Bu, fonksiyonların "tek bir kalıp ile farklı sonuçlar üretme" gücünü anlamalarını sağlayacaktır.

0.7. Kotlin Temelleri - Varsayılan Parametreler ve İsimlendirilmiş Argümanlar

Dersin bu bölümünde, Kotlin'in fonksiyon kullanımını esnek hale getiren ve Jetpack Compose kütüphanesinin temel çalışma prensibi olan **Varsayılan Parametreler (Default Arguments)** ve **İsimlendirilmiş Argümanlar (Named Arguments)** konuları işlenecektir.

0.7.1. 1. Varsayılan Parametreler (Default Arguments)

Bir fonksiyon tanımlanırken, bazı parametrelere başlangıçta bir değer atanabilir. Eğer fonksiyon çağrılırken bu parametre için yeni bir değer gönderilmezse, önceden tanımlanan "varsayılan" değer kullanılır.

- **Avantajı:** Fonksiyonu çağıran kişi, her seferinde tüm detayları belirtmek zorunda kalmaz. Sadece değiştirmek istediği değerleri gönderir.
- **Compose Örneği:** Bir `Text` bileşeni yüzlerce özelliğe (renk, font, hizalama vb.) sahiptir. Ancak biz sadece metni yazarız, diğer tüm özellikler varsayılan değerleriyle (siyah renk, standart font) otomatik olarak gelir.

0.7.2. 2. İsimlendirilmiş Argümanlar (Named Arguments)

Fonksiyonların parametre sayısı arttığında, hangi değerin hangi parametreye ait olduğunu karıştırmak kolaylaşır. Kotlin, fonksiyonu çağırırken parametre isimlerini açıkça belirtmemize izin verir.

- **Sıralama Özgürlüğü:** İsimlendirilmiş argüman kullanıldığında, parametrelerin sırasına uymak zorunda kalınmaz.
- **Okunabilirlik:** Kodun ne yaptığı, dışarıdan bakan biri için çok daha net anlaşılır. `(true, false, 15)` yerine `(aktifMi = true, hataVarMi = false, sure = 15)` yazmak hatayı minimize eder.

0.7.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod, özelleştirilebilir bir "Bilgi Kartı" fonksiyonu üzerinden varsayılan ve isimlendirilmiş argümanların kullanımını göstermektedir.

```java
package com.example.parametreler

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Column {
                // 1. Kullanım: Sadece zorunlu parametreyi gönderiyoruz (Varsayılan değerler devreye girer)
                MesajGoster(mesaj = "Sisteme giriş yapıldı.")

                // 2. Kullanım: Bazı varsayılan değerleri değiştiriyoruz
                MesajGoster(mesaj = "Hata oluştu!", gonderen = "Sistem")

                // 3. Kullanım: İsimlendirilmiş argümanlar sayesinde sıra gözetmeksizin tüm değerleri veriyoruz
                MesajGoster(
                    onemliMi = true,
                    gonderen = "Admin",
                    mesaj = "Yarın tatil duyurusu."
                )
            }
        }
    }
}
```

```

/**
 * @param mesaj Görüntülenecek ana metin (Zorunlu)
 * @param gonderen Mesajı gönderen kişi (Varsayılan: "Bilinmiyor")
 * @param onemliMi Mesajın önemi (Varsayılan: false)
 */
@Composable
fun MesajGoster(
    mesaj: String,
    gonderen: String = "Bilinmiyor",
    onemliMi: Boolean = false
) {
    Column(modifier = Modifier.padding(16.dp)) {
        Text(text = "Mesaj: $mesaj")
        Text(text = "Gönderen: $gonderen")

        // Mantıksal değere göre ek bilgi yazdıralım
        if (onemliMi) {
            Text(text = "DİKKAT: Bu çok önemli bir mesajdır!")
        }
        Text(text = "-------------------------")
    }
}
```

Listing 7 : `MainActivity.kt` *dosyası*

0.7.4. 4. Teknik Detaylar ve Uygulama Notları

- **Parametre Sıralaması:** Fonksiyon tanımlanırken, varsayılan değeri olmayan (zorunlu) parametrelerin en başa yazılması iyi bir programlama pratiğidir. Bu sayede isimlendirme yapmadan da temel çağrılar hızlıca gerçekleştirilebilir.
- **Compose Uyumu:** Jetpack Compose kütüphanesindeki neredeyse tüm arayüz bileşenleri (Button, Image, Row vb.) bu yapıyı kullanır. Öğrencilerin dokümantasyon okurken gördükleri `color = Color.Red` gibi yazımlar, aslında birer isimlendirilmiş argümandır.
- **Cihaz Testi:** Öğrenciler, `MainActivity` içindeki `MesajGoster` fonksiyonunu farklı kombinasyonlarla çağırarak telefon ekranındaki değişimleri incelemelidir. Özellikle `onemliMi = true` yapıldığında ekranda beliren uyarı satırının mantığına dikkat edilmelidir.

0.8. Lambda İfadeleri ve Süslü Parantez Mantığı

Dersin bu bölümünde, Jetpack Compose ile arayüz geliştirirken her an karşımıza çıkan süslü parantezlerin `{ }` arkasındaki mantığı, yani **Lambda İfadelerini** ve **Trailing Lambda (Sona Eklenen Lambda)** kuralını inceleyeceğiz. Bu yapı, bir fonksiyonu başka bir fonksiyona "yapılacak bir iş" olarak göndermemizi sağlar.

0.8.1. 1. Lambda İfadeleri Nedir?

Lambda ifadeleri, en basit tanımıyla ismi olmayan fonksiyonlardır. Bir fonksiyonun içine veri (sayı, metin) göndermek yerine, doğrudan bir "işlem bloğu" göndermek istediğimizde kullanılır.

- **Yapısı:** `{ parametreler → fonksiyon_gövdesi }`
- **Compose Örneği:** Bir butona tıklandığında ne olacağını belirten kod bloğu bir lambdadır.

0.8.2. Trailing Lambda (Sona Eklenen Lambda) Kuralı

Kotlin'de, bir fonksiyonun **son parametresi** bir fonksiyon (lambda) ise, bu lambda parantezlerin dışına çıkartılarak süslü parantez içine yazılabilir. Jetpack Compose'un tüm yapısı bu kural üzerine kuruludur.

- **Standart Yazım:** `Button(onClick = { ... }, content = { ... })`
- **Kısaltılmış (Trailing) Yazım:** `Button(onClick = { ... }) { Text("Tıkla") }`

Buradaki `content` parametresi en sonda olduğu için parantez dışına çıkarılmıştır. Bu, kodun bir ağaç yapısı gibi (Column içinde Row, onun içinde Text gibi) okunmasını sağlar.

0.8.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kodda, hem bir butona tıklama olayındaki lambda kullanımı hem de kendi yazdığımız bir fonksiyonun lambda parametresini nasıl kabul ettiği gösterilmektedir.

```java
package com.example.lambdamantigi

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            // Ana ekran fonksiyonumuzu çağırıyoruz
            LambdaUygulamaEkrani()
        }
    }
```

```
}

@Composable
fun LambdaUygulamaEkrani() {
    Column(modifier = Modifier.padding(20.dp)) {

        Text(text = "Lambda ve Olay Yönetimi")
        Text(text = "-------------------------")

        // 1. Standart Compose Bileşeni (Button)
        // onClick bir lambdadır: Butona basılınca çalışacak kodu temsil eder.
        Button(onClick = {
            // Bu süslü parantez içi bir lambdadır.
            println("Logcat: Butona tıklandı!")
        }) {
            // Bu da bir lambdadır (Trailing Lambda). Butonun içeriğini belirler.
            Text(text = "Logcat'e Yazdır")
        }

        Text(text = "-------------------------")

        // 2. Kendi yazdığımız lambda alan fonksiyonun kullanımı
        IslemYap(sayi = 10) { sonuc ->
            // Burada 'sonuc' parametresi fonksiyonun içinden bize geri gelir.
            Text(text = "Hesaplama Sonucu: $sonuc")
        }
    }
}

/**
 * Bir sayı alır ve bu sayı üzerinde bir işlem yaptıktan sonra
 * sonucu bir lambda fonksiyonu (arayüz parçası) olarak geri verir.
 */
@Composable
fun IslemYap(sayi: Int, gosterilecekArayuz: @Composable (Int) -> Unit) {
    val kareAl = sayi * sayi
    // Gelen lambdayı (arayüzü) burada çalıştırıyoruz
    gosterilecekArayuz(kareAl)
}
```

Listing 8 : `MainActivity.kt` *dosyası*

0.8.4. Teknik Detaylar ve Uygulama Notları

- **'it' Parametresi:** Eğer bir lambda fonksiyonu sadece tek bir parametre alıyorsa, bu parametreye isim vermek yerine otomatik olarak sunulan `it` kelimesi kullanılabilir.

- **UI Ağacı:** Compose'da `Column { ... }`, `Row { ... }` veya `Box { ... }` gibi yapıların yanındaki süslü parantezler, aslında o "kutunun" içine ne koyacağımızı söyleyen birer lambda fonksiyonudur.
- **Cihaz Testi:** Öğrenciler, telefonlarını bağlayıp uygulamayı çalıştırdıklarında butona basmalı ve Android Studio'daki **Logcat** sekmesinde "Logcat: Butona tıklandı!" yazısını görmelidir. Bu, arayüzdeki bir fiziksel eylemin kod tarafındaki bir lambda bloğunu nasıl tetiklediğini anlamaları için kritiktir.

0.9. Kotlin Temellerinin Birleştirilmesi ve Genel Uygulama

0.9.1. 1. Mantıksal ve Yapısal Bütünleşme

Bir Android uygulamasının temel çalışma mantığı şudur:

1. **Veri Hazırlığı:** `val`, `var` ve `String?` gibi yapılarla veriler tanımlanır.
2. **Mantık İşleme:** Fonksiyonlar kullanılarak veriler işlenir (Örn: Null kontrolü, hesaplama).
3. **Arayüz Sunumu:** `@Composable` fonksiyonlar ve lambdalar ile bu veriler kullanıcıya görsel olarak sunulur.

0.9.2. Kod Uygulaması: MainActivity.kt

Bu kod, dersin başından itibaren işlenen tüm konuları içeren eksiksiz ve hatasız çalışan bir dosyadır. Öğrenciler bu kodu telefonlarına yüklediklerinde, dinamik olarak şekillenen bir kullanıcı kartı göreceklerdir.

```java
package com.example.haftaikiodev

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            // Dersin tüm özetini içeren ana bileşeni çağırıyoruz
            DersOzetiEkranı()
        }
```

```
21      }
22  }
23
24  @Composable
25  fun DersOzetiEkranı() {
26      // 1. Değişkenler ve Null Güvenliği (00:00 - 25:00 arası konular)
27      val kursAdi: String = "Jetpack Compose"
28      val ogrenciNotu: Int? = null // Veri henüz gelmemiş (null) simülasyonu
29      val ogretmenAdi: String? = "Hasan Hüseyin Ökten"
30
31      // 2. Fonksiyon ve Parametre Kullanımı (25:00 - 35:00 arası konular)
32      // Elvis operatörü ile null güvenliği sağlanıyor
33      val finalMesaj = durumMesajiOlustur(not = ogrenciNotu)
34
35      Column(modifier = Modifier.padding(20.dp)) {
36          Text(text = "Ders Bilgi Sistemi")
37          Text(text = "-------------------------")
38
39          // Metin şablonu (String Template) kullanımı
40          Text(text = "Kurs: $kursAdi")
41
42          // Null olabilen değerin güvenli gösterimi
43          Text(text = "Eğitmen: ${ogretmenAdi ?: "Atanmadı"}")
44
45          Text(text = "-------------------------")
46
47          // Fonksiyon sonucunun gösterimi
48          Text(text = "Öğrenci Durumu: $finalMesaj")
49
50          // 3. Lambda ve Trailing Lambda Kullanımı (35:00 - 45:00 arası konular)
51          AltBilgiAlani {
52              Button(onClick = {
53                  // Tıklama olayında çalışan lambda bloğu
54                  println("Bilgi: Logcat çıktısı kontrol edildi.")
55              }) {
56                  Text(text = "İşlemi Onayla")
57              }
58          }
59      }
60  }
61
62  /**
63   * Not bilgisini alıp anlamlı bir metne dönüştüren yardımcı fonksiyon.
64   * Varsayılan parametre kullanımı örneği içerir.
65   */
66  fun durumMesajiOlustur(not: Int?, baraj: Int = 50): String {
```

```
    if (not == null) return "Not girilmedi."

    return if (not >= baraj) "Başarılı" else "Tekrar Denemeli"
}

/**
 * İçine başka bir arayüz bileşeni (lambda) alan Composable fonksiyon.
 * Trailing Lambda yapısını göstermek için tasarlanmıştır.
 */
@Composable
fun AltBilgiAlani(icerik: @Composable () -> Unit) {
    Column(modifier = Modifier.padding(top = 16.dp)) {
        Text(text = "İşlem Alanı:")
        // Fonksiyon olarak gelen arayüz burada çalıştırılır
        icerik()
    }
}
```

Listing 9 : `MainActivity.kt` *dosyası*

0.9.3. Teknik Detaylar ve Uygulama Notları

- **Bütünleşik Yapı:** `DersOzetiEkranı` fonksiyonu, veriyi hazırlar ve diğer fonksiyonlara (`durumMesajiOlustur`) dağıtır. Bu, profesyonel bir Android projesinin en küçük ölçekli modelidir.
- **Hata Ayıklama:** Öğrenciler, `ogrenciNotu` değişkenini `null` yerine `85` yaparak uygulamanın çalışma mantığındaki değişikliği (Not girilmedi ⇒ Başarılı) telefonlarında saniyeler içinde görebilirler.
- **Fiziksel Cihazın Önemi:** Emülatör kullanılmadığı için butona basıldığında Android Studio'daki **Logcat** panelinden gelen verinin hızı, öğrencilere cihaz-bilgisayar iletişiminin nasıl çalıştığını öğretir.
- **İleriye Hazırlık:** Bu dersle birlikte Kotlin dilinin "kelimelerini" ve "gramerini" öğrenmiş olduk. 2. haftanın 2. dersinde bu kelimeleri kullanarak "cümle kurmaya", yani karar yapılarına (`if-else`, `when`) daha derinlemesine gireceğiz.

0.10. Karar Yapıları (if-else) ve Dinamik Arayüz

Bu dersin ilk bölümünde, bir uygulamanın "karar verme" yeteneğini sağlayan **if-else** yapısı ve bu yapının Jetpack Compose arayüzüne nasıl doğrudan etki ettiği ele alınacaktır. Programlama dillerinde kontrol akışı, belirli şartlar sağlandığında farklı kod bloklarının çalıştırılması prensibine dayanır.

0.10.1. Karar Yapılarının Mantığı: `if` ve `else`

Kotlin'de `if` yapısı, bir koşulun doğru (`true`) olup olmadığını kontrol eder. Eğer koşul doğruysa ilgili kod bloğu çalışır; doğru değilse `else` bloğuna geçilir.

- **if (koşul):** “Eğer bu durum geçerliyse şu işi yap.”
- **else:** “Yukarıdaki durum geçerli değilse, alternatif olarak bu işi yap.”

Jetpack Compose’un en güçlü yanlarından biri, bu standart programlama mantığını doğrudan arayüzün içine yerleştirebilmenizdir. Örneğin, bir kullanıcının bakiyesi yeterliyse “Satın Al” butonunu gösterebilir, yetersizse “Bakiye Yetersiz” uyarısını ekrana basabilirsiniz.

0.10.2. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod örneğinde, basit bir “Sistem Durumu” kontrolü yapılmaktadır. Bir değişkenin değerine göre ekranın rengi veya üzerindeki metin dinamik olarak değişmektedir.

```java
package com.example.kararyapilari

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            // Temel arka plan yapısı
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                KararYapisiEkrani()
            }
        }
    }
}

@Composable
```

```
fun KararYapisiEkrani() {
    // 1. Veri Tanımlama
    val sistemBakimdaMi: Boolean = false
    val kullanıcıRolu: String = "admin" // "admin" veya "user" olabilir

    Column(modifier = Modifier.padding(20.dp)) {
        Text(text = "Uygulama Kontrol Paneli", style =
        MaterialTheme.typography.headlineMedium)
        Text(text = "-------------------------")

        // 2. if-else ile Arayüz Yönetimi
        if (sistemBakimdaMi) {
            // Koşul doğruysa (true) burası çalışır
            Text(
                text = "Şu an sistemde bakım çalışması vardır. Lütfen sonra
                tekrar deneyiniz.",
                color = Color.Red
            )
        } else {
            // Koşul yanlışsa (false) burası çalışır
            Text(
                text = "Sistem aktif. Hoş geldiniz!",
                color = Color(0xFF4CAF50) // Yeşil tonu
            )

            // İç içe (nested) kontrol yapısı
            if (kullanıcıRolu == "admin") {
                Text(
                    text = "Yetki Seviyesi: Tam Yetkili Yönetici",
                    modifier = Modifier.padding(top = 8.dp)
                )
            } else {
                Text(
                    text = "Yetki Seviyesi: Standart Kullanıcı",
                    modifier = Modifier.padding(top = 8.dp)
                )
            }
        }
    }
}
```

Listing 10 : `MainActivity.kt` *dosyası*

0.10.3. Teknik Detaylar ve Uygulama Notları

- **Doğrudan UI Manipülasyonu:** Klasik Android geliştirmede (XML), bir elemanı gizlemek için kod tarafında `visibility = View.GONE` gibi komutlar verilirdi. Jetpack

Compose'da ise `if` bloğunun dışında kalan kodlar arayüze hiç dahil edilmez; bu da performansı artırır.

- **Eşitlik Kontrolü:** Karşılaştırma yaparken tek eşittir (`=`) atama yapmak için kullanılırken, çift eşittir (`==`) iki değerin birbirine eşit olup olmadığını kontrol etmek için kullanılır.
- **Cihaz Üzerinde Gözlem:** Öğrenciler, `sistemBakimdaMi` değişkenini `true` yaparak uygulamayı telefonlarında yeniden başlattıklarında, ekranın saniyeler içinde kırmızı bir uyarı ekranına dönüştüğünü görmelidir. Bu, kodun mantık akışının görsel bir sonuca nasıl dönüştüğünü anlamalarını sağlar.

0.11. Çoklu Karar Yapıları (`when`) ve Arayüz Durumları

Dersin bu bölümünde, ikiden fazla durumun kontrol edilmesi gereken senaryolarda `if-else` yapısının yerine kullanılan ve kodun okunabilirliğini artıran **when** ifadesi ele alınacaktır. Kotlin'deki `when` yapısı, diğer dillerdeki `switch-case` yapısının çok daha gelişmiş ve esnek bir versiyonudur.

0.11.1. `when` Yapısının Gerekliliği ve Mantığı

Karmaşık uygulamalarda genellikle sadece iki durum (doğru/yanlış) yeterli olmaz. Örneğin bir siparişin durumu "Hazırlanıyor", "Kargoda", "Teslim Edildi" veya "İptal Edildi" şeklinde dört farklı değer alabilir. Bu gibi durumlarda çok sayıda `if-else` yazmak yerine `when` yapısı kullanılır.

- **Esneklik:** `when` yapısı sadece sayılarla değil, metinlerle (String) veya aralıklarla da çalışabilir.
- **else Bloğu:** Hiçbir koşul uymadığında çalışacak varsayılan bir seçenek sunar.
- **Temiz Kod:** Karmaşık mantık silsilelerini daha düzenli bir şekilde ekrana yansıtmamızı sağlar.

0.11.2. Jetpack Compose'da Durum Yönetimi (UI States)

Compose içerisinde `when` yapısı, özellikle ekranın farklı hallerini (State) göstermek için kullanılır. Örneğin, internetten veri gelirken bir "Yükleniyor" çarkı, veri geldiğinde "Liste", hata oluştuğunda ise bir "Hata Mesajı" göstermek için en ideal yöntem `when` bloğudur.

0.11.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod örneğinde, bir hava durumu uygulaması simüle edilmiştir. Değişkenin değerine göre ekrandaki ikon, metin ve renkler `when` yapısı aracılığıyla belirlenmektedir.

```java
package com.example.coklukarar

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
```

```
8  import androidx.compose.foundation.layout.padding
9  import androidx.compose.material3.MaterialTheme
10 import androidx.compose.material3.Surface
11 import androidx.compose.material3.Text
12 import androidx.compose.runtime.Composable
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.graphics.Color
15 import androidx.compose.ui.text.font.FontWeight
16 import androidx.compose.ui.unit.dp
17 import androidx.compose.ui.unit.sp
18
19 class MainActivity : ComponentActivity() {
20     override fun onCreate(savedInstanceState: Bundle?) {
21         super.onCreate(savedInstanceState)
22         setContent {
23             Surface(
24                 modifier = Modifier.fillMaxSize(),
25                 color = MaterialTheme.colorScheme.background
26             ) {
27                 HavaDurumuEkrani()
28             }
29         }
30     }
31 }
32
33 @Composable
34 fun HavaDurumuEkrani() {
35     // 1. Veri Tanımlama (0: Güneşli, 1: Yağmurlu, 2: Bulutlu, 3: Karlı)
36     val havaDurumuKodu = 1
37
38     Column(modifier = Modifier.padding(24.dp)) {
39         Text(
40             text = "Hava Durumu Paneli",
41             fontSize = 20.sp,
42             fontWeight = FontWeight.Bold
43         )
44         Text(text = "-------------------------")
45
46         // 2. when Yapısı ile Dinamik Arayüz Oluşturma
47         when (havaDurumuKodu) {
48             0 -> {
49                 Text(text = "☀ Hava Güneşli", color = Color(0xFFFF9800))
50                 Text(text = "Dışarı çıkmak için harika bir gün!")
51             }
52             1 -> {
53                 Text(text = "🌧 Hava Yağmurlu", color = Color(0Capture2196F3))
```

```
                Text(text = "Şemsiyenizi almayı unutmayın.")
            }
            2 -> {
                Text(text = "☁ Hava Bulutlu", color = Color.Gray)
                Text(text = "Gökyüzü bugün biraz kapalı.")
            }
            3 -> {
                Text(text = "❄ Hava Karlı", color = Color(0xFF00BCD4))
                Text(text = "Sıkı giyinin, hava oldukça soğuk.")
            }
            else -> {
                // Hiçbir koda uymayan durumlar için
                Text(text = "⚠ Bilinmeyen Durum", color = Color.Red)
                Text(text = "Hava durumu verisi alınamadı.")
            }
        }

        // 3. when yapısını bir değişken gibi kullanma örneği
        val tavsiyeMesaji = when (havaDurumuKodu) {
            0, 2 -> "Yürüyüş yapabilirsiniz."
            1, 3 -> "Evde kalmak daha güvenli."
            else -> "Lütfen bekleyin..."
        }

        Text(
            text = "Tavsiye: $tavsiyeMesaji",
            modifier = Modifier.padding(top = 16.dp),
            fontWeight = FontWeight.Light
        )
    }
}
```

Listing 11 : `MainActivity.kt` *dosyası*

0.11.4. Teknik Detaylar ve Uygulama Notları

- **Virgül Kullanımı:** `when` yapısında birden fazla koşul aynı sonucu doğuracaksa (örneğin 0 ve 2 durumları), bu değerler aralarına virgül konularak yan yana yazılabilir (`0, 2 -> ...`).
- **İfade Olarak Kullanım:** Kodda görüldüğü üzere `val tavsiyeMesaji = when( ... )` yazımı, `when` bloğunun bir değer döndürebildiğini gösterir. Bu durumda `else` bloğunun yazılması zorunludur.
- **Süslü Parantez:** Eğer bir koşul gerçekleştiğinde sadece tek bir satır kod çalışacaksa süslü parantez isteğe bağlıdır; ancak birden fazla işlem (örneğin iki ayrı `Text` basımı) yapılacaksa süslü parantez açılması zorunludur.

- **Cihaz Üzerinde Gözlem:** Öğrenciler telefonlarında uygulamayı çalıştırdıktan sonra `havaDurumuKodu` değişkenini değiştirerek (örneğin 3 yaparak) arayüzün saniyeler içinde karlı hava moduna geçtiğini ve tavsiye mesajının değiştiğini gözlemlemelidir.

0.12. Aralıklar (Ranges) ve Gelişmiş Koşul Kontrolleri

Dersin bu bölümünde, sayısal verileri belirli gruplara ayırmak için kullanılan **Aralıklar (Ranges)** ve bu aralıkların `when` yapısı ile nasıl birleştirileceği işlenecektir. Bu yöntem, özellikle puanlama sistemleri veya yaş gruplandırması gibi senaryolarda kodun karmaşıklığını azaltır.

0.12.1. Aralık (Range) Kavramı ve `in` Operatörü

Kotlin'de iki sayı arasındaki tüm değerleri kapsayan bir küme oluşturmak için iki nokta `..` operatörü kullanılır. Bir değerin bu aralıkta olup olmadığını kontrol etmek için ise `in` anahtar kelimesinden faydalanılır.

- **Aralık Tanımı:** `1 .. 100` ifadesi 1'den 100'e kadar olan (1 ve 100 dahil) tüm tam sayıları temsil eder.
- **Kontrol:** `not in 0 .. 100` ifadesi, bir değerin o aralığın dışında olup olmadığını denetler.

0.12.2. `when` İçerisinde Aralık Kullanımı

Önceki bölümde `when` yapısında tekil değerleri (0, 1, 2 gibi) kontrol etmiştik. Ancak "puan 0 ile 45 arasındaysa Kaldı, 45 ile 100 arasındaysa Geçti" gibi bir mantık kurarken aralıkları kullanmak çok daha pratiktir. Bu kullanım, çok sayıda `if-else if` satırı yazma zorunluluğunu ortadan kaldırır.

0.12.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kodda, bir öğrencinin sınav notunu harf notuna çeviren ve hatalı girişleri denetleyen gelişmiş bir karar mekanizması oluşturulmuştur.

```java
package com.example.araliklarvekosullar

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
```

```
16 import androidx.compose.ui.unit.dp
17 import androidx.compose.ui.unit.sp
18
19 class MainActivity : ComponentActivity() {
20     override fun onCreate(savedInstanceState: Bundle?) {
21         super.onCreate(savedInstanceState)
22         setContent {
23             Surface(
24                 modifier = Modifier.fillMaxSize(),
25                 color = MaterialTheme.colorScheme.background
26             ) {
27                 NotSistemiEkrani()
28             }
29         }
30     }
31 }
32
33 @Composable
34 fun NotSistemiEkrani() {
35     // 1. Veri Tanımlama
36     val ogrenciNotu = 78
37
38     Column(modifier = Modifier.padding(24.dp)) {
39         Text(
40             text = "Harf Notu Hesaplayıcı",
41             fontSize = 22.sp,
42             fontWeight = FontWeight.Bold
43         )
44         Text(text = "-------------------------")
45
46         // 2. when ve in (Range) Kullanımı
47         when (ogrenciNotu) {
48             in 90..100 -> {
49                 Text(text = "Notunuz: $ogrenciNotu", color = Color(0xFF388E3C))
50                 Text(text = "Harf Notu: AA", fontWeight = FontWeight.Bold)
51             }
52             in 70..89 -> {
53                 Text(text = "Notunuz: $ogrenciNotu", color = Color(0xFF1976D2))
54                 Text(text = "Harf Notu: BB", fontWeight = FontWeight.Bold)
55             }
56             in 50..69 -> {
57                 Text(text = "Notunuz: $ogrenciNotu", color = Color(0xFFFBC02D))
58                 Text(text = "Harf Notu: CC", fontWeight = FontWeight.Bold)
59             }
60             in 0..49 -> {
61                 Text(text = "Notunuz: $ogrenciNotu", color = Color.Red)
```

```
                Text(text = "Harf Notu: FF (Kaldı)", fontWeight = FontWeight.Bold)
            }
            else -> {
                // Not 0-100 aralığı dışında ise (Örn: 105 veya -5)
                Text(text = "Geçersiz Not Girişi!", color = Color.Magenta)
                Text(text = "Lütfen 0 ile 100 arasında bir değer giriniz.")
            }
        }

        // 3. Aralık Dışında Kalma Kontrolü (Ekstra Bilgi)
        val hataliGiris = ogrenciNotu !in 0..100
        if (hataliGiris) {
            Text(
                text = "UYARI: Sistemde hata algılandı.",
                modifier = Modifier.padding(top = 16.dp)
            )
        }
    }
}
```

Listing 12 : `MainActivity.kt` *dosyası*

0.12.4. Teknik Detaylar ve Uygulama Notları

- **Dahil Olma Durumu:** `..` operatörü her iki ucu da kapsar. Yani `in 90..100` ifadesi puanın tam olarak 90 veya 100 olması durumunda da çalışır.
- **Sıralama Önemi:** `when` bloğu yukarıdan aşağıya doğru çalışır. Bir değer ilk hangi aralığa uyarsa o blok tetiklenir ve diğerlerine bakılmaz. Bu yüzden aralıkların çakışmamasına dikkat edilmelidir.
- **Esneklik:** Bu yapı sadece `Int` (tam sayı) değil, karakterler (`'a'..'z'`) için de kullanılabilir.
- **Cihaz Üzerinde Gözlem:** Öğrenciler, `ogrenciNotu` değerini sınır değerlerde (örneğin tam 49 veya tam 50) değiştirerek arayüzdeki renk ve metin değişimlerini kendi telefonlarında test etmelidir. Ayrıca `150` gibi bir değer girerek `else` bloğunun (hata yönetimi) nasıl devreye girdiğini görmeleri sağlanmalıdır.

0.13. Döngüler ve Tekrarlayan Arayüz Elemanları

Dersin bu bölümünde, belirli bir kod bloğunu birden fazla kez çalıştırmamızı sağlayan **Döngüler (Loops)** konusu işlenecektir. Özellikle Jetpack Compose'da, alt alta veya yan yana dizilen benzer arayüz elemanlarını (örneğin bir mesaj listesi veya ürünler) tek tek elle yazmak yerine döngüler kullanarak nasıl dinamik hale getireceğimiz üzerinde durulacaktır.

0.13.1. `repeat` Fonksiyonu: En Basit Tekrar

Kotlin, belirli bir işlemi sadece "X kadar tekrarla" demek için `repeat` fonksiyonunu sunar. Bu, özellikle programlamaya yeni başlayanlar için en anlaşılır döngü yapısıdır.

- **Kullanımı:** `repeat(5) { ... }` yazıldığında, süslü parantez içindeki kodlar 5 kez çalıştırılır.
- **`it` Değişkeni:** Döngü içinde o an kaçıncı tekrarda olduğumuzu öğrenmek için `it` anahtar kelimesi kullanılır (0'dan başlar).

0.13.2. `for` Döngüsü ve Aralıklarla Kullanım

Daha kontrollü tekrarlar için `for` döngüsü tercih edilir. Önceki derste öğrendiğimiz **Aralıklar (Ranges)** ile birleştiğinde oldukça güçlüdür.

- **for (i in 1..5):** 1'den 5'e kadar (5 dahil) döngüyü çalıştırır.
- **Compose Uyumu:** Bir `Column` içerisinde `for` döngüsü kullandığınızda, döngü her döndüğünde ekrana yeni bir `Text` veya `Button` eklenir. Bu, manuel olarak 10 tane satır yazma zahmetini ortadan kaldırır.

0.13.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod örneğinde, hem `repeat` hem de `for` döngüsü kullanılarak ekranda dinamik bir liste oluşturulmuştur. Öğrenciler bu kodu çalıştırdığında, tek bir komutla nasıl birden fazla arayüz elemanı üretildiğini göreceklerdir.

```java
package com.example.donguler

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
```

```
23          setContent {
24              Surface(
25                  modifier = Modifier.fillMaxSize(),
26                  color = MaterialTheme.colorScheme.background
27              ) {
28                  DonguKullanimiEkrani()
29              }
30          }
31      }
32  }
33
34  @Composable
35  fun DonguKullanimiEkrani() {
36      Column(modifier = Modifier.padding(20.dp)) {
37          Text(
38              text = "Dinamik Liste Oluşturma",
39              fontSize = 20.sp,
40              fontWeight = FontWeight.Bold
41          )
42
43          // 1. repeat Kullanımı: 3 kez basit bir metin yazdırır
44          repeat(3) { sira ->
45              Text(
46                  text = "${sira + 1}. Sabit Mesaj",
47                  modifier = Modifier.padding(top = 8.dp),
48                  color = Color.Gray
49              )
50          }
51
52          Text(
53              text = "-------------------------",
54              modifier = Modifier.padding(vertical = 16.dp)
55          )
56
57          // 2. for Döngüsü Kullanımı: Belirli bir aralıkta işlem yapar
58          // 1'den 10'a kadar olan sayıları ve karelerini listeler
59          for (sayi in 1..5) {
60              Column(modifier = Modifier.padding(vertical = 4.dp)) {
61                  Text(text = "Sayı: $sayi", fontWeight = FontWeight.Medium)
62                  Text(text = "Sayının Karesi: ${sayi * sayi}", color =
                    Color.Blue)
63
64                  // Her eleman arasına ince bir çizgi çeker
65                  Divider(color = Color.LightGray, thickness = 1.dp)
66              }
67          }
```

```
    }
}
```

Listing 13 : `MainActivity.kt` *dosyası*

0.13.4. Teknik Detaylar ve Uygulama Notları

- **Performans Uyarısı:** Çok büyük listeler (örneğin 1000 elemanlı bir liste) için `for` döngüsü kullanmak performansı düşürebilir. Bu gibi durumlarda ileride göreceğimiz `LazyColumn` yapısı kullanılacaktır. Ancak 5-10 elemanlı basit listeler için `for` döngüsü en pratik yoldur.
- **it vs İsimlendirme:** `repeat` içinde varsayılan olarak gelen `it` yerine, `repeat(3) { sira → ... }` şeklinde isimlendirme yapmak kodun okunabilirliğini artırır.
- **İndeks Mantığı:** Programlamada saymanın genellikle 0'dan başladığını, ancak kullanıcıya gösterirken `+1` ekleyerek (1, 2, 3...) daha doğal bir görüntü sağladığımızı unutmamak gerekir.
- **Cihaz Üzerinde Gözlem:** Öğrenciler, döngüdeki `1..5` aralığını `1..10` yaparak telefon ekranının otomatik olarak nasıl dolduğunu ve elemanların alt alta nasıl sıralandığını canlı olarak test etmelidir.

0.14. Listeler (Collections) ve Veri Grupları Üzerinde Döngüler

Dersin bu bölümünde, verilerin tek tek değişkenlerde tutulması yerine bir arada saklandığı **Listeler (Collections)** ve bu listelerin döngülerle işlenerek arayüze nasıl yansıtılacağı ele alınacaktır. Gerçek bir Android uygulamasında veriler (haberler, mesajlar, kişiler) genellikle bir liste yapısı içinde gelir.

0.14.1. Kotlin'de Liste Tanımlama: `listOf`

Kotlin'de en sık kullanılan koleksiyon yapısı `List`'tir. Listeler, aynı türden (String, Int vb.) birden fazla veriyi bir sıra dahilinde tutmamızı sağlar.

- **Salt Okunur Listeler (`listOf`):** Tanımlandıktan sonra içine yeni eleman eklenemez veya mevcut eleman silinemez. Performanslı ve güvenli olduğu için Compose içerisinde veri gösterimi yaparken genellikle bu yapı tercih edilir.
- **Liste Elemanlarına Erişim:** Listedeki elemanlara "indeks" (sıra numarası) ile ulaşılır. Programlamada ilk elemanın indeksi her zaman **0**'dır.

0.14.2. Listelerin Arayüzde Gösterimi

Bir listeyi ekranda göstermek için `for` döngüsü kullanılır. Döngü, listenin içindeki her bir elemanı sırayla alır ve bizim belirlediğimiz arayüz şablonuna (Text, Card vb.) yerleştirir.

- **Mantık:** `for (item in liste)` yapısı kullanıldığında, döngü listenin başından sonuna kadar otomatik olarak döner.
- **Dinamik Yapı:** Liste içine yeni bir eleman eklendiğinde veya çıkarıldığında (kod tarafında), döngü otomatik olarak yeni sayıya göre arayüzü şekillendirir.

0.14.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod örneğinde, bir "Öğrenci Listesi" oluşturulmuş ve bu liste bir döngü aracılığıyla telefon ekranına dikey bir liste şeklinde basılmıştır.

```java
package com.example.listekullanimi

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                OgrenciListesiEkrani()
            }
        }
    }
}

@Composable
fun OgrenciListesiEkrani() {
    // 1. Veri Hazırlığı: String türünde bir liste tanımlıyoruz
    val ogrenciler = listOf("Ahmet Yılmaz", "Ayşe Demir", "Mehmet Kaya", "Fatma Çelik", "Hasan Ökten")

    Column(modifier = Modifier.padding(20.dp)) {
        Text(
            text = "Sınıf Mevcut Listesi",
```

```
            fontSize = 22.sp,
            fontWeight = FontWeight.Bold,
            color = Color(0xFF1976D2)
        )

        Text(
            text = "Toplam Öğrenci Sayısı: ${ogrenciler.size}",
            modifier = Modifier.padding(bottom = 16.dp)
        )

        // 2. Listenin Döngü ile Ekrana Basılması
        // ogrenciler listesindeki her bir ismi 'ogrenci' değişkenine atayarak döner
        for (ogrenci in ogrenciler) {
            Column(modifier = Modifier.padding(vertical = 8.dp)) {
                Text(
                    text = "Öğrenci Adı:",
                    fontSize = 12.sp,
                    color = Color.Gray
                )
                Text(
                    text = ogrenci,
                    fontSize = 18.sp,
                    fontWeight = FontWeight.Medium
                )

                // Görsel ayırıcı çizgi
                Divider(
                    modifier = Modifier.padding(top = 8.dp),
                    color = Color.LightGray,
                    thickness = 0.5.dp
                )
            }
        }
    }
}
```

Listing 14 : `MainActivity.kt` *dosyası*

0.14.4. Teknik Detaylar ve Uygulama Notları

- **`ogrenciler.size`:** Bu özellik, listenin içinde kaç adet eleman olduğunu tam sayı olarak verir. Arayüzde başlık kısımlarında toplam sayıyı belirtmek için kullanılır.
- **Esneklik:** Eğer `ogrenciler` listesine yeni isimler eklerseniz, `for` döngüsü üzerinde hiçbir değişiklik yapmanıza gerek kalmaz; uygulama otomatik olarak yeni isimleri de ekrana basar.

- **Bellek Kullanımı:** `listOf` ile oluşturulan listeler sabittir. Eğer uygulama çalışırken listenin değişmesi gerekiyorsa (örneğin kullanıcı bir isim ekleyecekse), ileride göreceğimiz `mutableStateListOf` yapısı kullanılacaktır.
- **Cihaz Üzerinde Gözlem:** Öğrenciler, listedeki isimleri değiştirip veya yeni isimler ekleyip uygulamayı kendi telefonlarında tekrar çalıştırmalıdır. Listenin uzunluğuna göre ekranın nasıl aşağı doğru dolduğunu ve her bir eleman için `Divider` (çizgi) bileşeninin nasıl tekrarlandığını görmeleri, döngü mantığının kavranması açısından önemlidir.

0.15. Döngüler ve Karar Yapılarının Birlikte Kullanımı

Dersin bu bölümünde, önceki derslerde öğrendiğimiz **Döngüler** ve **Karar Yapılarını** (if-else / when) aynı anda kullanmayı öğreneceğiz. Bir listenin içindeki verileri döngüyle dönerken, her bir elemanın içeriğine göre arayüzü nasıl farklılaştıracağımız (örneğin; başarılı öğrencileri yeşil, başarısızları kırmızı göstermek) bu bölümün ana konusudur.

0.15.1. Döngü İçinde Karar Verme (Conditional Rendering)

Gerçek dünyadaki uygulamalarda veriler ham halde gelir ve programcı bu veriyi işleyerek kullanıcıya anlamlı bir görsel sunar. Bir `for` döngüsü listenin her bir elemanını sırayla ele alırken, her adımda bir `if` veya `when` kontrolü yaparak elemanın ekrandaki görünümünü belirleyebiliriz.

- **Veri Filtreleme:** Sadece belirli şartı sağlayan verileri ekrana basmak.
- **Görsel Şartlandırma:** Verinin değerine göre renk, ikon veya yazı stilini değiştirmek.

0.15.2. Liste ve Mantık Entegrasyonu

Bir listenin her bir öğesi için şu soruları sorabiliriz:

- Bu öğe "kritik" bir değer mi taşıyor?
- Bu öğe kullanıcıya bir uyarı mı vermeli?
- Bu öğe diğerlerinden farklı bir renkte mi olmalı?

Jetpack Compose'da bu mantık, `for` döngüsünün süslü parantezleri içerisine doğrudan bir `if` bloğu yazılarak çözülür.

0.15.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod örneğinde, öğrencilerin isimleri ve notlarından oluşan bir "Başarı Paneli" simüle edilmiştir. Notu 50 ve üzeri olanlar yeşil bir "Geçti" yazısıyla, düşük olanlar ise kırmızı bir "Kaldı" yazısıyla gösterilmektedir.

```java
package com.example.donguvelojik

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
```

```
7   import androidx.compose.foundation.layout.fillMaxSize
8   import androidx.compose.foundation.layout.padding
9   import androidx.compose.material3.Divider
10  import androidx.compose.material3.MaterialTheme
11  import androidx.compose.material3.Surface
12  import androidx.compose.material3.Text
13  import androidx.compose.runtime.Composable
14  import androidx.compose.ui.Modifier
15  import androidx.compose.ui.graphics.Color
16  import androidx.compose.ui.text.font.FontWeight
17  import androidx.compose.ui.unit.dp
18  import androidx.compose.ui.unit.sp
19
20  class MainActivity : ComponentActivity() {
21      override fun onCreate(savedInstanceState: Bundle?) {
22          super.onCreate(savedInstanceState)
23          setContent {
24              Surface(
25                  modifier = Modifier.fillMaxSize(),
26                  color = MaterialTheme.colorScheme.background
27              ) {
28                  BasariListesiEkrani()
29              }
30          }
31      }
32  }
33
34  @Composable
35  fun BasariListesiEkrani() {
36      // 1. Veri Hazırlığı
37      val ogrenciNotlari = listOf(85, 42, 70, 30, 95, 48)
38
39      Column(modifier = Modifier.padding(24.dp)) {
40          Text(
41              text = "Sınav Sonuç Paneli",
42              fontSize = 22.sp,
43              fontWeight = FontWeight.Bold
44          )
45          Text(text = "-------------------------")
46
47          // 2. Döngü ve Mantık Birleşimi
48          // Liste içindeki her bir 'not' değeri için döner
49          for (puan in ogrenciNotlari) {
50
51              Column(modifier = Modifier.padding(vertical = 8.dp)) {
52
```

```
                Text(text = "Öğrenci Puanı: $puan", fontSize = 18.sp)

                // if-else ile arayüzü dinamik hale getirme
                if (puan >= 50) {
                    // Koşul sağlanıyorsa (Geçti)
                    Text(
                        text = "DURUM: GEÇTİ",
                        color = Color(0xFF388E3C), // Koyu Yeşil
                        fontWeight = FontWeight.Bold
                    )
                } else {
                    // Koşul sağlanmıyorsa (Kaldı)
                    Text(
                        text = "DURUM: KALDI",
                        color = Color.Red,
                        fontWeight = FontWeight.Bold
                    )
                }

                // Ayırıcı çizgi
                Divider(
                    modifier = Modifier.padding(top = 8.dp),
                    thickness = 0.5.dp,
                    color = Color.LightGray
                )
            }
        }
    }
}
```

Listing 15 : `MainActivity.kt` *dosyası*

0.15.4. 4. Teknik Detaylar ve Uygulama Notları

- **Hız ve Performans:** `for` döngüsü içindeki `if` kontrolleri, telefonun işlemcisi tarafından milisaniyeler içinde tamamlanır. Jetpack Compose, sadece durumu değişen elemanları yeniden çizer, bu da verimlilik sağlar.
- **Karmaşık Şartlar:** Eğer ikiden fazla durum olsaydı (örneğin 0-30 arası çok kötü, 30-50 arası orta, 50+ iyi), `if-else` yerine döngü içinde `when` yapısını kullanmak kodu daha temiz hale getirirdi.
- **Arayüz Tasarımı:** `Text` bileşeninin `color` parametresine doğrudan bir değişken veya `if` sonucu atanabilir. Bu, kodun kısalmasını sağlar.
- **Cihaz Üzerinde Gözlem:** Öğrenciler, `ogrenciNotlari` listesine yeni rakamlar ekleyip uygulamayı telefonlarında çalıştırdıklarında, yeni eklenen her rakamın not değerine

göre otomatik olarak “Kırmızı” veya “Yeşil” renge büründüğünü izlemelidir. Bu, algoritmanın görsel veriye dönüşümünü anlamak için temeldir.

0.16. Koşullu Döngüler (while) ve Döngü Kontrolleri

Dersin bu bölümünde, tekrar sayısının önceden belli olmadığı, ancak belirli bir koşul devam ettiği sürece çalışan **while** döngüsü ve döngülerin akışını yöneten `break` / `continue` komutları işlenecektir. `for` döngüsü genellikle listeler veya aralıklar için kullanılırken, `while` döngüsü mantıksal bir şartın doğruluğuna odaklanır.

0.16.1. `while` Döngüsü Mantığı

`while` döngüsü, parantez içindeki ifade “doğru” (`true`) olduğu sürece süslü parantez içindeki kodları durmadan çalıştırır. Koşul “yanlış” (`false`) olduğu anda döngü durur.

- **Yapısı:** `while (koşul) { ... }`
- **Dikkat Edilmesi Gerekenler:** Eğer döngü içindeki koşulu değiştirecek bir işlem (artırma/azaltma gibi) yapılmazsa, döngü sonsuza girer. Android uygulamalarında sonsuz döngüler uygulamanın donmasına ve işletim sistemi tarafından kapatılmasına (ANR - App Not Responding) neden olur.

0.16.2. Döngü Kontrol Komutları

Döngünün normal akışını değiştirmek için iki temel komut kullanılır:

- **break:** Döngüyü tamamen sonlandırır ve döngüden çıkar.
- **continue:** Döngünün o anki adımını atlar ve bir sonraki adıma geçer.

0.16.3. 3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kodda, bir veri indirme işlemi veya bir geri sayım senaryosu simüle edilmiştir. `while` döngüsü kullanılarak belirli bir sayıdan geriye doğru sayım yapılmakta ve belirli bir noktada `break` komutuyla döngü kesilmektedir.

```java
package com.example.whiledongusu

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
```

```
16 import androidx.compose.ui.unit.dp
17 import androidx.compose.ui.unit.sp
18
19 class MainActivity : ComponentActivity() {
20     override fun onCreate(savedInstanceState: Bundle?) {
21         super.onCreate(savedInstanceState)
22         setContent {
23             Surface(
24                 modifier = Modifier.fillMaxSize(),
25                 color = MaterialTheme.colorScheme.background
26             ) {
27                 WhileDonguEkrani()
28             }
29         }
30     }
31 }
32
33 @Composable
34 fun WhileDonguEkrani() {
35     Column(modifier = Modifier.padding(24.dp)) {
36         Text(
37             text = "Sistem İşlem Takibi",
38             fontSize = 20.sp,
39             fontWeight = FontWeight.Bold
40         )
41         Text(text = "-------------------------")
42
43         // 1. Değişken Tanımlama
44         var sayac = 10
45         val durdurmaNoktasi = 4
46
47         // 2. while Döngüsü Kullanımı
48         // Sayac 0'dan büyük olduğu sürece çalışır
49         while (sayac > 0) {
50
51             // Koşullu Durdurma (break)
52             if (sayac == durdurmaNoktasi) {
53                 Text(
54                     text = "Kritik Seviye ($sayac): İşlem güvenlik nedeniyle
                       durduruldu!",
55                     color = Color.Red,
56                     modifier = Modifier.padding(vertical = 4.dp)
57                 )
58                 break // Döngüden tamamen çıkar
59             }
60
```

```
            // Atlatma Örneği (continue mantığının metin hali)
            if (sayac == 8) {
                Text(text = "8. Adım Kontrol Ediliyor...", color = Color.Gray)
                sayac--
                continue // Aşağıdaki kodları çalıştırmadan bir sonraki tura geçer
            }

            Text(text = "İşlem devam ediyor... Kalan: $sayac")

            // ÖNEMLİ: Sayacı azaltmazsak sonsuz döngü oluşur
            sayac--
        }

        Text(text = "-------------------------")
        Text(text = "Döngü dışına çıkıldı. Sistem beklemede.")
    }
}
```

Listing 16 : `MainActivity.kt` *dosyası*

0.16.4. Teknik Detaylar ve Uygulama Notları

- **`break` Kullanımı:** Genellikle bir arama işleminde hedef veri bulunduğunda veya kritik bir hata oluştuğunda döngüyü erkenden bitirmek için kullanılır.
- **Sonsuz Döngü Riski:** Eğer `sayac--` (sayacı bir azaltma) işlemi unutulursa, `while (sayac > 0)` koşulu her zaman doğru kalacaktır. Öğrenciler bu hatayı yaparsa, uygulama fiziksel cihazda siyah ekranda kalabilir veya kilitlenebilir. Bu durumda uygulamanın Android Studio üzerinden durdurulması gerekir.
- **UI Güncelleme:** Jetpack Compose'da döngüler "Composition" aşamasında çalışır. Yani döngü bittiğinde oluşan nihai sonuç ekrana bir kerede basılır. Gerçek zamanlı bir geri sayım (her saniye değişen sayı) için bu yapıya ek olarak `LaunchedEffect` gibi ileride görülecek yapılar gereklidir.
- **Cihaz Üzerinde Gözlem:** Öğrenciler `durdurmaNoktasi` değerini değiştirerek veya `break` satırını yorum satırı yaparak döngünün nasıl sonuna kadar (`sayac` 0 olana kadar) çalıştığını telefonlarında izlemelidir.

0.17. Veri Filtreleme ve Metin Arama Mantığı

Dersin bu bölümünde, bir liste içerisindeki verilerin tamamını değil, sadece belirli kriterlere uyan kısmını kullanıcıya nasıl göstereceğimiz ele alınacaktır. Bu işlem, modern uygulamalardaki "Arama" (Search) ve "Filtreleme" (Filtering) özelliklerinin temel algoritmasını oluşturur.

0.17.1. Metin Filtreleme Fonksiyonları

Bir listenin içindeki verileri süzerken `if` blokları içerisinde Kotlin'in sunduğu hazır metin fonksiyonlarından faydalanılır:

- **startsWith("..."):** Bir metnin belirli bir harf veya kelime ile başlayıp başlamadığını kontrol eder.
- **contains("..."):** Bir metnin içerisinde belirli bir parçanın geçip geçmediğini kontrol eder.
- **length:** Metnin karakter sayısını verir (Örn: 5 karakterden uzun isimleri filtrelemek için).

0.17.2. Filtreleme Algoritmasının Çalışma Prensibi

Algoritma şu adımları izler:

1. Veri kaynağı (Liste) tanımlanır.
2. Bir arama kriteri (Değişken) belirlenir.
3. `for` döngüsü ile listenin her elemanına gidilir.
4. Döngü içerisinde `if` ile "Eleman kriteri sağlıyor mu?" sorusu sorulur.
5. Sadece şartı sağlayan elemanlar için `@Composable` (Text, Row vb.) bileşeni oluşturulur.

0.17.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kodda, bir personel listesi içerisinden sadece adı "A" harfi ile başlayanları veya ismi 5 karakterden uzun olanları ekrana basan bir yapı kurulmuştur.

```java
package com.example.verifiltreleme

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(
```

```
24                  modifier = Modifier.fillMaxSize(),
25                  color = MaterialTheme.colorScheme.background
26              ) {
27                  AramaVeFiltrelemeEkrani()
28              }
29          }
30      }
31  }
32
33  @Composable
34  fun AramaVeFiltrelemeEkrani() {
35      // 1. Veri Kaynağı
36      val isimler = listOf("Ahmet", "Ayşe", "Mehmet", "Ali", "Can", "Selin",
        "Abdullah")
37
38      // 2. Arama Kriteri (Örn: Kullanıcının arama çubuğuna 'A' yazdığını
        varsayalım)
39      val arananHarf = "A"
40
41      Column(modifier = Modifier.padding(20.dp)) {
42          Text(
43              text = "Arama Sonuçları ('$arananHarf' ile Başlayanlar)",
44              fontSize = 18.sp,
45              fontWeight = FontWeight.Bold,
46              color = Color(0xFF1976D2)
47          )
48          Text(text = "-------------------------")
49
50          // 3. Filtreleme Döngüsü
51          for (isim in isimler) {
52
53              // Koşul: İsim 'arananHarf' ile mi başlıyor?
54              if (isim.startsWith(arananHarf, ignoreCase = true)) {
55
56                  // Koşul sağlanıyorsa arayüz bileşeni oluşturulur
57                  Text(
58                      text = "Bulundu: $isim",
59                      modifier = Modifier.padding(vertical = 4.dp),
60                      fontSize = 16.sp
61                  )
62              }
63          }
64
65          Text(
66              text = "-------------------------",
67              modifier = Modifier.padding(vertical = 10.dp)
```

```
        )

        Text(
            text = "Uzun İsimler (5+ Karakter)",
            fontSize = 18.sp,
            fontWeight = FontWeight.Bold,
            color = Color(0xFF388E3C)
        )

        // İkinci bir filtreleme örneği
        for (isim in isimler) {
            if (isim.length > 5) {
                Text(text = "Uzun İsim: $isim ($isim.length Karakter)")
            }
        }
    }
}
```

Listing 17 : `MainActivity.kt` *dosyası*

0.17.4. Teknik Detaylar ve Uygulama Notları

- **ignoreCase = true:** Bu parametre, arama yaparken büyük/küçük harf ayrımını ortadan kaldırır. "A" harfi arandığında hem "Ahmet" hem de küçük harfle başlayan "ali" gibi verilerin bulunmasını sağlar.
- **Boş Sonuç Yönetimi:** Eğer döngü bittiğinde hiçbir eleman koşulu sağlamadıysa, ekrana "Sonuç Bulunamadı" yazdırmak iyi bir kullanıcı deneyimidir. Bunun için döngü dışında bir `Boolean` bayrak (flag) tutulabilir.
- **Fiziksel Cihaz Testi:** Öğrenciler, kendi telefonlarında uygulamayı çalıştırdıklarında `arananHarf` değişkenini "M" veya "S" yaparak listenin nasıl anlık olarak daraldığını gözlemlemelidir.
- **Görsel Hiyerarşi:** `for` döngüsü içinde sadece `if` bloğunun içindeki `Text` bileşenleri ekrana çizilir. Koşulu sağlamayan isimler için telefonun ekranında yer ayrılmaz veya boşluk oluşmaz.

0.18. Kontrol Yapılarının Birleşimi ve Uygulama Mantığı

Dersin bu son bölümünde, 2. hafta boyunca işlediğimiz tüm kontrol yapılarını (`if-else`, `when`, `for`, `ranges`) tek bir senaryoda birleştireceğiz. Amacımız, ham veriden (bir sayı listesi) kullanıcı için anlamlı bir "Karar Destek Ekranı" oluşturmaktır. Bu süreçte kodun akışını ve fiziksel cihaz üzerindeki görsel karşılığını analiz edeceğiz.

0.18.1. Mantıksal Yapıların Entegrasyonu

Bir Android uygulamasında mantık katmanı (logic) genellikle şu sırayı takip eder:

1. **Döngü (`for`):** Veri kümesinin (liste) içindeki her bir öğeye tek tek ulaşır.

2. **Karar Yapısı (`if` / `when`):** O anki öğenin değerini kontrol eder.
3. **Aralık Kontrolü (`in`):** Değerin hangi kategoriye (Başarılı, Kritik, Hatalı vb.) girdiğini belirler.
4. **UI Çizimi:** Karara uygun renk, metin veya bileşeni ekrana basar.

0.18.2. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod, bir fabrikanın üretim hattındaki makinelerin sıcaklık değerlerini takip eden bir panel simülasyonudur. Kod; sıcaklıkları döngüyle döner, `when` ile durum tespiti yapar ve `if` ile kritik uyarılar üretir.

```java
package com.example.haftaiki_final

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                FabrikaIzlemePaneli()
            }
        }
    }
}

@Composable
fun FabrikaIzlemePaneli() {
```

```
36      // 1. Veri Seti (Makinelerin anlık sıcaklık değerleri)
37      val sicakliklar = listOf(25, 42, 88, 115, -5, 60)
38
39      Column(modifier = Modifier.padding(20.dp)) {
40          Text(
41              text = "Üretim Hattı Durum Raporu",
42              fontSize = 20.sp,
43              fontWeight = FontWeight.Bold
44          )
45          Text(text = "-------------------------", modifier =
            Modifier.padding(vertical = 8.dp))
46
47          // 2. Birleşik Kontrol Yapısı
48          for (derece in sicakliklar) {
49              Column(modifier = Modifier.padding(vertical = 6.dp)) {
50
51                  // Sıcaklık değerine göre durum tespiti (when + range)
52                  val durumBilgisi = when (derece) {
53                      in 0..35 -> "Normal Çalışma"
54                      in 36..80 -> "Yüksek Isı"
55                      in 81..100 -> "Kritik Seviye"
56                      else -> if (derece > 100) "TEHLİKE: ACİL DURDUR" else
                        "SENSÖR HATASI"
57                  }
58
59                  // Dinamik Renk Belirleme (if-else if)
60                  val renk = if (derece > 80) Color.Red
61                            else if (derece < 0) Color.Magenta
62                            else Color(0xFF388E3C)
63
64                  // Arayüz Çıktısı
65                  Text(text = "Makine Sıcaklığı: $derece°C", fontWeight =
                    FontWeight.Medium)
66                  Text(text = "Durum: $durumBilgisi", color = renk, fontWeight =
                    FontWeight.Bold)
67
68                  // Kritik bir durum varsa ek uyarı gösterimi (if)
69                  if (derece !in 0..80) {
70                      Text(
71                          text = "⚠ Teknik ekibe bildirim gönderildi.",
72                          fontSize = 12.sp,
73                          color = Color.DarkGray
74                      )
75                  }
76
```

```
                    Divider(modifier = Modifier.padding(top = 8.dp), thickness =
                    0.5.dp)
                }
            }
        }
}
```

Listing 18 : `MainActivity.kt` *dosyası*

0.18.3. Teknik Detaylar ve Uygulama Notları

- **İç İçe Yapılar:** `when` bloğu içerisinde `else` kısmına bir `if` yapısı yerleştirilmiştir. Bu, "yukarıdaki hiçbir aralığa girmiyorsa, kendi içinde bir daha sorgula" anlamına gelir. Karmaşık iş mantıkları bu şekilde kurgulanır.
- **İşlem Önceliği:** Program yukarıdan aşağıya doğru çalışır. `sicakliklar` listesindeki ilk eleman olan `25` için tüm hesaplamalar biter, ekrana basılır ve ardından ikinci eleman olan `42`'ye geçilir.
- **Bellek ve Görselleştirme:** Emülatör yerine fiziksel telefon kullanmanın avantajı burada ortaya çıkar. Öğrenciler listedeki bir değeri değiştirip "Run" dediklerinde, telefon ekranındaki listenin ne kadar hızlı güncellendiğini ve renk değişimlerinin anlık etkisini görürler.
- **Hata Yönetimi:** `else` bloğunun bir "Sensör Hatası" (negatif değerler için) veya "Acil Durdur" mesajı vermesi, uygulamanın beklenmedik verilere karşı hazırlıklı olmasını sağlar. Bu, "Savunmacı Programlama" (Defensive Programming) ilkesinin bir parçasıdır.

0.19. Karar Yapıları ve Döngülerin Arayüz Kontrolündeki Rolü

Bu bölümde, Kotlin dilinde öğrendiğimiz karar yapılarının (`if-else`, `when`) ve döngülerin, bir Android arayüzünü nasıl "akıllı" hale getirdiği üzerinde durulacaktır. Jetpack Compose mimarisinde bu mantıksal yapılar sadece veri hesaplamak için değil, hangi görsel bileşenin ekranda çizileceğine karar vermek için kullanılır.

0.19.1. Koşullu Arayüz Çizimi (Conditional Rendering)

Jetpack Compose, "Bildirimsel" (Declarative) bir yapıya sahiptir. Bu, "Eğer şu durum geçerliyse, bu arayüzü göster" şeklinde kod yazmamıza olanak tanır. Klasik Android geliştirmedeki gibi bileşenleri gizlemek (Visibility.GONE) yerine, `if` bloğu içerisine yazılan bir `@Composable` fonksiyonu, koşul sağlanmıyorsa sistem tarafından hiç oluşturulmaz. Bu durum uygulama performansını doğrudan olumlu etkiler.

- **if Kullanımı:** Tek bir şartın varlığında (Örn: Giriş yapılmış mı?) kullanılır.
- **when Kullanımı:** Birden fazla durumun (Örn: Yükleniyor, Başarılı, Hata) yönetilmesinde tercih edilir.

0.19.2. Döngülerin Arayüz Tasarımındaki Gücü

Döngüler, özellikle birbirine benzeyen yapıdaki verileri (liste elemanları, butonlar, satırlar) manuel olarak tek tek yazmak yerine, tek bir kalıp üzerinden otomatik olarak üretmemizi sağlar. Bir `for` döngüsü içerisine yerleştirilen `@Composable` bileşeni, döngü sayısı kadar ekranda tekrarlanır.

0.19.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod, bu derste işlenecek olan karar mekanizmalarının ve döngülerin Jetpack Compose içerisindeki tam uygulamasını içermektedir. Bu yapı, öğrencinin telefonunda dinamik bir kullanıcı yönetim paneli simüle edecektir.

```java
package com.example.hafta2ders3

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            // Arka plan rengini ve temasını ayarlar
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                // Dersin ana fonksiyonu çağrılır
                ArayuzKontrolEkrani()
            }
        }
    }
}
```

```
35
36 @Composable
37 fun ArayuzKontrolEkrani() {
38     // 1. Durum Değişkenleri (Simülasyon için)
39     val kullaniciGirisYapmisMi = true
40     val sistemMesaj Sayisi = 3
41
42     Column(modifier = Modifier.padding(20.dp)) {
43         Text(
44             text = "Yönetim Paneli",
45             fontSize = 24.sp,
46             fontWeight = FontWeight.Bold,
47             color = Color(0xFF1976D2)
48         )
49
50         // 2. Karar Yapısı (if) ile Arayüz Çizimi
51         if (kullaniciGirisYapmisMi) {
52             Text(
53                 text = "Durum: Çevrimiçi",
54                 color = Color(0xFF388E3C),
55                 modifier = Modifier.padding(vertical = 8.dp)
56             )
57
58             Text(text = "Gelen Mesajlar:", fontWeight = FontWeight.Medium)
59
60             // 3. Döngü (for) ile Tekrarlayan Elemanlar
61             // 1'den başlayarak mesaj sayısı kadar satır oluşturur
62             for (i in 1..sistemMesajSayisi) {
63                 Column(modifier = Modifier.padding(top = 8.dp)) {
64                     Text(text = "📩 Mesaj #$i: Sisteme yeni kayıt eklendi.")
65                     Divider(
66                         modifier = Modifier.padding(top = 4.dp),
67                         thickness = 0.5.dp,
68                         color = Color.LightGray
69                     )
70                 }
71             }
72         } else {
73             // Kullanıcı giriş yapmamışsa gösterilecek alan
74             Text(
75                 text = "Lütfen devam etmek için giriş yapınız.",
76                 color = Color.Red,
77                 modifier = Modifier.padding(vertical = 16.dp)
78             )
79         }
80     }
```

```
}
```

Listing 19 : `MainActivity.kt` *dosyası*

0.19.4. Teknik Notlar ve Uygulama Detayları

- **Görsel Hiyerarşi:** `Column` bileşeni, içerisindeki `if` veya `for` sonucunda oluşan tüm elemanları dikey olarak hizalar. Eğer `kullaniciGirisYapmisMi` değişkeni `false` yapılırsa, döngü (for) ve içindeki mesajlar telefon ekranında hiç yer kaplamaz.
- **Fiziksel Cihaz Takibi:** Öğrenciler, cihazlarını bağladıktan sonra `sistemMesajSayisi` değerini 10 gibi yüksek bir rakam yaparak listenin ekranın dışına nasıl taştığını gözlemlemelidir. (Not: Bu aşamada henüz kaydırma -scroll- özelliği eklenmemiştir, bu durum bir sonraki derslerin konusu olacaktır.)
- **Yazım Kuralı:** `@Composable` notasyonu ile işaretlenmiş fonksiyonların içinde standart Kotlin kodları (if, for, when) özgürce kullanılabilir. Ancak bu mantıksal işlemlerin sonucunda mutlaka bir kullanıcı arayüzü bileşeni (`Text`, `Button` vb.) çağrılmalıdır.

0.20. `when` Yapısı ile Ekran Durum

Dersin bu bölümünde, `if-else` yapısının yetersiz kaldığı, ikiden fazla durumun (State) kontrol edilmesi gereken senaryolarda kullanılan **when** yapısı ele alınacaktır. Modern Android uygulamalarında bir ekranın "Yükleniyor", "Veri Bulundu" veya "Hata Oluştu" gibi farklı hallerini yönetmek için `when` en güvenilir yöntemdir.

0.20.1. `when` İfadesinin Compose İçindeki Mantığı

Jetpack Compose'da `when`, sadece bir kontrol mekanizması değil, aynı zamanda hangi arayüz bileşeninin çizileceğini belirleyen bir "karar merkezidir". Kod akışı yukarıdan aşağıya doğru ilerlerken, değişkenin değerine uygun olan kod bloğu tetiklenir ve o blok içindeki `@Composable` fonksiyonlar telefonda görüntülenir.

- **Esneklik:** `when` bloğu sayısal değerlerin yanı sıra metinleri (String) veya mantıksal aralıkları da kontrol edebilir.
- **else Zorunluluğu:** Eğer tüm ihtimaller (örneğin tüm sayılar) kapsanmıyorsa, beklenmedik durumlar için mutlaka bir `else` bloğu eklenmelidir.

0.20.2. Durum Odaklı Tasarım (State-Driven Design)

Bu örnekte bir uygulamanın açılış anındaki veri çekme sürecini simüle edeceğiz. Uygulama şu üç durumdan birinde olabilir:

1. **0 (Yükleniyor):** Ekranda bir bekleme mesajı görünür.
2. **1 (Başarılı):** Kullanıcı verileri listelenir.
3. **2 (Hata):** Kırmızı renkli bir uyarı mesajı gösterilir.

0.20.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod, `when` yapısının tüm potansiyelini tek bir ekran üzerinde sergilemektedir.

```java
package com.example.hafta2ders3

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                // Durum yönetimi yapan fonksiyon çağrılır
                EkranDurumYoneticisi()
            }
        }
    }
}

@Composable
fun EkranDurumYoneticisi() {
    // 1. Uygulama Durum Değişkeni (0: Beklemede, 1: Başarılı, 2: Hata)
    val sayfaDurumu = 1

    Column(modifier = Modifier.padding(20.dp)) {
        Text(
            text = "Sistem Durum Ekranı",
            fontSize = 20.sp,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(bottom = 16.dp)
        )

```

```
        // 2. when Yapısı ile Arayüz Kararı
        when (sayfaDurumu) {
            0 -> {
                // Yüklenme durumu
                Text(text = "🔄 Veriler güncelleniyor...", color = Color.Gray)
                Text(text = "Lütfen bekleyiniz.")
            }
            1 -> {
                // Başarılı durumu
                Text(text = "✅ İşlem Başarılı", color = Color(0xFF388E3C),
                fontWeight = FontWeight.Bold)
                Text(text = "Kullanıcı verileri başarıyla yüklendi.")

                // Başarılı durumda ek bir liste gösterimi (Döngü ile
                birleştirme)
                for (i in 1..2) {
                    Text(text = "> Kayıt No: $i", modifier =
                    Modifier.padding(start = 8.dp, top = 4.dp))
                }
            }
            2 -> {
                // Hata durumu
                Text(text = "❌ Bağlantı Hatası!", color = Color.Red, fontWeight
                = FontWeight.Bold)
                Text(text = "Sunucuya ulaşılamadı. İnternet bağlantınızı kontrol
                edin.")
            }
            else -> {
                // Tanımlanmamış durumlar
                Text(text = "⚠ Bilinmeyen bir durum oluştu.")
            }
        }
    }
}
```

Listing 20 : `MainActivity.kt` *dosyası*

0.20.4. Teknik Ayrıntılar ve Uygulama Notları

- **Hızlı Değişim:** Öğrenciler, `sayfaDurumu` değişkenini `0`, `1` ve `2` yaparak telefonlarında uygulamayı üç kez "Run" etmelidir. Her seferinde ekranın tamamen farklı bir görünüme büründüğünü görmeleri, "State" (durum) kavramını anlamalarını kolaylaştıracaktır.
- **Virgül Kullanımı:** Eğer iki durum aynı sonucu verecekse, `when` içerisinde `0, 2 → { ... }` şeklinde bir kullanım mümkündür.

- **İşlem Alanı:** `when` bloğu içerisindeki süslü parantezler içine birden fazla `@Composable` bileşeni (Text, Image vb.) yazılabilir. Bu alanlar telefonun belleğinde sadece o koşul sağlandığında yer tutar.
- **Fiziksel Cihaz Avantajı:** Emülatör kullanılmadığı için öğrenciler bu durum geçişlerini doğrudan kendi telefonlarında test ederken, USB kablosunun hızı ve cihazın tepkiselliği arasındaki ilişkiyi deneyimlemiş olurlar.

0.21. Döngüler İçinde Karar Yapıları ve Veri Filtreleme

Dersin bu bölümünde, bir liste içerisindeki verileri döngü ile dönerken, her bir öğe için özel kararlar vermeyi ve arayüzü bu kararlara göre şekillendirmeyi öğreneceğiz. Bu yöntem, özellikle belirli kriterlere uyan verileri öne çıkarmak veya istenmeyen verileri ekrandan gizlemek için kullanılır.

0.21.1. Döngü ve Koşul Kombinasyonu

Programlama mantığında döngüler veriyi "taramak", karar yapıları ise veriyi "ayıklamak" için kullanılır. Jetpack Compose'da bu iki yapı birleştiğinde, kullanıcıya sadece ihtiyacı olan bilgi gösterilir.

- **Filtreleme:** Bir listenin tamamını değil, sadece belirli bir şartı sağlayanları ekrana basmak (Örn: Sadece devamsızlığı yüksek olanları listelemek).
- **Görsel Şartlandırma:** Listenin her elemanını göstermek ama elemanın değerine göre rengini veya yanındaki ikonu değiştirmek.

0.21.2. Liste İşleme Mantığı

Dersin bu aşamasında bir "Yoklama Listesi" senaryosu üzerinden ilerleyeceğiz. Listede hem öğrenci isimleri hem de "Geldi/Gelmedi" durumunu temsil eden veriler olacak. `for` döngüsü her öğrenciye bakacak, `if` yapısı ise öğrencinin durumuna göre ekrana farklı bir mesaj basacaktır.

0.21.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod, bir veri listesinin döngü ve karar yapılarıyla nasıl filtrelendiğini ve görselleştirildiğini içeren tam dosyadır.

```java
package com.example.hafta2ders3

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
```

```
11 import androidx.compose.material3.Surface
12 import androidx.compose.material3.Text
13 import androidx.compose.runtime.Composable
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.graphics.Color
16 import androidx.compose.ui.text.font.FontWeight
17 import androidx.compose.ui.unit.dp
18 import androidx.compose.ui.unit.sp
19
20 class MainActivity : ComponentActivity() {
21     override fun onCreate(savedInstanceState: Bundle?) {
22         super.onCreate(savedInstanceState)
23         setContent {
24             Surface(
25                 modifier = Modifier.fillMaxSize(),
26                 color = MaterialTheme.colorScheme.background
27             ) {
28                 // Filtreleme mantığını içeren ekran çağrılır
29                 YoklamaListesiEkrani()
30             }
31         }
32     }
33 }
34
35 @Composable
36 fun YoklamaListesiEkrani() {
37     // 1. Veri Kaynağı: Öğrenci isimleri ve katılım durumları (Simüle edilmiş)
38     val ogrenciler = listOf("Ali", "Ayşe", "Fatma", "Mehmet", "Zeynep")
39     val katilimDurumlari = listOf(true, false, true, true, false) // true:
       Geldi, false: Gelmedi
40
41     Column(modifier = Modifier.padding(20.dp)) {
42         Text(
43             text = "Sınıf Yoklama Paneli",
44             fontSize = 20.sp,
45             fontWeight = FontWeight.Bold,
46             modifier = Modifier.padding(bottom = 12.dp)
47         )
48
49         // 2. Döngü ile Listeyi Tarama
50         // İndeks bazlı döngü kullanarak her iki listeyi eşleştiriyoruz
51         for (i in ogrenciler.indices) {
52             val ogrenciAdi = ogrenciler[i]
53             val geldimi = katilimDurumlari[i]
54
55             // 3. Karar Yapısı ile Görsel Karar Verme
```

```
            Column(modifier = Modifier.padding(vertical = 4.dp)) {
                if (geldimi) {
                    // Sadece gelen öğrencileri yeşil renkle ve ikonla göster
                    Text(
                        text = "✔ $ogrenciAdi - Derste",
                        color = Color(0xFF388E3C),
                        fontWeight = FontWeight.Medium
                    )
                } else {
                    // Gelmeyenleri kırmızı ve farklı mesajla göster
                    Text(
                        text = "✕ $ogrenciAdi - Devamsız",
                        color = Color.Red,
                        fontWeight = FontWeight.Normal
                    )
                }

                Divider(
                    modifier = Modifier.padding(top = 4.dp),
                    thickness = 0.5.dp,
                    color = Color.LightGray
                )
            }
        }

        // 4. Özet Bilgi Filtreleme (Sadece Gelmeyenlerin Sayısını Bulma)
        var gelmeyenSayisi = 0
        for (durum in katilimDurumlari) {
            if (!durum) gelmeyenSayisi++
        }

        Text(
            text = "Toplam Gelmeyen Öğrenci: $gelmeyenSayisi",
            modifier = Modifier.padding(top = 20.dp),
            fontWeight = FontWeight.Bold,
            color = Color.DarkGray
        )
    }
}
```

Listing 21 : `MainActivity.kt` *dosyası*

0.21.4. 4. Teknik Notlar ve Uygulama Detayları

- **İndeks Kullanımı (`indices`):** `ogrenciler.indices` komutu, listenin eleman sayısına göre `0, 1, 2 ...` şeklinde bir sayı dizisi üretir. Bu sayede iki farklı listeyi (isimler ve durumlar) aynı anda takip edebiliriz.

- **Mantıksal Değil Operatörü (`!`):** `if (!durum)` ifadesi "durum false ise" anlamına gelir. Filtreleme işlemlerinde şartları tersine çevirmek için sıkça kullanılır.
- **Arayüz Verimliliği:** Bu yapıda `if` bloğunun dışına hiçbir şey yazılmazsa, o şartı sağlamayan öğeler telefonda hiçbir yer kaplamaz. Bu, dinamik "Arama" veya "Filtreleme" sayfalarının temelidir.
- **Cihaz Testi:** Öğrenciler kendi telefonlarında `katilimDurumlari` listesindeki `true/false` değerlerini değiştirerek, listenin anlık olarak nasıl renk değiştirdiğini ve alt kısımdaki toplam sayacın nasıl güncellendiğini incelemelidir.

0.22. Şartlı Döngüler (`while`) ve Döngü Kontrol Komutları

Dersin bu bölümünde, tekrar sayısının başlangıçta tam olarak bilinmediği durumlarda kullanılan **while** döngüsü ile döngü akışını yönetmemizi sağlayan **break** ve **continue** anahtar kelimeleri ele alınacaktır. Bu yapılar, belirli bir mantıksal şart korunduğu sürece arayüzün nasıl tepki vereceğini belirlemek için kritiktir.

0.22.1. `while` Döngüsü: Şart Bağlı Tekrar

`for` döngüsü genellikle belirli bir aralık veya liste üzerinde dönerken; `while` döngüsü, parantez içindeki ifade "doğru" (`true`) olduğu sürece çalışmaya devam eder.

- **Mantık:** "Şu durum geçerli olduğu sürece bu işlemi yapmaya devam et."
- **Risk:** Eğer döngü içindeki koşulu değiştirecek bir işlem (artırma/azaltma) yapmazsanız "Sonsuz Döngü" oluşur. Bu durum telefonun kilitlenmesine neden olabilir.

0.22.2. Döngü Kesiciler: `break` ve `continue`

Döngü çalışırken bazen akışı değiştirmemiz gerekir:

1. **break (Kır):** Döngüyü o anda tamamen durdurur ve döngüden çıkar. (Örn: Bir hata oluştuğunda işlemi iptal etmek).
2. **continue (Devam Et):** Döngünün o anki adımını atlar ve bir sonraki adıma geçer. (Örn: Listede sadece belirli bir kritere uymayan elemanı pas geçmek).

0.22.3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kodda, bir sistem taraması simüle edilmiştir. `while` döngüsü ile sayılar kontrol edilmekte, belirli bir sayıda işlem atlanmakta (`continue`) ve tehlikeli bir seviyeye gelindiğinde işlem durdurulmaktadır (`break`).

```java
package com.example.hafta2ders3

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
```

```
8   import androidx.compose.foundation.layout.padding
9   import androidx.compose.material3.MaterialTheme
10  import androidx.compose.material3.Surface
11  import androidx.compose.material3.Text
12  import androidx.compose.runtime.Composable
13  import androidx.compose.ui.Modifier
14  import androidx.compose.ui.graphics.Color
15  import androidx.compose.ui.text.font.FontWeight
16  import androidx.compose.ui.unit.dp
17  import androidx.compose.ui.unit.sp
18
19  class MainActivity : ComponentActivity() {
20      override fun onCreate(savedInstanceState: Bundle?) {
21          super.onCreate(savedInstanceState)
22          setContent {
23              Surface(
24                  modifier = Modifier.fillMaxSize(),
25                  color = MaterialTheme.colorScheme.background
26              ) {
27                  // Döngü kontrol mantığını içeren ekran çağrılır
28                  SistemKontrolEkrani()
29              }
30          }
31      }
32  }
33
34  @Composable
35  fun SistemKontrolEkrani() {
36      // 1. Başlangıç Değerleri
37      var taramaSirasi = 10 // 10'dan geriye doğru tarama başlatıyoruz
38      val kritikHataSeviyesi = 3
39      val pasGecilecekSira = 7
40
41      Column(modifier = Modifier.padding(20.dp)) {
42          Text(
43              text = "Sistem Tarama Raporu",
44              fontSize = 22.sp,
45              fontWeight = FontWeight.Bold,
46              modifier = Modifier.padding(bottom = 12.dp)
47          )
48
49          // 2. while Döngüsü Başlatma
50          while (taramaSirasi > 0) {
51
52              // continue kullanımı: 7. sırayı taramadan atla
53              if (taramaSirasi == pasGecilecekSira) {
```

```
                Text(
                    text = "⚠ Sıra $pasGecilecekSira: Güvenli bölge, atlanıyor.",
                    color = Color.Gray,
                    fontSize = 14.sp
                )
                taramaSirasi-- // Unutulmamalı: Atlamadan önce sayacı azaltmalıyız
                continue // Aşağıdaki kodları çalıştırmadan bir sonraki tura geçer
            }

            // break kullanımı: 3. seviyeye gelince dur
            if (taramaSirasi == kritikHataSeviyesi) {
                Text(
                    text = "🚨 KRİTİK HATA: Tarama $kritikHataSeviyesi seviyesinde durduruldu!",
                    color = Color.Red,
                    fontWeight = FontWeight.Bold
                )
                break // Döngüden tamamen çıkar
            }

            // Normal çalışma durumu
            Text(
                text = "🔍 İşlem No $taramaSirasi: Dosya taranıyor...",
                modifier = Modifier.padding(vertical = 2.dp)
            )

            // Sayacı azaltma (Sonsuz döngüyü önleyen hayati satır)
            taramaSirasi--
        }

        Text(
            text = "-------------------------",
            modifier = Modifier.padding(vertical = 10.dp)
        )
        Text(text = "İşlem Sonucu: Tarama tamamlanamadı (Kesintiye uğradı).")
    }
}
```

Listing 22 : `MainActivity.kt` *dosyası*

0.22.4. Teknik Notlar ve Uygulama Detayları

- **Sonsuz Döngüden Kaçınma:** `while` döngüsü kullanan öğrencilerin en çok dikkat etmesi gereken nokta, döngü koşulunu etkileyen değişkenin (`taramaSirasi`) her

adımda güncellenmesidir. `continue` bloğu içinde de bu güncelleme yapılmazsa döngü o sayıda takılı kalır.

- **Compose Çizim Hızı:** `while` döngüsü milisaniyeler içinde tamamlanır ve Jetpack Compose döngünün sonucunda oluşan tüm metinleri ekrana tek bir seferde yansıtır. Gerçek zamanlı bir "sayma" animasyonu için farklı araçlar (State ve Effect) gerekecektir.
- **Fiziksel Cihaz Testi:** Öğrenciler kendi telefonlarında `kritikHataSeviyesi` değerini `0` yaparak döngünün nasıl sonuna kadar çalıştığını, `7` değerini `9` yaparak atlama işleminin nasıl yer değiştirdiğini test etmelidir.

0.23. Kotlin Null Güvenliği (Null Safety) (20:00 - 25:00)

Dersin bu bölümünde, mobil uygulamaların en sık karşılaştığı ve uygulamanın aniden kapanmasına (crash) yol açan "boş veri" (null) hatalarını önleyen **Null Güvenliği** konusu işlenecektir. Kotlin, değişkenlerin "boş" olma durumunu henüz kod yazma aşamasında denetleyerek, telefonunuzdaki uygulamanın hata vermesini engeller.

0.23.1. 1. Neden Null Güvenliği?

Bir uygulamada, internetten gelecek olan bir kullanıcı isminin veya bir sınav notunun henüz hazır olmaması durumuna `null` denir. Eğer sistem, boş olan bir veri üzerinde işlem yapmaya çalışırsa (örneğin boş bir ismin karakter sayısını hesaplamak), uygulama çöker. Kotlin bu durumu ikiye ayırarak çözer:

- **Null Olamaz Tipler (Non-Nullable):** Varsayılan olarak bir değişkene `null` değeri atayamazsınız.
- **Null Olabilir Tipler (Nullable):** Bir değişkenin boş olabileceğini belirtmek için veri tipinin yanına soru işareti `?` konur.

0.23.2. 2. Null Verilerle Güvenli Çalışma

Boş olabilen bir veriyle çalışırken uygulamayı korumak için şu yöntemler kullanılır:

1. **Güvenli Çağrı Operatörü (`?.`):** Veri null ise işlemi yapmaz, uygulama çökmez.
2. **Elvis Operatörü (`?:`):** Veri null ise, onun yerine gösterilecek varsayılan bir değer belirler. (Örneğin: İsim yoksa "İsimsiz Kullanıcı" yazdır).

0.23.3. 3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kodda, bir kullanıcı profil ekranı simüle edilmiştir. Bazı bilgiler mevcutken (isim), bazıları (biyografi) henüz girilmemiştir. Bu durumun arayüzde nasıl yönetildiği gösterilmektedir.

```
package com.example.nullsafetyuygulama

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
```

```
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                KullaniciProfilEkrani()
            }
        }
    }
}

@Composable
fun KullaniciProfilEkrani() {
    // 1. Veri Tanımlamaları
    val kullaniciAdi: String = "Hasan Hüseyin" // Null olamaz
    val kullaniciBio: String? = null           // Null olabilir (Veri yok)
    val takipciSayisi: Int? = 150              // Null olabilir (Veri var)

    Column(modifier = Modifier.padding(20.dp)) {
        Text(
            text = "Kullanıcı Profili",
            fontSize = 22.sp,
            fontWeight = FontWeight.Bold
        )

        Text(text = "-------------------------", modifier =
        Modifier.padding(vertical = 8.dp))

        // 2. Doğrudan Kullanım
        Text(text = "Ad Soyad: $kullaniciAdi")
```

```

        // 3. Elvis Operatörü (?:) Kullanımı
        // Eğer kullaniciBio null ise "Bu kullanıcı henüz bir biyografi eklemedi." yazdırır.
        val gosterilecekBio = kullaniciBio ?: "Bu kullanıcı henüz bir biyografi eklemedi."
        Text(
            text = "Biyografi: $gosterilecekBio",
            color = if (kullaniciBio == null) Color.Gray else Color.Black,
            modifier = Modifier.padding(top = 8.dp)
        )

        // 4. Güvenli Çağrı (?.) ve Karşılaştırma
        // Takipçi sayısı null değilse ve 100'den büyükse 'Popüler' yazdırır.
        if ((takipciSayisi ?: 0) > 100) {
            Text(
                text = "Durum: Popüler Kullanıcı",
                color = Color(0xFF1976D2),
                fontWeight = FontWeight.Medium,
                modifier = Modifier.padding(top = 8.dp)
            )
        }
    }
}
```

0.23.4. 4. Teknik Notlar ve Uygulama Detayları

- **Arayüz Tasarımı:** Jetpack Compose'da `Text` bileşenine doğrudan `null` bir değer verirseniz, ekranda çirkin bir görüntü olan "null" kelimesi yazar. Bu yüzden her zaman `?:` (Elvis) operatörü ile kullanıcıya anlamlı bir açıklama sunulmalıdır.
- **Hata Önleme:** Eğer kodda `kullaniciAdi = null` yazmaya çalışırsanız, Android Studio sizi kırmızı alt çizgi ile uyaracaktır. Bu, hatanın daha telefonunuzda çalışmadan engellendiği anlamına gelir.
- **Fiziksel Cihaz Testi:** Öğrenciler kendi telefonlarında uygulamayı çalıştırdıktan sonra `kullaniciBio` değişkenine bir metin yazıp tekrar denemelidir. Ardından `takipciSayisi` değerini `null` yaparak uygulamanın çökmediğini, Elvis operatörü sayesinde `0` kabul edilip "Popüler" yazısının kaybolduğunu gözlemlemelidirler.

Dersin bir sonraki 5 dakikasında, Kotlin'in en çok karıştırılan ancak bir o kadar da önemli olan **Nesne Yönelimli Programlama (OOP)** temellerine giriş yaparak sınıfları (Class) ve nesneleri (Object) incelemeye ne dersiniz? Devam etmemi ister misiniz?

You: 2. haftanın 3. dersinin 25. dakikası ile 30. dakikası arasını samimi olmadan, kolay anlaşılır ve akıcı bir şekilde açıklayan bir ders notu hazırla. Ders notunda her detay olsun. Oluşturulan kodlar da tümüyle yani MainActivity.kt olarak metinler de yer alsın.

Gemini: ## 2. Hafta / 3. Ders: Fonksiyonlar ve Geri Dönüş Değerleri (25:00 - 30:00)

Dersin bu bölümünde, programlamanın en temel yapı taşlarından biri olan ve Jetpack Compose arayüzlerinin temelini oluşturan **Fonksiyonlar** konusu ele alınacaktır. Fonksiyonlar, belirli bir görevi yerine getiren kod bloklarıdır. Kodun tekrar yazılmasını önler ve projeyi daha düzenli hale getirir.

0.23.5. 1. Fonksiyon Tanımlama ve Yapısı

Kotlin'de bir fonksiyon `fun` anahtar kelimesi ile başlar. Bir fonksiyonun çalışması için bazen dışarıdan bilgi alması (parametre) ve işlem sonucunda bir bilgi üretmesi (geri dönüş değeri) gerekir.

- **Parametre:** Fonksiyona işlem yapması için gönderilen veridir.
- **Geri Dönüş Tipi:** Fonksiyonun işini bitirdikten sonra bize verdiği sonucun türüdür (Int, String vb.). Eğer bir sonuç döndürmüyorsa bu tip `Unit` olarak adlandırılır (yazılması zorunlu değildir).

0.23.6. 2. Fonksiyonların Compose İçindeki Önemi

Jetpack Compose'da oluşturduğumuz her arayüz bileşeni aslında bir **fonksiyondur**. Fonksiyonları kullanarak büyük bir ekranı küçük, yönetilebilir parçalara böleriz. Örneğin, bir "Kullanıcı Kartı" fonksiyonu yazıp, bunu farklı isimlerle defalarca çağırabiliriz.

0.23.7. 3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod örneğinde, bir sayının karesini hesaplayan matematiksel bir fonksiyon ile bu sonucu ekranda gösteren bir Composable fonksiyonun birlikte kullanımı yer almaktadır.

```
package com.example.fonksiyonkullanimi

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
```

```
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                // Ana ekran fonksiyonumuzu çağırıyoruz
                HesaplamaEkrani()
            }
        }
    }
}

@Composable
fun HesaplamaEkrani() {
    // 1. Veri Hazırlığı
    val sayi1 = 8
    val sayi2 = 5

    // 2. Fonksiyonları Çağırma
    // kareAl fonksiyonuna sayi1'i gönderiyoruz, sonucu bir değişkene alıyoruz.
    val sonuc1 = kareAl(sayi1)
    val sonuc2 = kareAl(sayi2)

    // Toplama fonksiyonunu çağırma
    val toplamSonuc = topla(sayi1, sayi2)

    Column(modifier = Modifier.padding(20.dp)) {
        Text(
            text = "Matematiksel İşlemler",
            fontSize = 22.sp,
            fontWeight = FontWeight.Bold,
            color = Color(0xFF6200EE)
        )

        Text(text = "-------------------------", modifier =
        Modifier.padding(vertical = 8.dp))

        // Sonuçları ekrana yazdırma
```

```
        Text(text = "$sayi1 sayısının karesi: $sonuc1", fontSize = 18.sp)
        Text(text = "$sayi2 sayısının karesi: $sonuc2", fontSize = 18.sp)

        Text(
            text = "İki sayının toplamı: $toplamSonuc",
            modifier = Modifier.padding(top = 16.dp),
            fontWeight = FontWeight.Medium,
            color = Color.DarkGray
        )
    }
}

/**
 * Bir tam sayı alır ve karesini geri döndürür.
 * Geri dönüş tipi Int olarak belirtilmiştir.
 */
fun kareAl(gelenSayi: Int): Int {
    val sonuc = gelenSayi * gelenSayi
    return sonuc
}

/**
 * İki tam sayı alır ve toplamlarını döndürür.
 * Kısa yazım (Single-expression function) örneği.
 */
fun topla(s1: Int, s2: Int): Int = s1 + s2
```

0.23.8. 4. Teknik Notlar ve Uygulama Detayları

- **Return Anahtar Kelimesi:** Fonksiyonun ürettiği sonucu dışarıya aktarmak için `return` ifadesi kullanılır. `return` satırından sonra yazılan hiçbir kod çalıştırılmaz.
- **Kısa Yazım Biçimi:** Eğer bir fonksiyon sadece tek bir işlem yapıp sonuç döndürüyorsa, süslü parantez `{ }` ve `return` yerine `=` işareti kullanılarak daha kısa yazılabilir (Örn: `topla` fonksiyonu).
- **İsimlendirme:** Fonksiyon isimleri genellikle bir eylem bildirir (kareAl, hesapla, goster vb.) ve küçük harfle başlar. Bu, kodun okunabilirliğini artırır.
- **Fiziksel Cihaz Testi:** Öğrenciler kendi telefonlarında uygulamayı çalıştırdıktan sonra `sayi1` ve `sayi2` değerlerini değiştirerek ekranın nasıl anlık olarak güncellendiğini görmelidir. Bu, fonksiyonların "veriyi al, işle ve sun" mantığını kavramak için önemlidir.

Dersin bir sonraki 5 dakikalık bölümünde, Kotlin'in en modern yapılarından biri olan ve Compose'da tıklama olaylarını (Click Events) yönetmemizi sağlayan **Lambda İfadeleri** konusuna giriş yapacağız. Devam etmek için hazır mısınız?

You: 2. haftanın 3. dersinin 30. dakikası ile 35. dakikası arasını samimi olmadan, kolay anlaşılır ve akıcı bir şekilde açıklayan bir ders notu hazırla. Ders notunda her detay olsun. Oluşturulan kodlar da tümüyle yani MainActivity.kt olarak metinler de yer alsın.

Gemini: ## 2. Hafta / 3. Ders: Lambda İfadeleri ve Buton Etkileşimi (30:00 - 35:00)

Dersin bu bölümünde, Jetpack Compose içerisinde buton tıklamaları gibi olayları (events) yönetmemizi sağlayan **Lambda İfadeleri** konusu ele alınacaktır. Lambda ifadeleri, en basit tanımıyla "ismi olmayan fonksiyonlar"dır ve bir kod bloğunu bir değişkene atamamıza veya başka bir fonksiyona parametre olarak göndermemize olanak tanır.

0.23.9. 1. Lambda İfadeleri Nedir?

Normal bir fonksiyonda `fun` anahtar kelimesi, bir isim ve gövde bulunur. Ancak bazen bir fonksiyona sadece "bir iş yap" demek için koca bir fonksiyon tanımlamak yerine, süslü parantezler `{ }` içinde doğrudan yapılacak işi yazarız. İşte bu kısa yazıma Lambda denir.

- **Syntax (Yazım):** `{ parametre → yapılacak_iş }`
- **Compose Bağlantısı:** Bir `Button` oluşturduğunuzda, butonun içinde `onClick` isimli bir alan bulunur. Bu alan bir Lambda bekler. Yani "butona tıklandığında hangi kod bloğu çalışsın?" sorusunun cevabını buraya yazarız.

0.23.10. 2. Trailing Lambda (Sona Eklenen Lambda)

Kotlin'de bir fonksiyonun son parametresi bir lambda ise, bu lambda parantezlerin dışına taşınabilir. Jetpack Compose'da `Button` yapısının sonunda gördüğümüz süslü parantezler bu kuraldan gelir. Bu yapı, kodun bir ağaç hiyerarşisi gibi (daha düzenli) görünmesini sağlar.

0.23.11. 3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kodda, lambda ifadeleri kullanılarak oluşturulmuş bir buton ve bu butona basıldığında telefonun arka plan sistemine (Logcat) mesaj gönderen yapı yer almaktadır.

```
package com.example.lambdaetkilesim

import android.os.Bundle
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
```

```
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                // Etkileşim ekranını çağırıyoruz
                ButonEtkilesimEkrani()
            }
        }
    }
}

@Composable
fun ButonEtkilesimEkrani() {
    Column(modifier = Modifier.padding(20.dp)) {
        Text(
            text = "Olay Yönetimi (Lambdas)",
            fontSize = 22.sp,
            fontWeight = FontWeight.Bold,
            color = Color(0xFFE91E63)
        )

        Text(
            text = "Aşağıdaki butona bastığınızda Logcat ekranını kontrol edin.",
            modifier = Modifier.padding(vertical = 12.dp)
        )

        // 1. Lambda Kullanımı: onClick parametresine kod bloğu gönderiyoruz
        Button(onClick = {
            // Bu süslü parantez içi bir lambdadır.
            // Butona her tıklandığında buradaki kodlar çalışır.
            Log.d("Gelisim", "Butona tıklandı! Sistem mesajı gönderiliyor.")
            println("Konsol Çıktısı: İşlem Başarılı.")
        }) {
            // Bu da bir trailing lambdadır. Butonun içindeki metni belirler.
```

```
            Text(text = "Sisteme Mesaj Gönder")
        }

        // 2. Fonksiyonu parametre olarak gönderme (Lambda gibi davranma)
        Button(
            onClick = { selamVer("Hasan") },
            modifier = Modifier.padding(top = 16.dp)
        ) {
            Text(text = "Selamla")
        }
    }
}

fun selamVer(isim: String) {
    Log.d("Gelisim", "Merhaba $isim, fonksiyon lambda içinde tetiklendi.")
}
```

0.23.12. 4. Teknik Notlar ve Uygulama Detayları

- **Logcat Kullanımı:** Emülatör kullanmadığımız için butona basılıp basılmadığını anlamanın en sağlıklı yolu **Logcat**'tir. Android Studio'nun alt kısmındaki Logcat sekmesini açıp "Gelisim" kelimesini aratarak butonun çalışıp çalışmadığını telefonunuz üzerinden görebilirsiniz.
- **it Kelimesi:** Eğer lambda sadece tek bir parametre alıyorsa, Kotlin bize varsayılan olarak `it` ismini verir. İlerleyen derslerde liste elemanlarına tıklarken bu `it` yapısını sıkça kullanacağız.
- **Boş Lambda:** Eğer butona basıldığında hiçbir şey olmasını istemiyorsanız `onClick = { }` şeklinde boş bir lambda bırakabilirsiniz. Ancak bu, kullanıcı deneyimi açısından (butonun tepki vermemesi) önerilmez.
- **Fiziksel Cihaz Testi:** Öğrenciler kendi telefonlarını bağlayıp uygulamayı çalıştırdıktan sonra butona basmalı ve Android Studio ekranındaki mesajları takip etmelidir. Bu, "yazdığım kod fiziksel bir cihazda nasıl bir eyleme dönüşüyor?" sorusunun en somut cevabıdır.

Dersin bir sonraki 5 dakikasında, 2. haftayı bitirirken öğrendiğimiz tüm bu yapıları (değişkenler, döngüler, kararlar ve lambdalar) içeren kapsamlı bir "Haftalık Değerlendirme Uygulaması" tasarlayabiliriz. Bir sonraki aşamaya geçmemi ister misiniz?

You: 2. haftanın 3. dersinin 35. dakikası ile 40. dakikası arasını samimi olmadan, kolay anlaşılır ve akıcı bir şekilde açıklayan bir ders notu hazırla. Ders notunda her detay olsun. Oluşturulan kodlar da tümüyle yani MainActivity.kt olarak metinler de yer alsın.

Gemini: ## 2. Hafta / 3. Ders: Veri Grupları (Data Classes) ve Dinamik Liste Tasarımı (35:00 - 40:00)

Dersin bu bölümünde, birbiriyle ilişkili verileri (örneğin bir ürünün adı ve fiyatı) bir arada tutmamızı sağlayan **Data Class (Veri Sınıfı)** yapısı ve bu yapıdaki verileri döngülerle nasıl ekrana yansıtacağımız ele alınacaktır. Bu, modern Android uygulamalarında veri modellerinin oluşturulması için en temel adımdır.

0.23.13. 1. Data Class (Veri Sınıfı) Mantığı

Bir uygulamada sadece tek bir değişken (örneğin sadece isim) çoğu zaman yeterli olmaz. Bir "Öğrenci" dendiğinde adı, numarası ve notu gibi birden fazla verinin tek bir paket içinde olması istenir. Kotlin'de bu paketleme işlemi `data class` ile yapılır.

- **Yapısı:** `data class SinifAdi(val degisken: Tip, ...)`
- **Avantajı:** Verileri gruplayarak kodun daha düzenli olmasını sağlar. Tek bir liste içinde hem ismi hem de sayıyı aynı anda taşıyabiliriz.

0.23.14. 2. Nesne Listesi ve Arayüz Entegrasyonu

Oluşturduğumuz bu veri gruplarını bir liste içerisine koyduğumuzda (`listOf`), her bir eleman artık birden fazla özellik barındırır. Jetpack Compose içerisinde bir döngü kullanarak bu listenin her bir özelliğine ayrı ayrı erişip (Örn: `it.ad`, `it.fiyat`) farklı görsel özellikler atayabiliriz.

0.23.15. 3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kodda, bir "Alışveriş Listesi" senaryosu işlenmiştir. Ürünlerin adı ve fiyatı bir `data class` içerisinde tutulmakta, fiyatı yüksek olan ürünler otomatik olarak kırmızı renk ile vurgulanmaktadır.

```
package com.example.verigruplari

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
```

```
17 import androidx.compose.ui.unit.dp
18 import androidx.compose.ui.unit.sp
19
20 // 1. Veri Grubunun (Data Class) Tanımlanması
21 // Bu yapı her bir ürünün hangi özelliklere sahip olacağını belirler.
22 data class Urun(
23     val ad: String,
24     val fiyat: Double
25 )
26
27 class MainActivity : ComponentActivity() {
28     override fun onCreate(savedInstanceState: Bundle?) {
29         super.onCreate(savedInstanceState)
30         setContent {
31             Surface(
32                 modifier = Modifier.fillMaxSize(),
33                 color = MaterialTheme.colorScheme.background
34             ) {
35                 // Dinamik alışveriş listesi fonksiyonunu çağırıyoruz
36                 AlisverisListesiEkrani()
37             }
38         }
39     }
40 }
41
42 @Composable
43 fun AlisverisListesiEkrani() {
44     // 2. Veri Setinin Oluşturulması
45     // Liste içerisinde 'Urun' tipinde nesneler saklıyoruz.
46     val urunler = listOf(
47         Urun("Ekmek", 10.0),
48         Urun("Süt", 35.5),
49         Urun("Kahve", 120.0),
50         Urun("Peynir", 95.0),
51         Urun("Çikolata", 15.0)
52     )
53
54     Column(modifier = Modifier.padding(20.dp)) {
55         Text(
56             text = "Güncel Alışveriş Sepeti",
57             fontSize = 22.sp,
58             fontWeight = FontWeight.Bold,
59             color = Color(0xFFE91E63)
60         )
61         Text(text = "-------------------------", modifier =
           Modifier.padding(vertical = 8.dp))
```

```

        // 3. Döngü ve Karar Yapısının Nesne Üzerinde Kullanımı
        for (item in urunler) {
            Column(modifier = Modifier.padding(vertical = 6.dp)) {

                // Ürün adını gösteriyoruz
                Text(text = "Ürün: ${item.ad}", fontSize = 18.sp, fontWeight = FontWeight.Medium)

                // Fiyata göre dinamik renk belirleme (100 TL üzeri kırmızı)
                val fiyatRengi = if (item.fiyat > 100) Color.Red else Color(0xFF4CAF50)

                Text(
                    text = "Fiyat: ${item.fiyat} TL",
                    color = fiyatRengi,
                    fontWeight = if (item.fiyat > 100) FontWeight.ExtraBold else FontWeight.Normal
                )

                // Kritik bir uyarı (if kullanımı)
                if (item.fiyat > 100) {
                    Text(
                        text = "⚠ Yüksek fiyat uyarısı!",
                        fontSize = 12.sp,
                        color = Color.DarkGray
                    )
                }

                Divider(modifier = Modifier.padding(top = 8.dp), thickness = 0.5.dp)
            }
        }
    }
}
```

0.23.16. 4. Teknik Notlar ve Uygulama Detayları

- **Neden Data Class?:** Eğer verilerimizi sadece düz listeler halinde (`val isimler`, `val fiyatlar`) tutsaydık, araya yeni bir veri girdiğinde listeler arasındaki uyum bozulabilirdi. Data class kullanımı, her ismin kendi fiyatıyla her zaman "bağlı" kalmasını garanti eder.
- **Dinamik Stil:** Kodda görüldüğü üzere `fontWeight` (yazı kalınlığı) bile bir `if` koşuluna bağlanmıştır. Bu, Jetpack Compose'un veriye dayalı tasarım (Data-Driven Design) gücünü gösterir.

- **Fiziksel Cihaz Testi:** Öğrenciler kendi telefonlarında uygulamayı çalıştırdıklarında, `urunler` listesine 100 TL'den büyük ve küçük yeni ürünler ekleyerek ekranın nasıl otomatik olarak şekillendiğini ve renklerin nasıl değiştiğini izlemelidir.
- **Logcat Takibi:** Eğer bir ürünün fiyatı yanlış hesaplanıyorsa, döngü içinde `println("Hesaplanan: ${item.ad}")` komutu eklenerek Android Studio üzerinden veri akışı takip edilebilir.

Dersin son 5 dakikalık bölümünde, 2. haftanın genel bir tekrarını yaparak öğrencilerin kendi telefonlarında çalıştıracağı küçük bir "Haftalık Kapanış Uygulaması" tasarlamamı ister misiniz?

You: 2. haftanın 3. dersinin 40. dakikası ile 45. dakikası arasını samimi olmadan, kolay anlaşılır ve akıcı bir şekilde açıklayan bir ders notu hazırla. Ders notunda her detay olsun. Oluşturulan kodlar da tümüyle yani MainActivity.kt olarak metinler de yer alsın.

Gemini: ## 2. Hafta / 3. Ders: Kotlin Temellerinin Sentezi ve Haftalık Kapanış Uygulaması (40:00 - 45:00)

Dersin bu son 5 dakikasında, hafta boyunca parça parça işlediğimiz Değişkenler, Null Güvenliği, Karar Yapıları, Döngüler, Fonksiyonlar ve Veri Sınıfları (Data Classes) tek bir projede birleştirilecektir. Bu sentez, bir Android uygulamasının mantık katmanı (Kotlin) ile sunum katmanının (Jetpack Compose) nasıl bir bütün olarak çalıştığını anlamanızı sağlayacaktır.

0.23.17. 1. Uygulama Mimarisi ve Veri Akışı

Bir mobil uygulama temel olarak şu üç aşamadan oluşur:

1. **Veri Modelleme:** `data class` ile verilerin yapısı belirlenir.
2. **Mantık İşleme:** Fonksiyonlar ve kontrol yapıları ile veriler analiz edilir (Örn: Geçti/Kaldı kontrolü).
3. **Arayüz Sunumu:** İşlenen veriler, döngüler aracılığıyla `@Composable` bileşenlere aktarılarak telefonda görüntülenir.

0.23.18. 2. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod, 2. haftanın tüm kazanımlarını içeren eksiksiz bir "Öğrenci Not Sistemi" simülasyonudur. Bu kodu doğrudan Android Studio projenize yapıştırıp telefonunuzda test edebilirsiniz.

```
package com.example.haftalıkkapanis

import android.os.Bundle
import android.util.Log
import androidx.activity.ComponentActivity
```

```
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

// 1. Veri Modeli
data class Ogrenci(val ad: String, val not: Int)

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                // Ana uygulama ekranını başlatıyoruz
                HaftalikKapanisEkrani()
            }
        }
    }
}

@Composable
fun HaftalikKapanisEkrani() {
    // 2. Değişkenler ve Null Güvenliği
    val dersAdi: String = "Android Programlama"
    val ogretmenAdi: String? = null // Henüz atanmamış eğitmen simülasyonu

    // 3. Veri Listesi (Data Class ve List kullanımı)
    val ogrenciListesi = listOf(
        Ogrenci("Ali Yılmaz", 85),
        Ogrenci("Ayşe Demir", 42),
        Ogrenci("Mehmet Kaya", 70),
        Ogrenci("Fatma Çelik", 30)
```

```
52      )
53
54      Column(modifier = Modifier.padding(20.dp)) {
55          // Başlık Bölümü
56          Text(text = dersAdi, fontSize = 24.sp, fontWeight = FontWeight.Bold)
57
58          // Null Güvenliği (Elvis Operatörü)
59          Text(
60              text = "Eğitmen: ${ogretmenAdi ?: "Bilinmiyor"}",
61              color = Color.Gray,
62              modifier = Modifier.padding(bottom = 16.dp)
63          )
64
65          Divider(thickness = 2.dp, color = Color.Black)
66
67          // 4. Döngü ve Karar Yapısı Entegrasyonu
68          for (ogrenci in ogrenciListesi) {
69              OgrenciSatiri(ogrenci)
70          }
71
72          // 5. Lambda ve Etkileşim
73          Button(
74              onClick = {
75                  Log.d("Kapanis", "Rapor telefon üzerinden oluşturuldu.")
76              },
77              modifier = Modifier.padding(top = 20.dp)
78          ) {
79              Text(text = "Sistem Raporu Al (Logcat)")
80          }
81      }
82  }
83
84  /**
85   * Bir öğrenci nesnesini parametre olarak alan ve
86   * nota göre renk belirleyen görsel fonksiyon.
87   */
88  @Composable
89  fun OgrenciSatiri(ogrenci: Ogrenci) {
90      // Harf notu belirleme (when yapısı)
91      val harfNotu = when (ogrenci.not) {
92          in 85..100 -> "AA"
93          in 70..84 -> "BB"
94          in 50..69 -> "CC"
95          else -> "FF"
96      }
97
```

```
    // Renk belirleme (if-else)
    val renk = if (ogrenci.not >= 50) Color(0xFF388E3C) else Color.Red

    Column(modifier = Modifier.padding(vertical = 8.dp)) {
        Text(text = "Öğrenci: ${ogrenci.ad}", fontWeight = FontWeight.Medium)
        Text(
            text = "Not: ${ogrenci.not} - Harf: $harfNotu",
            color = renk,
            fontWeight = FontWeight.Bold
        )
        Divider(modifier = Modifier.padding(top = 4.dp), thickness = 0.5.dp)
    }
}
```

0.23.19. 3. Uygulama Notları ve Teknik Detaylar

- **Fiziksel Cihazda Gözlem:** Kodun en altındaki "Sistem Raporu Al" butonuna bastığınızda, Android Studio'nun alt kısmındaki **Logcat** sekmesinde mesajın anlık olarak belirdiğini göreceksiniz. Emülatör yerine kendi telefonunuzu kullanmanız, bu veri alışverişinin hızını doğrudan deneyimlemenizi sağlar.
- **Modüler Yapı:** `OgrenciSatiri` isimli fonksiyon, karmaşık bir arayüzü nasıl daha küçük parçalara bölebileceğimizi göstermektedir. Bu yöntem, kodun okunabilirliğini artırır.
- **Veriye Dayalı Tasarım:** Uygulama ekranındaki hiçbir renk veya metin rastgele değildir; tamamı `ogrenciListesi` içindeki verilere bağlı olarak `if` ve `when` yapıları tarafından belirlenmiştir.
- **Hata Denetimi:** `ogretmenAdi` değişkenine bir isim yazdığınızda veya `null` bıraktığınızda ekranın nasıl tepki verdiğini inceleyerek Kotlin'in çökme koruması sağlayan Null Güvenliği mekanizmasını test edebilirsiniz.

Bu dersle birlikte Android geliştirme yolculuğunun temel "mantık" aşamasını tamamlamış bulunuyoruz. Bir sonraki hafta, bu mantığı daha estetik ve profesyonel bir görünüme kavuşturmak için **Layout (Düzen) Yönetimi** konusuna başlayacağız.

1. haftanın ilk dersi olan **"Column, Row ve Box ile Sayfa Düzeni"** içeriği için hazırlık yapmamı ister misiniz?

You: 2. haftanın 4. dersinin ilk 5 dakikasını samimi olmadan, kolay anlaşılır ve akıcı bir şekilde açıklayan bir ders notu hazırla. Ders notunda her detay olsun. Oluşturulan kodlar da tümüyle yani MainActivity.kt olarak metinler de yer alsın.

Gemini: ## 2. Hafta / 4. Ders: Kotlin Mantığı ile Arayüz Etkileşimi (00:00 - 05:00)

Bu dersin ilk 5 dakikasında, hafta boyunca öğrendiğimiz tüm Kotlin temellerinin (değişkenler, fonksiyonlar, karar yapıları ve döngüler) Jetpack Compose içerisinde nasıl bir

“etkileşim” yarattığı ele alınacaktır. Programlama mantığı ile görsel tasarımın birleştiği bu noktada, verinin nasıl işlendiğini ve fiziksel cihaza nasıl yansıtıldığını analiz edeceğiz.

0.23.20. 1. Kotlin ve Compose Arasındaki Köprü

Jetpack Compose bildirimsel (declarative) bir mimari kullanır. Bu mimaride, yazdığınız Kotlin kodundaki veriler (data), UI fonksiyonlarını besleyen birer kaynaktır. Kod içerisinde yaptığınız her matematiksel işlem veya mantıksal sorgu, telefon ekranındaki görüntünün saniyeler içinde şekillenmesini sağlar.

- **Veri Yönetimi:** Değişkenler (val/var) veriyi tutar.
- **Mantık Yönetimi:** `if-else` ve `when` yapıları hangi bileşenin çizileceğine karar verir.
- **Tekrar Yönetimi:** Döngüler (`for`) listenin her elemanı için bir görsel şablon oluşturur.

0.23.21. 2. Fonksiyonel Bileşen Yapısı

Jetpack Compose projelerinde her bir görsel parça bir `@Composable` fonksiyonudur. Bu fonksiyonlar parametre alarak çalışır. Örneğin, bir “Ders Kartı” fonksiyonu yazıp içine dersin adını ve saatini gönderdiğimizde, fonksiyon bu veriyi alıp ekranda uygun yerlere yerleştirir. Bu yapı, kodun tekrar kullanılabilirliğini sağlar ve hata payını düşürür.

0.23.22. 3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod, hafta boyunca işlenen tüm kavramların bir sentezidir. Bir “Akademik Görev Paneli” simülasyonu içermektedir. Bu kodda veri sınıfı (Data Class), liste (List), döngü (For Loop) ve karar yapıları (When) bir arada kullanılmıştır.

```
package com.example.hafta2final

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

// 1. Veri Yapısı: Bir görevin temel özelliklerini gruplar
```

```
22  data class Gorev(val baslik: String, val oncelik: Int)
23
24  class MainActivity : ComponentActivity() {
25      override fun onCreate(savedInstanceState: Bundle?) {
26          super.onCreate(savedInstanceState)
27          setContent {
28              // Sistemin arka plan ve tema ayarları
29              Surface(
30                  modifier = Modifier.fillMaxSize(),
31                  color = MaterialTheme.colorScheme.background
32              ) {
33                  // Haftalık değerlendirme ekranını başlatır
34                  AkademikPanelEkrani()
35              }
36          }
37      }
38  }
39
40  @Composable
41  fun AkademikPanelEkrani() {
42      // 2. Veri Hazırlığı
43      val kullaniciAdi: String = "Hasan Hüseyin Ökten"
44      val bildirimVarMi: Boolean = true
45
46      // Görev listesi (Data Class ve List entegrasyonu)
47      val gorevler = listOf(
48          Gorev("Sınav Kağıdı Okuma", 1), // 1: Kritik
49          Gorev("Müfredat Güncelleme", 2), // 2: Normal
50          Gorev("Evrak İmzalama", 3)      // 3: Düşük
51      )
52
53      Column(modifier = Modifier.padding(20.dp)) {
54          // Kullanıcı selamlama (Metin şablonu kullanımı)
55          Text(text = "Hoş Geldiniz,", fontSize = 16.sp)
56          Text(
57              text = kullaniciAdi,
58              fontSize = 20.sp,
59              fontWeight = FontWeight.Bold,
60              color = Color(0xFF1976D2)
61          )
62
63          Divider(modifier = Modifier.padding(vertical = 12.dp))
64
65          // 3. Karar Yapısı (if) ile Durum Kontrolü
66          if (bildirimVarMi) {
67              Text(
```

```
                    text = "🔔 Tamamlanması gereken görevleriniz var.",
                    color = Color(0xFFD32F2F),
                    modifier = Modifier.padding(bottom = 8.dp)
                )
            }

            // 4. Döngü (for) ile Listenin Görselleştirilmesi
            for (item in gorevler) {
                GorevSatiriOlustur(item)
            }

            // 5. Lambda ve Etkileşim: Butona tıklandığında yapılacak işlem
            Button(
                onClick = { /* Tıklama eylemi Logcat'te izlenebilir */ },
                modifier = Modifier.padding(top = 16.dp)
            ) {
                Text(text = "Tümünü Tamamlandı İşaretle")
            }
        }
    }

    /**
     * Görev nesnesini alıp öncelik sırasına göre renklendiren yardımcı fonksiyon
     */
    @Composable
    fun GorevSatiriOlustur(gorev: Gorev) {
        // Karar Yapısı (when) ile görsel özellik belirleme
        val oncelikRengi = when (gorev.oncelik) {
            1 -> Color.Red
            2 -> Color.Blue
            else -> Color.Gray
        }

        val oncelikMetni = when (gorev.oncelik) {
            1 -> "ACİL"
            2 -> "Normal"
            else -> "Düşük"
        }

        Column(modifier = Modifier.padding(vertical = 6.dp)) {
            Text(text = gorev.baslik, fontWeight = FontWeight.Medium)
            Text(
                text = "Öncelik: $oncelikMetni",
                color = oncelikRengi,
                fontSize = 12.sp,
                fontWeight = FontWeight.Bold
```

```
        )
        Divider(modifier = Modifier.padding(top = 4.dp), thickness = 0.5.dp)
    }
}
```

0.23.23. 4. Teknik Notlar ve Uygulama Detayları

- **Fiziksel Cihaz Bağlantısı:** Yazılan bu kodun sonuçlarını görmek için Android Studio üzerinden telefonunuzun modelini seçip "Run" tuşuna basınız. Emülatör kullanılmadığı için telefon ekranındaki her bir `Text` ve `Button` bileşeni doğrudan donanım üzerinde çizilecektir.
- **Hata Denetimi:** Kod içerisinde yer alan `GorevSatiriOlustur` fonksiyonuna hatalı bir öncelik rakamı (örneğin 5) gönderilirse, `when` yapısındaki `else` bloğu devreye girerek "Düşük" ve "Gri" rengi varsayılan olarak atayacaktır. Bu, uygulamanın çökmesini engelleyen bir güvenlik önlemidir.
- **Öğrenci Takibi:** Öğrenciler, kendi telefonlarında `kullaniciAdi` değişkenini değiştirerek veya `bildirimVarMi` değerini `false` yaparak ekranın nasıl dinamik olarak güncellendiğini bizzat deneyimlemelidir.

Dersin bir sonraki 5 dakikalık bölümünde, bu statik listeyi daha etkileşimli hale getirmek için kullanıcıdan veri alma yöntemlerini tartışmamı ister misiniz?

You: 2. haftanın 4. dersinin 5. dakikası ile 10. dakikası arasını samimi olmadan, kolay anlaşılır ve akıcı bir şekilde açıklayan bir ders notu hazırla. Ders notunda her detay olsun. Oluşturulan kodlar da tümüyle yani MainActivity.kt olarak metinler de yer alsın.

Gemini: ## 2. Hafta / 4. Ders: Koşullu Arayüz Yapıları ve Dinamik Görselleştirme (05:00 - 10:00)

Dersin bu bölümünde, Kotlin'deki karar yapılarının (if-else) Jetpack Compose içerisinde sadece metin değiştirmek için değil, arayüzün tüm yapısını değiştirmek için nasıl kullanıldığı üzerinde durulacaktır. "Koşullu Çizim" (Conditional Rendering) olarak adlandırılan bu kavram, modern Android uygulamalarının dinamik doğasını oluşturur.

0.23.24. 1. Koşullu Çizim ve Görünürlük Mantığı

Geleneksel Android (XML) sisteminde bir bileşeni gizlemek için o bileşenin özelliklerinden birini (visibility) değiştirmek gerekirken, Jetpack Compose'da durum farklıdır. Bir `if` bloğu içerisine yazılan Composable fonksiyonu, eğer koşul sağlanmıyorsa sistem tarafından hiç çalıştırılmaz ve telefonun belleğinde (RAM) yer tutmaz.

- **Dinamik Yapı:** Koşul sağlandığında bileşen anında ekrana eklenir.
- **Bellek Yönetimi:** Çizilmeyen bileşenler işlemciyi yormaz.

- **Parametrik Değişim:** Sadece bileşenin kendisi değil, rengi, boyutu veya metin stili de bir `if` sonucuna bağlanabilir.

0.23.25. 2. Değişkenler ile Stil Yönetimi

Arayüz elemanlarının görsel özellikleri (renk, kalınlık vb.), doğrudan Kotlin değişkenleri veya bu değişkenleri işleyen mantıksal sorgularla belirlenebilir. Örneğin, bir öğrencinin devamsızlık durumu kritik seviyeye ulaştığında metnin rengini otomatik olarak kırmızıya çeviren bir yapı, kodun içine gömülü bir `if` ifadesi ile sağlanır.

0.23.26. 3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod, bir öğrencinin akademik durumunu ve ders kayıt kontrolünü yapan tam bir Android uygulama dosyasıdır. Kod içerisinde `if-else` yapısının arayüzün hem içeriğini hem de stilini nasıl kontrol ettiği detaylandırılmıştır.

```
package com.example.hafta2ders4

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                // Akademik durum takip ekranını başlatır
                AkademikDurumEkrani()
            }
        }
```

```
    }
}

@Composable
fun AkademikDurumEkrani() {
    // 1. Veri Hazırlığı
    val ogrenciAdi = "Hasan Hüseyin Ökten"
    val ortalama = 48
    val kayitYenilemeYapildiMi = false

    Column(modifier = Modifier.padding(20.dp)) {
        Text(
            text = "Öğrenci Bilgi Sistemi",
            fontSize = 22.sp,
            fontWeight = FontWeight.Bold,
            color = Color(0xFF1976D2)
        )

        Text(text = "Öğrenci: $ogrenciAdi", modifier = Modifier.padding(top = 8.dp))

        Divider(modifier = Modifier.padding(vertical = 12.dp))

        // 2. Stil Üzerinde Koşul Kullanımı
        // Ortalama 50'den küçükse metin kırmızı, değilse yeşil olur
        val durumRengi = if (ortalama < 50) Color.Red else Color(0xFF388E3C)
        val durumMetni = if (ortalama < 50) "Başarısız" else "Başarılı"

        Text(
            text = "Akademik Ortalama: $ortalama",
            fontSize = 18.sp
        )

        Text(
            text = "Genel Durum: $durumMetni",
            color = durumRengi,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(bottom = 16.dp)
        )

        // 3. Arayüzün Tamamını Etkileyen Karar Yapısı
        if (kayitYenilemeYapildiMi) {
            // Koşul true ise bu bloktaki tüm bileşenler çizilir
            Text(
                text = "✅ Ders kayıtları aktif. Ders seçimi yapabilirsiniz.",
                color = Color(0xFF388E3C)
```

```
            )
        } else {
            // Koşul false ise kullanıcıya uyarı paneli gösterilir
            Column {
                Text(
                    text = "⚠ Kayıt Yenileme Bekleniyor!",
                    color = Color.Red,
                    fontWeight = FontWeight.Black
                )
                Text(
                    text = "Lütfen harç ödemenizi kontrol ediniz.",
                    fontSize = 14.sp,
                    color = Color.Gray
                )
            }
        }
    }
}
```

0.23.27. 4. Teknik Ayrıntılar ve Uygulama Notları

- **Değişkenlerin Rolü:** Kodda tanımlanan `durumRengi` değişkeni, doğrudan bir `if` ifadesinden değer almaktadır. Bu yöntem, Composable fonksiyonun gövdesini temiz tutar ve görsel mantığın okunabilirliğini artırır.
- **Hiyerarşik Yapı:** `if (kayitYenilemeYapildiMi)` bloğu içerisinde bir `Column` (dikey dizilim) kullanılmıştır. Bu, bir koşula bağlı olarak birden fazla arayüz elemanının bir grup halinde ekrana getirilmesini sağlar.
- **Fiziksel Cihaz Gözlemi:** Öğrenciler kendi telefonlarında uygulamayı çalıştırdıktan sonra `ortalama` değerini 75 yaparak `durumRengi` ve `durumMetni` değişkenlerinin nasıl anlık olarak değiştiğini; `kayitYenilemeYapildiMi` değerini `true` yaparak alt kısımdaki uyarı panelinin yerini nasıl onay mesajına bıraktığını gözlemlemelidir.
- **Emülatörsüz Çalışma:** Cihaz üzerindeki tepki hızı, Kotlin kodunun ne kadar optimize çalıştığının bir göstergesidir. Öğrencilerin USB bağlantılarını kontrol etmeleri ve kod güncellendikten sonra “Apply Changes” (Değişiklikleri Uygula) butonunu kullanmaları teşvik edilmelidir.

Dersin bir sonraki 5 dakikalık bölümünde, birden fazla seçeneği yönetmek için kullanılan `when` yapısının arayüzdeki gelişmiş uygulamalarını inceleyeceğiz. Bir sonraki aşamaya geçmemi ister misiniz?

You: 2. haftanın 4. dersinin 10. dakikası ile 15. dakikası arasını samimi olmadan, kolay anlaşılır ve akıcı bir şekilde açıklayan bir ders notu hazırla. Ders notunda her detay olsun. Oluşturulan kodlar da tümüyle yani MainActivity.kt olarak metinler de yer alsın.

Gemini: ## 2. Hafta / 4. Ders: “when” İfadesi ile Gelişmiş Durum Yönetimi (10:00 - 15:00)

Dersin bu bölümünde, ikiden fazla durumun kontrol edilmesi gereken senaryolarda `if-else` yapısının yerini alan ve kodun okunabilirliğini zirveye taşıyan **when** yapısı ele alınacaktır. Jetpack Compose içerisinde `when`, bir ekranın farklı hallerini (State) yönetmek için en profesyonel yaklaşımdır.

0.23.28. 1. “when” Yapısının Kontrol Mekanizması

Kotlin’deki `when` yapısı, bir değişkenin değerini alır ve bu değeri tanımlanan seçeneklerle karşılaştırır. Eşleşen seçenek bulunduğunda ilgili kod bloğu çalıştırılır. Diğer dillerdeki `switch-case` yapısına benzese de çok daha esnektir; aralıkları, veri tiplerini ve fonksiyon sonuçlarını kontrol edebilir.

- **İfade Olarak Kullanım:** `when` bir değer döndürebilir. Bu sayede bir değişkenin değerini (örneğin bir rengi veya metni) doğrudan `when` sonucuna eşitleyebiliriz.
- **else Bloğu:** Tüm ihtimallerin kapsanmadığı durumlarda `else` kullanımı zorunludur. Bu, uygulamanın beklenmedik bir veri geldiğinde çökmesini engeller.

0.23.29. 2. Jetpack Compose ve Durum Dallanması

Bir Android uygulamasında ekran genellikle statik değildir. Veri beklerken “Yükleniyor”, veri geldiğinde “Liste”, bir sorun çıktığında “Hata” ekranı gösterilir. `when` yapısı, bu üç farklı ekranı tek bir fonksiyon içinde, tertemiz bir hiyerarşiyle yönetmemizi sağlar.

0.23.30. 3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod, bir “Laboratuvar Durum Takip” sistemi simülasyonudur. Bilgisayar laboratuvarının doluluk veya teknik durumuna göre ekranın temasını ve içeriğini tamamen değiştiren bir yapı kurulmuştur.

```
package com.example.hafta2ders4

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
```

```
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                // Laboratuvar takip ekranını başlatır
                LaboratuvarTakipEkrani()
            }
        }
    }
}

@Composable
fun LaboratuvarTakipEkrani() {
    // 1. Veri Tanımlama (Durum kodları: 0: Bakım, 1: Müsait, 2: Derste)
    val labDurumKodu = 1
    val labAdi = "Bilgisayar Laboratuvarı - 1"

    Column(modifier = Modifier.padding(24.dp)) {
        Text(
            text = "Bölüm Yönetim Sistemi",
            fontSize = 18.sp,
            color = Color.Gray
        )
        Text(
            text = labAdi,
            fontSize = 22.sp,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(bottom = 16.dp)
        )

        Divider(modifier = Modifier.padding(bottom = 16.dp))

        // 2. when İfadesi ile Dinamik Renk ve Metin Belirleme
        // when burada bir değer döndürmek (expression) için kullanılıyor
        val durumRengi = when (labDurumKodu) {
            0 -> Color.Red         // Bakım
            1 -> Color(0xFF388E3C) // Müsait (Yeşil)
            2 -> Color(0xFF1976D2) // Derste (Mavi)
            else -> Color.Black    // Bilinmeyen durum
```

```
        }

        val durumAciklamasi = when (labDurumKodu) {
            0 -> "⚠ Laboratuvar teknik bakım nedeniyle kapalıdır."
            1 -> "✅ Laboratuvar şu an kullanıma uygundur."
            2 -> "📋 Bu saatte aktif ders işlenmektedir."
            else -> "? Durum bilgisi alınamadı."
        }

        // 3. Arayüz Elemanlarının Çizilmesi
        Text(
            text = "Güncel Durum:",
            fontSize = 16.sp,
            fontWeight = FontWeight.Medium
        )

        Text(
            text = durumAciklamasi,
            color = durumRengi,
            fontSize = 18.sp,
            fontWeight = FontWeight.ExtraBold,
            modifier = Modifier.padding(top = 8.dp)
        )

        // 4. Ek Bilgiler (Sadece belirli durumlarda gösterilir)
        if (labDurumKodu == 2) {
            Text(
                text = "Lütfen gürültü yapmayınız.",
                fontSize = 14.sp,
                modifier = Modifier.padding(top = 12.dp),
                color = Color.Gray
            )
        }
    }
}
```

0.23.31. 4. Teknik Ayrıntılar ve Uygulama Notları

- **Bloksuz Yazım:** Eğer `when` dallarında sadece tek bir satır kod (örneğin sadece renk ataması) varsa, süslü parantez kullanmadan doğrudan `→` işaretinden sonra değeri yazabilirsiniz. Bu, kodu oldukça sadeleştirir.
- **Hata Payı:** `else` bloğu, bir veritabanı hatası veya yanlış veri girişi olduğunda uygulamanın siyah ekranda kalmasını veya çökmesini engelleyerek "Savunmacı Programlama" yapmamızı sağlar.

- **Fiziksel Cihaz Testi:** Öğrenciler kendi telefonlarında uygulamayı çalıştırdıktan sonra `labDurumKodu` değişkenini `0` ve `2` yaparak ekranın nasıl bir "Mod Değişimi" yaşadığını gözlemlemelidir.
- **Önemli Hatırlatma:** `labDurumKodu` değiştikçe Jetpack Compose ekranı "Recomposition" (Yeniden Oluşturma) sürecine sokar. Bu sayede `durumRengi` ve `durumAciklamasi` değişkenleri yeni değere göre tekrar hesaplanır ve ekran saniyeler içinde güncellenir.

Dersin bir sonraki 5 dakikasında, bu yapıları listelerle birleştirerek birden fazla laboratuvarın durumunu aynı anda gösteren bir "Döngüsel Panel" tasarlamamı ister misiniz?

You: 2. haftanın 4. dersinin 15. dakikası ile 20. dakikası arasını samimi olmadan, kolay anlaşılır ve akıcı bir şekilde açıklayan bir ders notu hazırla. Ders notunda her detay olsun. Oluşturulan kodlar da tümüyle yani MainActivity.kt olarak metinler de yer alsın.

Gemini: ## 2. Hafta / 4. Ders: Veri Listeleri ve Dinamik Arayüz Oluşturma (15:00 - 20:00)

Dersin bu bölümünde, Kotlin'deki **Data Class (Veri Sınıfı)** yapısının listelerle (`List`) birleştirilerek Jetpack Compose içerisinde nasıl dinamik bir arayüz döngüsüne dönüştürüldüğü ele alınacaktır. Önceki bölümlerde tekil veriler üzerinde durmuştuk; şimdi ise çoklu veri setlerini profesyonel bir yapıda nasıl organize edeceğimizi inceleyeceğiz.

0.23.32. 1. Veri Gruplama: Data Class ve List Entegrasyonu

Bir Android uygulamasında veriler genellikle birbirine bağlı gruplar halindedir. Örneğin, bir "Ders" nesnesi; adı, kredisi ve zorunlu olup olmadığı bilgisini aynı anda taşır. `data class` kullanarak bu verileri bir paket haline getiririz. Ardından `listOf` ile bu paketlerden oluşan bir koleksiyon oluştururuz.

0.23.33. 2. Yinelemeli Arayüz (Iterative UI) Mantığı

Jetpack Compose'da bir listeyi ekrana basmak için standart Kotlin `for` döngüsünü kullanırız. Döngü, listenin her bir elemanını sırayla ele alır ve her adımda o elemana özel bir görsel bileşen (örneğin bir `Text` veya `Row`) oluşturur. Bu yöntem, 100 elemanlı bir listeyi bile sadece birkaç satır kodla ekrana yansıtmamızı sağlar.

0.23.34. 3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod örneği, bir "Akademik Müfredat Paneli" simülasyonudur. Veri sınıfı içinde tutulan ders bilgileri, döngü aracılığıyla okunmakta ve dersin zorunluluk durumuna göre farklı görsel özelliklerle ekrana yansıtılmaktadır.

```
package com.example.hafta2ders4

import android.os.Bundle
```

```
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

// 1. Veri Yapısı (Data Class)
// Her bir dersin özelliklerini tanımlayan şablon
data class AkademikDers(
    val dersAdi: String,
    val kredi: Int,
    val zorunluMu: Boolean
)

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                // Dinamik ders listesi ekranını başlatır
                MufredatListesiEkrani()
            }
        }
    }
}

@Composable
fun MufredatListesiEkrani() {
    // 2. Veri Setinin Hazırlanması (Koleksiyon)
    val dersListesi = listOf(
        AkademikDers("Nesne Yönelimli Programlama", 5, true),
        AkademikDers("Web Teknolojileri", 4, true),
        AkademikDers("Girişimcilik", 2, false),
```

```
        AkademikDers("Veri Yapıları", 5, true),
        AkademikDers("Yabancı Dil - II", 2, false)
    )

    Column(modifier = Modifier.padding(24.dp)) {
        Text(
            text = "Bölüm Müfredat Paneli",
            fontSize = 22.sp,
            fontWeight = FontWeight.Bold,
            color = Color(0xFF1976D2)
        )
        Text(
            text = "Dönemlik aktif ders listesi aşağıdadır.",
            fontSize = 14.sp,
            color = Color.Gray,
            modifier = Modifier.padding(bottom = 16.dp)
        )

        Divider(modifier = Modifier.padding(bottom = 12.dp))

        // 3. Döngü (for) ile Listenin Dinamik Olarak Çizilmesi
        // dersListesi içindeki her bir 'ders' nesnesi için bu blok çalışır
        for (ders in dersListesi) {

            Column(modifier = Modifier.padding(vertical = 8.dp)) {

                Text(
                    text = ders.dersAdi,
                    fontSize = 18.sp,
                    fontWeight = FontWeight.Medium
                )

                // Karar Yapısı (if) ile Zorunluluk Durumuna Göre Stil Belirleme
                val durumMetni = if (ders.zorunluMu) "Zorunlu Ders" else "Seçmeli Ders"
                val durumRengi = if (ders.zorunluMu) Color(0xFFD32F2F) else Color(0xFF388E3C)

                Text(
                    text = "$durumMetni | Kredi: ${ders.kredi}",
                    color = durumRengi,
                    fontSize = 14.sp,
                    fontWeight = FontWeight.Bold
                )

```

```
                    Divider(
                        modifier = Modifier.padding(top = 10.dp),
                        thickness = 0.5.dp,
                        color = Color.LightGray
                    )
                }
            }
        }
}
```

0.23.35. 4. Teknik Notlar ve Uygulama Detayları

- **Veri Yönetimi:** `dersListesi` içerisindeki her bir eleman `AkademikDers` tipindedir. Bu sayede döngü içerisinde `ders.dersAdi` veya `ders.kredi` yazarak ilgili veriye güvenli bir şekilde erişebiliriz.
- **Ölçeklenebilirlik:** Listeye yeni bir ders eklediğinizde, arayüz kodunda (`for` döngüsü kısmında) herhangi bir değişiklik yapmanıza gerek kalmaz. Uygulama, yeni elemanı otomatik olarak algılar ve ekrana basar.
- **Fiziksel Cihaz Gözlemi:** Öğrenciler kendi telefonlarında uygulamayı çalıştırdıklarında, listedeki `zorunluMu` değerini `false` yaparak kırmızı renkli metinlerin nasıl yeşile döndüğünü ve metin içeriğinin nasıl değiştiğini anlık olarak test etmelidir.
- **Hiyerarşik Düzen:** `for` döngüsü içerisindeki `Column` ve `Divider` yapısı, her bir listenin görsel olarak birbirinden ayrılmasını sağlar. Bu, kullanıcı deneyimi (UX) açısından listenin okunabilirliğini artıran kritik bir detaydır.

Dersin bir sonraki 5 dakikalık bölümünde, döngülerin akışını kontrol eden `break` ve `continue` komutlarının listeler üzerindeki stratejik kullanımını inceleyeceğiz. Bir sonraki aşamaya geçmemi ister misiniz?

You: 2. haftanın 4. dersinin 20. dakikası ile 25. dakikası arasını samimi olmadan, kolay anlaşılır ve akıcı bir şekilde açıklayan bir ders notu hazırla. Ders notunda her detay olsun. Oluşturulan kodlar da tümüyle yani MainActivity.kt olarak metinler de yer alsın.

Gemini: ## 2. Hafta / 4. Ders: Döngü Kontrol Mekanizmaları ve Arayüz Akış Yönetimi (20:00 - 25:00)

Dersin bu bölümünde, listeler üzerinde döngü kurarken akışı dinamik olarak yönetmemizi sağlayan **break** ve **continue** komutlarının Jetpack Compose içerisindeki kullanımı ele alınacaktır. Bu komutlar, veriyi sadece taramakla kalmayıp, belirli şartlar altında taramayı durdurmak veya belirli verileri atlayarak arayüzü filtrelemek için kullanılır.

0.23.36. 1. Döngü Akışını Yönlendirmek

Bir listenin tamamını ekrana basmak her zaman verimli veya istenen bir durum değildir.

- **continue (Atla):** Döngünün o anki adımını iptal eder ve bir sonraki elemana geçer. Arayüz tarafında, belirli bir kritere uyan verinin (örneğin; "Pasif" durumdaki bir kullanıcının) ekranda hiç oluşturulmamasını sağlar.
- **break (Dur dur):** Döngüyü tamamen sonlandırır. Genellikle bir sınıra ulaşıldığında (örneğin; ekranda en fazla 3 duyuru gösterilmek istendiğinde) kullanılır.

0.23.37. 2. Compose İçerisinde Mantıksal Filtreleme

Jetpack Compose "declarative" (bildirimsel) bir yapı sunduğu için, `for` döngüsü içinde kullanılan bir `continue` komutu, o adımda hiçbir Composable fonksiyonunun çağrılmamasını sağlar. Bu da telefonun o veri için ekran üzerinde yer ayırmaması ve işlem gücü harcamaması anlamına gelir.

0.23.38. 3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kodda, bir "Sistem Log Kayıtları" paneli simüle edilmiştir. Kod; "Gizli" olarak işaretlenen kayıtları `continue` ile atlar ve "Maksimum Görünüm" sınırına ulaşıldığında `break` ile listelemeyi durdurur.

```
package com.example.hafta2ders4

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

// 1. Veri Yapısı
data class SistemKaydi(
    val mesaj: String,
    val seviye: String, // "Bilgi", "Hata", "Gizli"
    val id: Int
)

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
        setContent {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                // Döngü kontrol ekranını başlatır
                SistemKontrolPaneli()
            }
        }
    }
}

@Composable
fun SistemKontrolPaneli() {
    // 2. Veri Seti
    val kayitlar = listOf(
        SistemKaydi("Sistem başlatıldı", "Bilgi", 1),
        SistemKaydi("Veritabanı bağlantısı gizli", "Gizli", 2), // Atlanacak
        SistemKaydi("Hatalı giriş denemesi", "Hata", 3),
        SistemKaydi("Yeni kullanıcı kaydı", "Bilgi", 4),
        SistemKaydi("Sunucu aşırı yüklendi", "Hata", 5), // Buradan sonrası durdurulacak
        SistemKaydi("Yedekleme tamamlandı", "Bilgi", 6)
    )

    val gosterimSiniri = 4

    Column(modifier = Modifier.padding(24.dp)) {
        Text(
            text = "Gelişmiş Sistem Logları",
            fontSize = 20.sp,
            fontWeight = FontWeight.Bold,
            color = Color(0xFF455A64)
        )

        Divider(modifier = Modifier.padding(vertical = 12.dp))

        // 3. Kontrollü Döngü Yapısı
        var sayac = 0
        for (kayit in kayitlar) {

            // Koşul 1: Gizli kayıtları arayüzde gösterme (continue)
            if (kayit.seviye == "Gizli") {
                continue
            }
```

```

            // Koşul 2: Belirli bir sayıya ulaşıldığında listelemeyi kes (break)
            if (sayac >= gosterimSiniri) {
                Text(
                    text = "... Daha fazla kayıt gösterilemiyor (Sınır).",
                    fontSize = 12.sp,
                    color = Color.Gray,
                    modifier = Modifier.padding(top = 8.dp)
                )
                break
            }

            // Görselleştirme Katmanı
            Column(modifier = Modifier.padding(vertical = 6.dp)) {
                val renk = if (kayit.seviye == "Hata") Color.Red else Color.DarkGray

                Text(text = "ID: ${kayit.id} | ${kayit.seviye}", color = renk, fontSize = 12.sp)
                Text(text = kayit.mesaj, fontSize = 16.sp, fontWeight = FontWeight.Medium)

                Divider(modifier = Modifier.padding(top = 8.dp), thickness = 0.5.dp)
            }

            sayac++ // Sadece ekrana basılan kayıtları sayıyoruz
        }
    }
}
```

0.23.39. 4. Teknik Notlar ve Uygulama Detayları

- **Mantıksal Sayaç:** `sayac` değişkeni döngünün dışında tanımlanmıştır. `continue` ile atlanan elemanlarda bu sayaç artmaz; böylece ekranda tam olarak `gosterimSiniri` kadar eleman görünmesi garanti edilir.
- **Arayüz Performansı:** `break` komutu çalıştırıldığında, listenin geri kalan elemanları (örneğin 6. kayıt) işlemci tarafından hiç kontrol edilmez. Bu, binlerce verinin olduğu senaryolarda uygulamanın donmasını engeller.
- **Fiziksel Cihaz Testi:** Öğrenciler kendi telefonlarında uygulamayı çalıştırdıklarında, listede 6 eleman olmasına rağmen sadece 4 elemanın (ve “Gizli” olanın hariç tutularak) göründüğünü doğrulamalıdır.

- **Görsel Hiyerarşi:** `if` ve `when` yapıları ile `break/continue` komutlarının iç içe kullanımı, karmaşık iş kurallarının (business logic) arayüze nasıl yansıtılacağını somut bir şekilde gösterir.

Dersin bir sonraki 5 dakikalık bölümünde, Kotlin'in en kritik güvenlik özelliklerinden biri olan **Null Safety (Null Güvenliği)** konusunun bu listeler üzerindeki koruyucu etkisini incelememizi ister misiniz?

You: 2. haftanın 4. dersinin 25. dakikası ile 30. dakikası arasını samimi olmadan, kolay anlaşılır ve akıcı bir şekilde açıklayan bir ders notu hazırla. Ders notunda her detay olsun. Oluşturulan kodlar da tümüyle yani MainActivity.kt olarak metinler de yer alsın.

Gemini: ## 2. Hafta / 4. Ders: Null Güvenliği ve Hata Önleme Mekanizmaları (25:00 - 30:00)

Dersin bu bölümünde, modern Android uygulamalarının en büyük sorunu olan "NullPointerException" (boş veri hatası) facialarını önleyen **Kotlin Null Güvenliği (Null Safety)** konusu ele alınacaktır. Kotlin, bir değişkenin "boş" (null) olup olamayacağını henüz kod yazım aşamasında denetleyerek, uygulamanın kullanıcı elindeyken çökmesini engeller.

0.23.40. 1. Null (Boşluk) Kavramı ve Riskleri

Bir mobil uygulamada veriler her zaman eksiksiz gelmez. Örneğin; bir kullanıcının ikinci adı olmayabilir, bir profil fotoğrafı henüz yüklenmemiş olabilir veya internetten beklenen bir veri o an ulaşmamış olabilir. Bu tür "eksik veri" durumlarına programlamada `null` denir.

- **Sorun:** Eğer sistem, içerisinde veri olmayan (`null`) bir değişken üzerinde işlem yapmaya çalışırsa (Örn: Boş bir metnin uzunluğunu ölçmek), uygulama anında kapanır.
- **Çözüm:** Kotlin'de değişkenler varsayılan olarak `null` değer alamaz. Eğer bir değişkenin boş olabilme ihtimali varsa, tipi belirtilirken sonuna soru işareti (`?`) eklenmelidir.

0.23.41. 2. Null Veri Yönetim Operatörleri

Boş olabilen bir veriyle çalışırken uygulamanın çökmemesi için şu iki ana operatör kullanılır:

1. **Güvenli Çağrı Operatörü (`?.`):** "Eğer bu değişken null değilse işlemi yap, null ise hiçbir şey yapma ve çökme."
2. **Elvis Operatörü (`?:`):** "Eğer bu değişken null ise, onun yerine şu varsayılan (default) değeri kullan."

0.23.42. 3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod, bir öğrencinin kişisel bilgilerini ve henüz girilmemiş olabilecek "Adres" ve "Burs" bilgilerini yöneten tam bir Android uygulama örneğidir.

```
package com.example.nullguvenligi
```

```

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

// 1. Veri Yapısı: Nullable (Boş Olabilir) Alanlar
data class AkademikProfil(
    val isim: String,          // İsim boş olamaz
    val ikinciIsim: String?,   // İkinci isim olmayabilir
    val adres: String?,        // Adres girilmemiş olabilir
    val bursMiktari: Int?      // Burs alıyor veya almıyor olabilir
)

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                // Null güvenliği test ekranını başlatır
                NullGuvenlikEkrani()
            }
        }
    }
}

@Composable
fun NullGuvenlikEkrani() {
    // 2. Örnek Veri (İkinci ismi ve adresi olmayan bir öğrenci)
    val ogrenci = AkademikProfil(
        isim = "Hasan",
```

```
        ikinciIsim = null,
        adres = null,
        bursMiktari = 1500
    )

    Column(modifier = Modifier.padding(24.dp)) {
        Text(
            text = "Öğrenci Kayıt Detayları",
            fontSize = 20.sp,
            fontWeight = FontWeight.Bold,
            color = Color(0xFF1976D2)
        )

        Divider(modifier = Modifier.padding(vertical = 12.dp))

        // 3. Null Güvenliği Uygulamaları

        // A. Güvenli Yazdırma (İsim zorunlu olduğu için doğrudan yazılır)
        Text(text = "Adı: ${ogrenci.isim}", fontSize = 18.sp)

        // B. Elvis Operatörü (?:) ile Varsayılan Değer Atama
        // ikinciIsim null ise '-' karakteri basılır.
        val ekIsim = ogrenci.ikinciIsim ?: "-"
        Text(text = "İkinci Adı: $ekIsim")

        // C. Güvenli Çağrı (?.) ve Koşullu Stil
        // adres null olduğu için buradaki Text gri renkte ve uyarı mesajıyla görünür.
        val adresBilgisi = ogrenci.adres ?: "Adres bilgisi girilmemiştir."
        Text(
            text = "Adres: $adresBilgisi",
            color = if (ogrenci.adres == null) Color.Gray else Color.Black,
            fontSize = 14.sp,
            modifier = Modifier.padding(top = 8.dp)
        )

        Divider(modifier = Modifier.padding(vertical = 12.dp))

        // D. Sayısal Verilerde Null Kontrolü
        if (ogrenci.bursMiktari != null) {
            Text(
                text = "Burs Durumu: Aktif (${ogrenci.bursMiktari} TL)",
                color = Color(0xFF388E3C),
                fontWeight = FontWeight.Bold
            )
        } else {
```

```
            Text(text = "Burs Durumu: Alınmıyor", color = Color.Red)
        }
    }
}
```

0.23.43. 4. Teknik Notlar ve Uygulama Detayları

- **Neden `String?`?:** Kotlin'de bir tipin yanına `?` koymazsanız ve ona `null` atamaya çalışırsanız, kod daha derlenmeden hata verir. Bu, Android geliştiriciler için en büyük güvenlik kalkanıdır.
- **Elvis Operatörü Mantığı:** `ogrenci.ikinciIsim ?: "-"` ifadesi; "Eğer sol taraf null değilse onu kullan, null ise sağ taraftakini (`-`) kullan" anlamına gelir.
- **Fiziksel Cihaz Gözlemi:** Öğrenciler kendi telefonlarında uygulamayı çalıştırdıklarında, null olan alanların çökme yaratmak yerine nasıl düzenli bir şekilde "Girilmemiştir" veya "-" şeklinde göründüğünü doğrulamalıdır.
- **Uygulama Pratiği:** `ogrenci` nesnesindeki `adres` kısmına bir metin ekleyip uygulamayı tekrar çalıştırdıklarında, gri renkli uyarının otomatik olarak normal siyah metne dönüştüğünü görmeleri sağlanmalıdır.

Dersin bir sonraki 5 dakikalık bölümünde, 2. haftayı tamamlarken öğrendiğimiz tüm bu yapıları (Döngüler, Kararlar, Null Güvenliği) içeren kapsamlı bir **"Akademik Otomasyon Simülasyonu"** üzerinde çalışacağız. Bir sonraki aşamaya geçmemi ister misiniz?

You: 2. haftanın 4. dersinin 30. dakikası ile 35. dakikası arasını samimi olmadan, kolay anlaşılır ve akıcı bir şekilde açıklayan bir ders notu hazırla. Ders notunda her detay olsun. Oluşturulan kodlar da tümüyle yani MainActivity.kt olarak metinler de yer alsın.

Gemini: ## 2. Hafta / 4. Ders: Fonksiyonel Liste İşlemleri (filter ve map) (30:00 - 35:00)

Dersin bu bölümünde, Kotlin'in koleksiyonlar (listeler) üzerinde sunduğu ve `for` döngülerini çok daha kısa ve okunabilir hale getiren **Fonksiyonel Operatörler** (filter ve map) ele alınacaktır. Jetpack Compose mimarisinde veriyi arayüze göndermeden önce ayıklamak ve dönüştürmek için bu yöntemler standart kabul edilir.

0.23.44. 1. Operatörlerin Çalışma Mantığı

Klasik yöntem olan "döngü kurup içine if yazmak" yerine, Kotlin bize veriyi bir boru hattından (pipeline) geçiriyormuşuz gibi işleme imkanı tanır:

- **filter { ... }:** Bir listenin içinden sadece belirli bir şartı sağlayan elemanları seçip yeni bir liste oluşturur.
- **map { ... }:** Bir listenin her bir elemanını alır, üzerinde bir işlem yapar ve sonuçlardan oluşan yeni bir liste üretir (Örn: Sayıları alıp karelerinden oluşan bir liste yapmak).

Operatör	Görevi	Sonuç
filter	Ayıklama (Seçme)	Eleman sayısı azalabilir, tipi aynı kalır.
map	Dönüştürme	Eleman sayısı aynı kalır, tipi değişebilir.

Table 1

E-Tablolar'a aktar—

0.23.45. 2. Arayüzde Fonksiyonel Yaklaşım

Jetpack Compose içerisinde bu operatörler, `@Composable` fonksiyonların gövdesini temiz tutmak için kullanılır. Veri listesini doğrudan döngüye sokmak yerine, önce `.filter` ile ayıklayıp ardından ekrana basmak, kodun bakımını kolaylaştırır ve hata payını minimize eder.

0.23.46. 3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kodda, bir "Bölüm Görev Takip Sistemi" simülasyonu yer almaktadır. Sistemdeki görevler arasından sadece "Tamamlanmamış" olanlar `.filter` ile seçilmekte ve bu görevlerin başlıkları `.map` ile büyük harfe dönüştürülmektedir.

```
package com.example.fonksiyonelislemler

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

// 1. Veri Yapısı
data class Gorev(
    val id: Int,
    val tanim: String,
    val tamamlandiMi: Boolean
)

```

```
27 class MainActivity : ComponentActivity() {
28     override fun onCreate(savedInstanceState: Bundle?) {
29         super.onCreate(savedInstanceState)
30         setContent {
31             Surface(
32                 modifier = Modifier.fillMaxSize(),
33                 color = MaterialTheme.colorScheme.background
34             ) {
35                 // Görev listesi ekranını başlatır
36                 GorevYonetimEkrani()
37             }
38         }
39     }
40 }
41
42 @Composable
43 fun GorevYonetimEkrani() {
44     // 2. Veri Seti (Ham Liste)
45     val tumGorevler = listOf(
46         Gorev(1, "Müfredat Belirleme", false),
47         Gorev(2, "Sınav Takvimi Hazırlama", true),
48         Gorev(3, "Bölüm Kurulu Toplantısı", false),
49         Gorev(4, "Öğrenci Danışmanlıkları", false),
50         Gorev(5, "Yayın Revizyonu", true)
51     )
52
53     // 3. Fonksiyonel İşlemler (Mantık Katmanı)
54     // Sadece tamamlanmamış (false) olanları filtreliyoruz
55     val bekleyenGorevler = tumGorevler.filter { it.tamamlandiMi == false }
56
57     // Görev tanımlarını daha dikkat çekici olması için büyük harfe çeviriyoruz
58     val vurgulananGorevler = bekleyenGorevler.map { it.tanim.uppercase() }
59
60     Column(modifier = Modifier.padding(24.dp)) {
61         Text(
62             text = "Aktif Bekleyen Görevler",
63             fontSize = 20.sp,
64             fontWeight = FontWeight.Bold,
65             color = Color(0xFFD32F2F)
66         )
67         Text(
68             text = "Toplam ${vurgulananGorevler.size} adet tamamlanmamış işlem
               bulundu.",
69             fontSize = 14.sp,
70             modifier = Modifier.padding(bottom = 16.dp)
71         )
```

```

        Divider(modifier = Modifier.padding(bottom = 12.dp))

        // 4. İşlenmiş Verinin Arayüze Basılması
        // Burada 'vurgulananGorevler' artık sadece String listesidir
        for (gorevMetni in vurgulananGorevler) {
            Column(modifier = Modifier.padding(vertical = 8.dp)) {
                Text(
                    text = "• $gorevMetni",
                    fontSize = 16.sp,
                    fontWeight = FontWeight.Medium,
                    color = Color.Black
                )
                Divider(
                    modifier = Modifier.padding(top = 8.dp),
                    thickness = 0.5.dp,
                    color = Color.LightGray
                )
            }
        }
    }
}
```

0.23.47. 4. Teknik Notlar ve Uygulama Detayları

- **`it` Anahtar Kelimesi:** Lambda fonksiyonu içinde listenin o anki elemanını temsil eder. `it.tamamlandiMi` yazımı, "listenin içindeki her bir görev nesnesinin 'tamamlandiMi' özelliğine bak" anlamına gelir.
- **Zincirleme Kullanım:** Kotlin'de bu operatörler uç uca eklenebilir. Örneğin: `val sonuc = liste.filter { ... }.map { ... }.take(3)` şeklinde bir yazımla; veriyi filtreleyip, dönüştürüp, ilk 3 elemanını almak tek satırda mümkündür.
- **Fiziksel Cihaz Gözlemi:** Öğrenciler kendi telefonlarında uygulamayı çalıştırdıklarında, `tumGorevler` listesinde 5 eleman olmasına rağmen ekranda sadece 3 elemanın ve hepsinin büyük harfle yazılmış olarak göründüğünü doğrulamalıdır.
- **Verimlilik:** Bu operatörler arka planda optimize edilmiş döngüler kullanır. Binlerce veri içeren listelerde, manuel yazılan `for` döngülerine göre çok daha güvenli ve okunaklı bir yapı sunarlar.

Dersin bir sonraki 5 dakikalık bölümünde, bu operatörlerin en gelişmiş hali olan ve koleksiyonları tek bir değere (Örn: Toplam bütçe veya genel başarı puanı) indirgeyen **Reduce** ve **Fold** kavramlarını incelememizi ister misiniz?

You: 2. haftanın 4. dersinin 35. dakikası ile 40. dakikası arasını samimi olmadan, kolay anlaşılır ve akıcı bir şekilde açıklayan bir ders notu hazırla. Ders notunda her detay olsun. Oluşturulan kodlar da tümüyle yani MainActivity.kt olarak metinler de yer alsın.

Gemini: ## 2. Hafta / 4. Ders: Veri Toplulaştırma (Aggregate) ve Sıralama İşlemleri (35:00 - 40:00)

Dersin bu bölümünde, koleksiyonlar (listeler) içindeki verileri analiz ederek tek bir sonuç üretmemizi sağlayan **Toplulaştırma (Aggregate)** fonksiyonları ile verileri belirli bir düzene sokan **Sıralama (Sorting)** işlemleri ele alınacaktır. Bu teknikler, bir sınıftaki not ortalamasını hesaplamak veya öğrencileri en yüksek puandan en düşüğe doğru sıralamak gibi gerçek dünya senaryolarının temelini oluşturur.

0.23.48. 1. Veri Toplulaştırma Fonksiyonları

Bir listedeki tüm sayıları toplamak veya ortalamasını almak için manuel döngüler kurmak yerine Kotlin'in hazır fonksiyonlarını kullanırız:

- **sumOf { ... }:** Listedeki elemanların belirli bir özelliğini (örneğin fiyat veya not) toplar.
- **average():** Sayısal bir listenin aritmetik ortalamasını hesaplar.
- **maxByOrNull / minByOrNull:** Listedeki en büyük veya en küçük değere sahip olan nesneyi bulur.

0.23.49. 2. Sıralama (Sorting) Mekanizması

Verileri arayüzde karmaşık bir şekilde göstermek kullanıcı deneyimini olumsuz etkiler. Kotlin, listeleri kolayca hizalamak için iki temel fonksiyon sunar:

- **sortedBy { ... }:** Belirlenen kritere göre küçükten büyüğe (A'dan Z'ye) sıralar.
- **sortedByDescending { ... }:** Belirlenen kritere göre büyükten küçüğe (Z'den A'ya) sıralar.

0.23.50. 3. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod, bir "Bölüm Başarı Takip Paneli" simülasyonudur. Veriler önce sıralanmakta, ardından tüm sınıfın not ortalaması hesaplanarak arayüze dinamik olarak yansıtılmaktadır.

```
package com.example.verianalizi

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
```

```
11  import androidx.compose.material3.Surface
12  import androidx.compose.material3.Text
13  import androidx.compose.runtime.Composable
14  import androidx.compose.ui.Modifier
15  import androidx.compose.ui.graphics.Color
16  import androidx.compose.ui.text.font.FontWeight
17  import androidx.compose.ui.unit.dp
18  import androidx.compose.ui.unit.sp
19
20  // 1. Veri Yapısı
21  data class SinavSonucu(
22      val ogrenciAd: String,
23      val puan: Int
24  )
25
26  class MainActivity : ComponentActivity() {
27      override fun onCreate(savedInstanceState: Bundle?) {
28          super.onCreate(savedInstanceState)
29          setContent {
30              Surface(
31                  modifier = Modifier.fillMaxSize(),
32                  color = MaterialTheme.colorScheme.background
33              ) {
34                  // Akademik analiz ekranını başlatır
35                  AkademikAnalizEkrani()
36              }
37          }
38      }
39  }
40
41  @Composable
42  fun AkademikAnalizEkrani() {
43      // 2. Ham Veri Listesi
44      val sonuclar = listOf(
45          SinavSonucu("Hasan", 85),
46          SinavSonucu("Ayşe", 92),
47          SinavSonucu("Mehmet", 45),
48          SinavSonucu("Selin", 78),
49          SinavSonucu("Ali", 30)
50      )
51
52      // 3. Veri İşleme (Mantık Katmanı)
53      // Öğrencileri puana göre yüksekten düşüğe sıralıyoruz
54      val siraliListe = sonuclar.sortedByDescending { it.puan }
55
56      // Sınıf ortalamasını hesaplıyoruz
```

```
    val toplamPuan = sonuclar.sumOf { it.puan }
    val ortalama = if (sonuclar.isNotEmpty()) toplamPuan.toDouble() / sonuclar.size else 0.0

    // En yüksek notu alan öğrenciyi buluyoruz
    val enBasariliOgrenci = sonuclar.maxByOrNull { it.puan }

    Column(modifier = Modifier.padding(24.dp)) {
        Text(
            text = "Akademik Performans Analizi",
            fontSize = 20.sp,
            fontWeight = FontWeight.Bold,
            color = Color(0xFF1976D2)
        )

        // Özet İstatistikler
        Text(
            text = "Sınıf Ortalaması: ${"%.2f".format(ortalama)}",
            modifier = Modifier.padding(top = 8.dp),
            fontSize = 16.sp,
            fontWeight = FontWeight.Medium
        )

        enBasariliOgrenci?.let {
            Text(
                text = "Dönem Birincisi: ${it.ogrenciAd} (${it.puan})",
                color = Color(0xFF388E3C),
                fontSize = 14.sp
            )
        }

        Divider(modifier = Modifier.padding(vertical = 16.dp), thickness = 2.dp)

        Text(text = "Sıralı Öğrenci Listesi:", fontWeight = FontWeight.Bold)

        // 4. İşlenmiş ve Sıralanmış Verinin Döngü ile Basılması
        for (sonuc in siraliListe) {
            Column(modifier = Modifier.padding(vertical = 4.dp)) {
                val renk = if (sonuc.puan < 50) Color.Red else Color.Black

                Text(
                    text = "${sonuc.ogrenciAd}: ${sonuc.puan}",
                    color = renk,
                    fontSize = 16.sp
                )
```

```
                    Divider(modifier = Modifier.padding(top = 4.dp), thickness = 0.5.dp, color = Color.LightGray)
            }
        }
    }
}
```

0.23.51. 4. Teknik Notlar ve Uygulama Detayları

- **Null Güvenliği (`maxByOrNull`):** Liste boş olduğunda uygulamanın çökmemesi için Kotlin bu fonksiyonun sonuna "OrNull" ekler. Kullanırken `?.let` veya `if (null != ...)` kontrolü yapmak güvenli kod yazımı (safe coding) için şarttır.
- **Formatlı Yazım:** `ortalama` değerini ekrana basarken kullanılan `"%.2f".format(ortalama)` yapısı, virgülden sonra sadece 2 basamak gösterilmesini sağlayarak arayüzün daha profesyonel görünmesini sağlar.
- **Performans:** `.sortedByDescending` fonksiyonu listenin kopyasını oluşturur ve orijinal listeyi bozmaz. Bu sayede verinin orijinal halini başka bir işlem için (örneğin kayıt sırasına göre göstermek) saklamaya devam edebilirsiniz.
- **Cihaz Üzerinde Gözlem:** Öğrenciler kendi telefonlarında uygulamayı çalıştırdıklarında, listedeki puanları değiştirip uygulamayı tekrar başlattıklarında listenin sırasının ve ortalama değerinin nasıl otomatik olarak yeniden hesaplandığını test etmelidir.

1. haftanın bu son dersiyle birlikte Kotlin'in veri işleme yeteneklerini tamamlamış olduk. 3. haftada, bu verileri içinde barındıracak olan **Kullanıcı Arayüzü (UI) Tasarımı** detaylarına giriş yapacağız.

Bir sonraki ders olan **"3. Hafta: Layout ve Modifier Temelleri"** için bir ön hazırlık planı oluşturmamı ister misiniz?

You: 2. haftanın 4. dersinin 40. dakikası ile 45. dakikası arasını samimi olmadan, kolay anlaşılır ve akıcı bir şekilde açıklayan bir ders notu hazırla. Ders notunda her detay olsun. Oluşturulan kodlar da tümüyle yani MainActivity.kt olarak metinler de yer alsın.

Gemini: ## 2. Hafta / 4. Ders: Kotlin ve Compose Sentezi - Akıllı Veri Yönetimi (40:00 - 45:00)

Dersin bu son 5 dakikasında, hafta boyunca öğrendiğimiz tüm temel Kotlin kavramlarını (Değişkenler, Karar Yapıları, Döngüler, Null Güvenliği ve Fonksiyonel Operatörler) tek bir potada eritiyoruz. Bu bölüm, ham verinin bir "mantık filtresinden" geçirilerek kullanıcı arayüzüne (UI) nasıl profesyonelce yansıtıldığının final özetidir.

0.23.52. 1. Mantıksal Akış Şeması: Veriden Görsele

Bir Android uygulamasında verinin ekrana yolculuğu şu aşamaları takip eder:

1. **Modelleme:** `data class` ile verinin anatomisi çıkarılır.

2. **Temizlik ve Güvenlik:** Null değerler (`?.`, `?:`) kontrol altına alınır.
3. **İşleme:** `.filter` ve `.map` gibi fonksiyonel operatörlerle veri rafine edilir.
4. **Sunum:** İşlenmiş veri, bir döngü aracılığıyla `@Composable` bileşenlere dağıtılır.

0.23.53. 2. Kod Uygulaması: MainActivity.kt

Aşağıdaki kod, 2. haftanın tüm teknik kazanımlarını içeren kapsamlı bir **"Akademik Kaynak ve Yayın Takip"** simülasyonudur. Bu yapı, gerçek bir projede karşılaşabileceğiniz veri karmaşıklığını ve çözüm yollarını simüle eder.

```
package com.example.hafta2_final_sentez

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Divider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

// 1. Veri Yapısı (Modelleme)
data class AkademikYayin(
    val baslik: String,
    val yazar: String?, // Null olabilir
    val yil: Int,
    val aktifMi: Boolean
)

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colorScheme.background
            ) {
                // Sentez ekranını başlatıyoruz
```

```
                    HaftalikOzetEkrani()
                }
            }
        }
}

@Composable
fun HaftalikOzetEkrani() {
    // 2. Ham Veri Listesi (Koleksiyon)
    val yayinlar = listOf(
        AkademikYayin("Yapay Zeka Temelleri", "Ali Can", 2024, true),
        AkademikYayin("Mobil Programlama Notları", null, 2023, true), // Yazar null
        AkademikYayin("Eski Müfredat Arşivi", "Veli Demir", 2015, false), // Aktif değil
        AkademikYayin("Kotlin ile Android", "Ayşe Kaya", 2025, true)
    )

    // 3. Veri İşleme (Mantık Katmanı)
    // Sadece aktif olanları filtrele ve yıla göre büyükten küçüğe sırala
    val gosterilecekYayinlar = yayinlar
        .filter { it.aktifMi }
        .sortedByDescending { it.yil }

    Column(modifier = Modifier.padding(24.dp)) {
        Text(
            text = "Haftalık Veri İşleme Özeti",
            fontSize = 22.sp,
            fontWeight = FontWeight.Bold,
            color = Color(0xFF1976D2)
        )
        Text(
            text = "Rafine edilmiş veri listesi aşağıdadır:",
            fontSize = 14.sp,
            color = Color.Gray,
            modifier = Modifier.padding(bottom = 16.dp)
        )

        Divider(thickness = 2.dp, modifier = Modifier.padding(bottom = 12.dp))

        // 4. Döngü ve Koşul Entegrasyonu
        for (yayin in gosterilecekYayinlar) {

            Column(modifier = Modifier.padding(vertical = 8.dp)) {
                // Başlık Yazımı
```

```
                    Text(text = yayin.baslik, fontSize = 18.sp, fontWeight = FontWeight.Medium)

                    // Null Güvenliği ve Karar Yapısı
                    // Yazar bilgisi yoksa Elvis operatörü ile varsayılan değer atanır
                    val yazarBilgisi = yayin.yazar ?: "Yazar Belirtilmemiş"

                    // when yapısı ile yayın yılına göre "Yeni/Eski" etiketi belirleme
                    val etiket = when (yayin.yil) {
                        2025 -> "GÜNCEL"
                        2024 -> "YENİ"
                        else -> "STANDART"
                    }

                    Text(
                        text = "Yazar: $yazarBilgisi | Durum: $etiket",
                        color = if (etiket == "GÜNCEL") Color(0xFF388E3C) else Color.DarkGray,
                        fontSize = 14.sp,
                        fontWeight = FontWeight.Bold
                    )

                    Divider(modifier = Modifier.padding(top = 8.dp), thickness = 0.5.dp)
                }
            }
        }
}
```

0.23.54. 3. Kritik Teknik Analiz

Özellik	Kod İçindeki Karşılığı	Amacı
Filtreleme	`.filter { it.aktifMi }`	Gereksiz veriyi arayüzden temizlemek.
Sıralama	`.sortedByDescending`	Kullanıcıya anlamlı bir düzen sunmak.
Null Güvenliği	`yayin.yazar ?: "..."`	Boş veri nedeniyle uygulamanın çökmesini önlemek.
Koşullu Stil	`if (etiket == "GÜNCEL")`	Önemli bilgiyi görsel olarak vurgulamak.

Table 2

E-Tablolar'a aktar—

0.23.55. 4. Uygulama ve Test Notları

- **Fiziksel Cihaz Gözlemi:** Öğrenciler kendi telefonlarında bu kodu çalıştırdıklarında, listedeki "Eski Müfredat Arşivi" isimli yayının ekranda hiç görünmediğini (çünkü `aktifMi = false`) bizzat gözlemlemelidir.
- **Hata Ayıklama (Debug):** Eğer bir veri görünmüyorsa, `filter` bloğunu geçici olarak yorum satırı yaparak tüm listenin dökülüp dökülmediği kontrol edilebilir.
- **Esneklik:** Bu yapı sayesinde, yarın listede 1000 adet yayın olsa dahi yazdığımız 5-6 satırlık mantık kodu saniyeler içinde hepsini işleyip düzenli bir liste oluşturacaktır.

1. haftayı bu sentez ile başarıyla tamamladık. Bu temel, 3. haftada işleyeceğimiz daha karmaşık arayüz bileşenleri ve kullanıcıdan veri alma (State) konuları için sağlam bir zemin oluşturacaktır.
2. haftanın planlamasına geçmeden önce, bu sentez yapısındaki herhangi bir bölümü (örneğin `filter` ve `map` zincirini) daha detaylı incelemek ister misiniz?

pandoc version 3.9